CSE-307
OPERATING SYSTEM

Virtualization, Virtual Machines and Cloud Computing

Detailed Student Notes, Working Mechanisms, Numerical Examples, Comparisons, Industry Use Cases and Scenario-Based Questions

Cloud Services and Applications
Virtual Machines / Containers / Managed Platforms
Hypervisor and Cloud Management Layer
Physical Servers, Storage and Networking

Prepared by
Lec Khaled Hasan
Department of Computer Science and Engineering
Military Institute of Science and Technology

Purpose: to expand the brief lecture slides into a complete, student-friendly study resource.

1. Virtualization: Definition and Core Idea

Virtualization makes one physical computer appear as several independent computing environments.

Definition

Virtualization is the technique of abstracting physical computing resources such as CPU, memory, storage and networking and presenting them as one or more logical or virtual resources. A virtual machine is one complete virtual execution environment created through this abstraction.

Real-life example

Think of a large apartment building. The building has one physical structure, one water connection and one electricity connection, but it is divided into separate apartments. Each family experiences a private space even though the infrastructure is shared. Virtualization similarly divides one physical computer into isolated virtual computers.

Technical example

A Windows laptop with 16 GB RAM and an 8-core CPU runs VMware Workstation. Inside VMware, one Ubuntu VM is assigned 4 GB RAM and 2 virtual CPUs. Windows remains the host operating system, Ubuntu is the guest operating system, and VMware is the hypervisor.

Working mechanism

1. The physical machine provides real CPU cores, RAM, storage and network interfaces.

2. The hypervisor creates virtual versions of these resources: vCPU, vRAM, virtual disk and virtual network adapter.

3. The guest operating system boots as if it were running on a real computer.

4. When the guest requests CPU time or I/O, the hypervisor maps that request to the real hardware.

5. The hypervisor schedules and isolates multiple VMs so that one VM cannot normally access another VM's memory or disk.

Why it matters

Virtualization improves hardware utilization, supports safe experimentation, allows different operating systems to run together, simplifies backup and recovery, and forms a major technical foundation of modern data centers and cloud computing.

Quick comparison: physical machine vs virtual machine

Aspect	Physical machine	Virtual machine
Hardware	Uses dedicated physical hardware directly	Uses virtual hardware mapped to shared physical hardware
Operating system	Installed directly on the machine or hypervisor	Installed inside an isolated virtual environment
Mobility	Harder to move because it is tied to hardware	VM files can be copied, cloned or migrated
Failure scope	Hardware failure affects the whole machine	Guest failure is usually isolated, but host failure may affect many VMs

Performance	Highest direct performance	Small overhead may exist due to virtualization

V1 - Scenario question: A university has one powerful server but needs 20 isolated Linux environments for 20 student groups. Why is virtualization more suitable than buying 20 separate computers?

V2 - Scenario question: A student says, “A VM is only a fake screen.” Explain why the statement is incorrect.

2. The Virtualization Triad: Safety, Fidelity and Efficiency

A useful virtualization system should isolate guests, behave like real hardware and avoid unnecessary overhead.

Safety: The hypervisor must maintain strong control over physical resources. A guest should not directly escape its assigned environment or interfere with other guests.

Fidelity: Software inside the VM should observe an environment that is sufficiently equivalent to real hardware. Most normal guest instructions should behave as expected.

Efficiency: Most guest instructions should execute efficiently, ideally directly on the CPU with hardware virtualization support, while sensitive operations are controlled by the hypervisor.

Real-life example

A hotel must satisfy three goals: guests receive private rooms (safety), each room has the expected facilities (fidelity), and hotel staff do not manually approve every small action such as turning on a light (efficiency).

Technical example

Intel VT-x and AMD-V provide processor support that lets guest operating systems execute many instructions directly while trapping privileged operations to the hypervisor. This improves efficiency while preserving control.

Trade-off

Maximum isolation may require additional checks, which can add overhead. Aggressive performance optimization can increase complexity. Practical hypervisors balance isolation, compatibility and speed rather than maximizing only one property.

Property	What can go wrong if it is weak?	Example control
Safety	A compromised VM accesses another VM or the host	Memory isolation, privilege levels, IOMMU, access control
Fidelity	Guest software behaves differently or fails	Accurate virtual devices and CPU behavior
Efficiency	VM becomes too slow for practical use	Hardware-assisted virtualization and optimized device drivers

T1 - Scenario question: A hypervisor runs applications correctly but allows one VM to read another VM's memory. Which virtualization requirement has failed?

T2 - Scenario question: A guest OS works correctly, but every instruction is emulated slowly in software. Which requirement is weakest?

3. Anatomy of a Virtual Machine

A VM is a complete software-defined computer containing virtual hardware and usually a guest operating system.

Applications inside the guest
Guest operating system
Virtual hardware: vCPU, vRAM, vDisk, vNIC
Hypervisor / Virtual Machine Monitor
Physical host: CPU, RAM, storage, network

Main components

Component	Meaning	Example
Host	The physical computer and, for Type 2, its main operating system	A Windows laptop running VMware Workstation
Guest	The OS or workload inside the VM	Ubuntu 24.04 running inside the VM
Hypervisor / VMM	Software layer that creates, runs and controls VMs	VMware ESXi, Hyper-V, KVM, VMware Workstation

vCPU	A virtual processor scheduled onto physical CPU cores	A VM configured with 2 vCPUs
vRAM	Memory assigned to the VM and mapped to host memory	4 GB assigned to Ubuntu
Virtual disk	A file or logical volume presented as a disk	A 50 GB VMDK or VHDX
vNIC	A virtual network adapter connected to a virtual switch	NAT, bridged or host-only networking

Real-life example

A furnished rental apartment provides rooms, electricity points and water taps to the tenant. The tenant uses these facilities without owning the building. A guest OS similarly uses virtual CPU, memory, disk and network devices without owning the physical host.

Significance

Understanding these layers is essential for troubleshooting. A network problem may exist in the guest configuration, the virtual switch, the host adapter or the physical network. A performance problem may be caused by insufficient vRAM, CPU contention, slow host storage or another VM on the same host.

A1 - Scenario question: A VM can access local files but not the internet. List three layers where the problem may exist.

A2 - Scenario question: A guest OS shows 4 GB RAM although the host has 16 GB. Why does the guest not see all 16 GB?

4. How a Virtual Machine Works

The guest behaves as if it owns a computer, while the hypervisor translates and schedules its resource requests.

VM creation and boot process

1. The administrator creates a VM configuration and selects virtual CPU count, memory size, disk size, firmware and network mode.
2. The hypervisor allocates metadata and creates or attaches the virtual disk.
3. When the VM is powered on, virtual firmware such as BIOS or UEFI begins execution.
4. The guest bootloader loads the guest operating system from the virtual disk.
5. The guest OS detects virtual devices and loads drivers for them.
6. The hypervisor schedules vCPUs on physical cores, maps guest memory to host memory, and forwards virtual I/O to host devices.
7. The VM runs until it is shut down, suspended, reset, migrated or deleted.

CPU virtualization

A vCPU is not a permanently dedicated physical core in most systems. It is a schedulable execution context. The hypervisor gives different vCPUs time on physical cores. With hardware-assisted virtualization, ordinary instructions can run directly while privileged operations transfer control to the hypervisor.

Memory virtualization

The guest uses guest virtual addresses, which the guest OS translates to guest physical addresses. The hypervisor and hardware then map guest physical addresses to actual host physical memory. This extra mapping allows isolation and flexible placement.

I/O virtualization

The guest may see an emulated device, a high-performance paravirtual device, or a directly assigned physical device. Emulation improves compatibility; paravirtual drivers improve performance; direct assignment offers high speed but reduces mobility and sharing.

Important distinction

Virtualization is not the same as pure simulation. Modern VMs usually execute a large portion of guest code directly on the real CPU using hardware support. The hypervisor mainly controls privileged actions, scheduling and device access.

W1 - Scenario question: A VM has four vCPUs on a host with two physical cores. Can all four vCPUs execute at the exact same instant? Explain.

W2 - Scenario question: Why can a VM continue to run the same guest OS after being moved to a different compatible physical server?

5. Resource Allocation and Physical Constraints

Virtual resources are flexible abstractions, but every active VM ultimately consumes finite physical resources.

Key principle

Virtualization does not create unlimited hardware. It allows physical resources to be divided, scheduled and sometimes overcommitted. If total demand becomes greater than the real capacity, performance falls or allocation fails.

Numerical example: RAM allocation

A host has 16 GB RAM. The host OS and hypervisor need 4 GB, leaving approximately 12 GB for VMs. VM-A receives 4 GB, VM-B receives 4 GB and VM-C receives 2 GB. Total assigned active memory is

10 GB, so about 2 GB remains for additional demand and overhead. Adding another 4 GB VM would require overcommitment or would fail, depending on the platform and policy.

Numerical example: CPU overcommitment

An 8-core host runs four VMs with 4 vCPUs each. Total vCPUs = 16. CPU overcommit ratio = 16 / 8 = 2:1. This may work well when the VMs are mostly idle, but if all VMs continuously use 100% CPU, each vCPU cannot receive full physical-core performance.

What happens when the host is overloaded?

- CPU ready time increases because vCPUs wait for physical CPU time.
- Memory pressure may cause ballooning, compression or swapping, which can be very slow.
- Storage latency increases when many VMs perform I/O together.
- Network throughput is shared and packet delay may rise.
- A noisy neighbor VM may reduce the performance of other VMs.

Pros and cons of overcommitment

	Advantages	Risks
CPU	Improves utilization because workloads rarely peak together	Severe contention when many workloads peak simultaneously
Memory	Allows more VMs if actual usage is below assigned memory	Swapping can cause dramatic slowdown or VM failure
Storage	Supports flexible growth with thin disks	Datastore may run out of real capacity
Network	Many VMs share high-speed links efficiently	Congestion and unpredictable latency

R1 - Scenario question: A 32 GB server reserves 6 GB for the host. Four active VMs need 8 GB each. Is there enough physical memory without overcommitment? Show the calculation.

R2 - Scenario question: A lab host is fast in the morning but slow when every student starts a VM. What resource-contention causes should be investigated?

6. Thin and Thick Provisioning

Provisioning determines whether a virtual disk consumes its maximum capacity immediately or grows as data is written.

Definitions

Thin provisioning: The virtual disk advertises a maximum size, but physical storage is allocated gradually as the guest writes data.

Thick provisioning: The full selected virtual disk capacity is reserved on physical storage when the disk is created, although implementation details may vary.

Real-life example

A restaurant can reserve an entire table for each possible customer in advance, similar to thick provisioning. Alternatively, it can arrange tables only as customers arrive, similar to thin provisioning. The second approach uses space efficiently but can fail if too many customers arrive together.

Numerical example (not needed for 307 course; only basic concepts about thick and thin provisioning and their use cases examples)

Eight VMs are each configured with a 100 GB virtual disk. Thick provisioning may reserve 800 GB immediately. With thin provisioning, if each VM currently stores only 20 GB, actual use is about 160 GB plus metadata. On a 500 GB datastore, the provisioned-to-physical ratio is 800/500 = 1.6:1. If every VM later grows to 70 GB, total demand becomes 560 GB, which exceeds the 500 GB datastore.

Aspect	Thin provisioning	Thick provisioning
Initial space use	Low; grows with written data	High; full capacity reserved
Creation speed	Usually faster	May be slower when space is initialized
Capacity efficiency	High	Lower when much reserved space remains unused
Risk	A physical datastore can become full unexpectedly	Less risk of capacity surprise
Typical use	Labs, development, general-purpose workloads	Predictable performance or strict capacity control

Operational rule

Thin provisioning must be combined with monitoring, alerts and capacity planning. A 100 GB thin disk does not mean the guest can always safely grow to 100 GB unless enough physical capacity will remain available.

P1 - Scenario question: A host has 300 GB free storage. Six thin-provisioned VMs each have a 100 GB maximum disk but currently use 30 GB. Is the current state safe? What future risk exists?

P2 - Scenario question: Which provisioning method would you choose for a classroom lab and which for a latency-sensitive production database? Justify each answer.

7. Benefits, Limitations and Real-World VM Use Cases

Virtual machines offer isolation and flexibility, but they also introduce overhead, complexity and concentration of risk.

Major advantages

- Server consolidation: Several lightly used workloads can share one powerful host.
- Isolation: a failure or software conflict in one guest is usually contained.
- Portability: a VM can be represented by configuration and disk files and moved between compatible hosts.
- Rapid provisioning: templates and clones can create systems quickly.
- Testing and learning: students can safely install software, modify settings and take snapshots.
- Legacy support: old operating systems and applications can run in controlled environments.
- Disaster recovery: VM images and backups can be restored on replacement hardware.
- Security sandboxing: suspicious or untrusted workloads can be placed in an isolated environment.

Major disadvantages

- Performance overhead compared with direct bare-metal execution, especially for some I/O workloads.
- A host failure can affect many VMs at once unless high-availability design is used.
- Resource contention and noisy-neighbor effects can make performance less predictable.
- VM sprawl occurs when unused VMs are created and never removed.
- Licensing, patching, backup and monitoring remain necessary for every guest OS.
- Misconfiguration can weaken isolation or expose management interfaces.
- Snapshots consume storage and can reduce performance when retained for too long.

Common use cases

Use case	Why a VM is appropriate	Example
Education and labs	Safe reset, snapshots and different OS environments	Ubuntu VM on a Windows laptop
Software testing	Repeatable isolated test systems	Test an application on Windows and Linux
Legacy application	Preserves an older OS environment	Old accounting software in an offline VM
Server consolidation	Raises utilization and reduces hardware count	Web, DNS and monitoring servers on one host
Virtual desktop	Users access managed desktops remotely	Cloud PC or VDI environment
Security analysis	Contains risky files and tools	Malware analysis in a disposable isolated lab

Cloud IaaS	Provides on-demand server instances	EC2, Azure VMs, Google Compute Engine

U1 - Scenario question: A company wants to keep a 15-year-old application that only runs on an old OS. What VM benefits and risks should the company consider?

U2 - Scenario question: Why is “one large host with 50 VMs” not automatically safer than 50 separate physical machines?

8. VM, Dual Boot, Bare Metal and Containers

These approaches solve different problems and should not be treated as interchangeable.

Dual boot

Dual boot installs two or more operating systems on separate partitions. Only one operating system normally runs at a time, and changing systems requires a reboot. The active OS gets direct access to the physical hardware.

Bare metal

A bare-metal installation runs an operating system or application directly on physical hardware without a host OS underneath. It is chosen when maximum performance, direct device access or deterministic latency is important.

Container

A container packages an application and its user-space dependencies but shares the host operating system kernel. Containers are usually smaller and start faster than VMs, while VMs provide a full guest operating system and generally stronger kernel-level separation.

Feature	Bare metal	Dual boot	Virtual machine	Container
Concurrent OS environments	No	No, normally one active OS	Yes	Shares one host kernel
Reboot to switch	Not applicable	Yes	No	No
Guest OS included	Main OS only	Each installed OS	Yes	No full guest kernel
Isolation	Physical separation	Partition and time separation	Strong virtual hardware boundary	Process-level isolation
Startup time	Normal boot	Normal boot	Seconds to minutes	Usually seconds or less
Performance	Highest	Near native	Near native with some overhead	Very low overhead

Best for	High-performance dedicated work	Personal use of two native OSes	Labs, servers, legacy apps, cloud IaaS	Microservices and portable application deployment

Decision guide

- Choose dual boot when you need native hardware performance from either OS but do not need both simultaneously.
- Choose a VM when you need multiple OSes at once, snapshots, portability and safe experimentation.
- Choose bare metal for specialized performance, direct accelerator access or tightly controlled production systems.
- Choose containers for lightweight application packaging when sharing the host kernel is acceptable.
- In industry, containers frequently run inside VMs, combining VM isolation with container deployment speed.

C1 - Scenario question: A developer needs to test the same web application on Ubuntu and Windows while keeping both environments active. Which approach is most suitable?

C2 - Scenario question: A high-performance game needs direct GPU access and maximum frame rate. Compare dual boot and a desktop VM for this requirement.

9. Hypervisors and Their Role

The hypervisor is the control layer that creates virtual hardware, schedules resources and enforces isolation.

Definition

A hypervisor, also called a Virtual Machine Monitor or VMM, is software that creates and manages virtual machines. It presents virtual hardware to guests while retaining control of the physical machine.

Core responsibilities

- Create, start, stop, suspend, reset and delete VMs.
- Schedule vCPUs on physical processors.
- Map and protect guest memory.
- Provide virtual storage, network and device interfaces.
- Trap and handle privileged guest operations.
- Enforce resource limits, reservations and priorities.
- Support snapshots, cloning, migration and high availability when available.
- Expose management interfaces for administrators and automation tools.

Hypervisor vs operating system

Aspect	Hypervisor	Operating system
Primary purpose	Create and isolate virtual machines	Provide services and resource management for applications
Main abstraction	Virtual hardware / VM	Process, file, socket, virtual memory and device abstractions
Workloads managed	Guest operating systems and VMs	Applications and user processes
Examples	ESXi, Hyper-V, KVM, Xen, VMware Workstation	Linux, Windows, macOS, Android
Relationship	May run directly on hardware or on a host OS	May run directly on hardware or as a guest inside a VM

Important nuance
Some platforms blur the boundary. For example, KVM is integrated into the Linux kernel, and management tools in user space help create and control VMs. The architectural classification is still useful even when implementation details overlap.

H1 - Scenario question: A student says, “The hypervisor and the guest OS do exactly the same job.” Give two differences.

H2 - Scenario question: Which hypervisor function prevents two VMs from writing to the same physical memory location without authorization?

10. Type 1 and Type 2 Hypervisors

Type 1 runs directly on hardware; Type 2 runs as an application on a host operating system.

Type 1 stack

Virtual machines
Type 1 hypervisor
Physical hardware

Type 2 stack

Virtual machines
Type 2 hypervisor application
Host operating system
Physical hardware

Criterion	Type 1: Bare-metal	Type 2: Hosted
Placement	Directly on physical hardware	Runs on top of a host operating system
Examples	VMware ESXi, Microsoft Hyper-V, Xen; KVM is commonly used in server virtualization	VMware Workstation, Oracle VirtualBox, Parallels Desktop
Performance	Usually lower overhead and more predictable	Additional host OS overhead
Security surface	Smaller dedicated stack when properly configured	Includes host OS and its applications
Management	Centralized enterprise tooling	Desktop application interface
Typical use	Data centers, private cloud, production server consolidation	Learning, development, testing, personal workstation
Hardware support	Enterprise server hardware often preferred	Runs on general desktop/laptop hardware

Real-life example

Type 1 is like a building designed from the beginning as separate apartments. Type 2 is like creating a secure room inside an already functioning house. Both provide separation, but the second depends on the house and its existing systems.

Selection logic

A university student running Ubuntu on a personal Windows laptop should normally use a Type 2 hypervisor. A company hosting hundreds of production VMs should normally use Type 1 virtualization with centralized management, redundancy and monitoring.

Y1 - Scenario question: A bank needs 200 production VMs with centralized control and high availability. Which hypervisor type is more appropriate and why?

Y2 - Scenario question: A student needs one Ubuntu VM for a two-hour lab on a Windows laptop. Which type is more practical?

11. Cloud Computing: Definition and Relationship with Virtualization

Cloud computing delivers configurable computing services over a network with self-service, pooling, elasticity and measured usage.

Definition

Cloud computing is a model for providing convenient, on-demand network access to a shared pool of configurable computing resources such as servers, storage, networks, platforms and applications that can be rapidly provisioned and released with limited manual effort.

Real-life example

Electricity is consumed as a utility: users do not build a power station for every appliance; they request power when needed and pay according to usage. Cloud computing follows a similar service model for IT resources, although customers still remain responsible for architecture, access control, data and cost decisions.

Virtualization vs cloud

Virtualization	Cloud computing
Technology that abstracts physical resources	Service and operating model that delivers resources on demand
Can exist on one offline laptop	Normally accessed through a network and APIs
Creates VMs or virtual resources	Adds self-service, automation, orchestration, pooling and metering
Does not automatically include billing or elasticity	Typically includes measured usage and rapid scaling
One foundation of many clouds	May use VMs, containers, bare metal and managed services

Key exam statement
Virtualization is an enabling technology; cloud computing is a broader service-delivery model. A personal Ubuntu VM in VMware is virtualization, but it is not a complete cloud because it lacks broad self-service, resource pooling, automated elasticity and measured service.

On-premises vs cloud

Factor	On-premises infrastructure	Public cloud
Ownership	The organization buys and owns hardware	Provider owns physical infrastructure
Provisioning	Procurement may take days or months	Resources may be created in minutes through portal or API

Cost model	High upfront capital cost plus maintenance	Usage-based operating cost, commitments or subscriptions
Control	High physical and configuration control	Control depends on service model
Scaling	Limited by purchased capacity	Rapid scale within quotas and regional capacity
Responsibility	Organization manages almost everything	Responsibilities are shared with provider

CL1 - Scenario question: A company installs VMware on one server and manually creates VMs. Is this automatically cloud computing? Explain which cloud characteristics may be missing.

CL2 - Scenario question: Why can cloud computing use bare-metal servers and containers even though virtualization is a major foundation?

12. Five Essential Characteristics of Cloud Computing

These characteristics help distinguish a cloud service from ordinary remote hosting.

Characteristic	Meaning	Example
On-demand self-service	Users provision resources without waiting for a human administrator for every request.	A developer creates a VM from a portal or API.
Broad network access	Services are accessible through standard networks and usable from different client devices.	A student opens a cloud-hosted application from laptop or phone.
Resource pooling	Provider resources are shared among multiple consumers with location abstraction and isolation.	Many customer VMs share a large data-center cluster.
Rapid elasticity	Capacity can expand or contract quickly according to demand.	An e-commerce site increases from 3 to 20 instances during a sale.
Measured service	Usage is monitored, controlled and often billed by units such as time, storage, requests or data transfer.	A monthly bill shows VM hours and storage consumption.

Working mechanism

A cloud management platform receives an API request, authenticates the user, checks policy and quota, selects resources from a pool, provisions the service, configures networking and security, monitors usage and records metrics for operations and billing.

Numerical example: elasticity

A service receives 1,800 requests per second. One instance can safely handle 400 requests per second. Applying a 25% safety factor, required instances = ceiling((1,800 / 400) x 1.25) = ceiling(5.625) = 6 instances.

E1 - Scenario question: A hosting company requires customers to email an administrator and wait two days for every new server. Which cloud characteristic is weak or missing?

E2 - Scenario question: An application automatically adds servers during peak hours but does not remove them afterward. Which characteristic is only partially implemented, and what cost problem can result?

13. Cloud Service Models: IaaS, PaaS and SaaS

Service models define how much of the technology stack the provider manages and how much remains under customer control.

IaaS - Infrastructure as a Service

The provider supplies virtualized compute, storage and networking. The customer usually manages the guest operating system, patches, middleware, runtime, application configuration and data. Examples include Amazon EC2, Azure Virtual Machines and Google Compute Engine.

PaaS - Platform as a Service

The provider manages the infrastructure, operating system and application platform. Developers deploy code and manage application logic and data without managing ordinary server setup. Examples include Azure App Service, Google App Engine and AWS Elastic Beanstalk.

SaaS - Software as a Service

The provider delivers a complete application over the network. Users configure and use the software rather than managing servers or application runtime. Examples include Microsoft 365, Google Workspace, Salesforce and Zoom.

Layer	On premises	IaaS	PaaS	SaaS
Application	Customer	Customer	Customer	Provider
Data and access configuration	Customer	Customer	Customer	Shared: provider protects platform; customer governs users/data

Runtime and middleware	Customer	Customer	Provider	Provider
Operating system	Customer	Customer	Provider	Provider
Virtualization	Customer	Provider	Provider	Provider
Servers, storage, network	Customer	Provider	Provider	Provider
Physical facility	Customer	Provider	Provider	Provider

Choosing a service model

- Choose IaaS when you need control over the operating system, custom network design or legacy software.
- Choose PaaS when the team wants to focus on code and accepts provider platform constraints.
- Choose SaaS when a ready-made business function is available and custom infrastructure is unnecessary.
- Greater provider management usually reduces operational work but also reduces low-level control.

Real-world developer example

A team building a small course-registration web app may begin with PaaS because deployment and scaling are managed. A legacy ERP requiring a specific OS may require IaaS. For email and office documents, the university is more likely to consume SaaS rather than build its own system.

SM1 - Scenario question: A team needs root access and a specific kernel module. Which service model is most appropriate?

SM2 - Scenario question: A department only needs email, file sharing and online meetings. Which service model offers the least infrastructure-management burden?

14. Cloud Deployment Models

Deployment models describe who owns, operates and accesses the cloud environment.

Model	Definition	Example use	Advantages	Limitations
Public cloud	Services offered over provider infrastructure to many customers	Startup hosts a web application on AWS, Azure or Google Cloud	Fast provisioning, broad service catalog, global reach	Ongoing cost, data-governance and vendor-dependence concerns
Private cloud	Cloud environment dedicated to one organization	Bank operates an internal OpenStack or VMware cloud	High control and customization	Requires capital, skilled staff and maintenance

Community cloud	Shared by organizations with common requirements	Government or research institutions share compliant infrastructure	Shared cost and aligned policy	Smaller scale and complex governance
Hybrid cloud	Integrated use of private/on-premises and public cloud	Sensitive database on premises, scalable web tier in public cloud	Flexibility, gradual migration, data-placement choice	Integration, networking and security complexity

Comparison with multi-cloud

Hybrid cloud combines private or on-premises resources with public cloud. Multi-cloud means using services from more than one cloud provider. An organization may be hybrid, multi-cloud, both or neither.

Significance

Deployment choice affects compliance, latency, cost, control, disaster recovery and required skills. Industry decisions are rarely based on technology alone; legal requirements, data residency, organizational capability and business risk are equally important.

D1 - Scenario question: A hospital keeps patient records in a controlled private environment but uses public cloud for a public information website. Which deployment model best describes the combined design?

D2 - Scenario question: A company uses AWS for analytics and Azure for identity-related services, with no private cloud. Is this hybrid cloud, multi-cloud or both?

15. Cloud Benefits, Risks and Cost Management

Cloud changes how infrastructure is obtained and paid for; it does not remove the need for engineering discipline.

Potential advantage	Explanation
Speed and agility	Resources can be provisioned rapidly through portals and APIs.
Elasticity	Capacity can match changing demand.
Global reach	Applications can be deployed near users in different regions.
Managed services	Teams can use databases, queues, analytics and monitoring without operating every component.
Reduced upfront capital	Organizations can begin without purchasing a full data center.
Resilience options	Multiple zones, regions, backups and automation support fault-tolerant designs.

Risk or disadvantage	Explanation / response
Vendor lock-in	Use open standards, portable architecture and an exit plan where justified.
Uncontrolled cost	Use budgets, tagging, right-sizing, schedules and FinOps practices.
Data privacy and residency	Choose suitable regions, contracts, encryption and governance.
Internet and provider dependency	Design offline procedures, redundancy and recovery plans.
Misconfiguration	Use templates, policy-as-code, review and automated security checks.
Skill gap	Train teams and define operational ownership.
Jurisdiction and compliance	Map legal and regulatory requirements before migration.

16. Industry Use Cases and Architecture Examples

Real systems combine virtualization, networking, security, data services and automation according to business requirements.

Use Case A: University Operating System Lab

Each student receives an Ubuntu VM with a standard image. Snapshots allow safe experimentation, host-only networking supports isolated exercises, and templates ensure every student starts with the same configuration. The main risks are insufficient host RAM, VM sprawl and students storing important work only inside disposable VM disks.

Use Case B: Startup Web Application

A small team may begin with PaaS to deploy code quickly. As requirements grow, it may use a managed database, object storage, CDN, monitoring and autoscaling. IaaS is introduced only when the team needs OS-level control or specialized software. This reduces operational burden during early development.

Use Case C: E-commerce Sale Event

The company predicts a tenfold traffic spike. The architecture uses a load balancer, stateless application instances, autoscaling, a cache, a scalable database and a content-delivery network. Capacity is tested before the event, and maximum scaling limits and budgets are configured to prevent runaway cost.

Use Case D: Bank or Government Hybrid Cloud

Sensitive core systems may remain in a private environment because of legacy integration, control or policy. Public cloud may host development, public web services, analytics or disaster-recovery capacity. Secure connectivity, identity federation, encryption, audit logging and clear data-classification rules are essential.

Use Case E: AI and Research Workload

Researchers rent GPU-enabled instances for training instead of buying expensive hardware that may remain idle. Data is stored in object storage, experiments are tracked, and instances are stopped after use. Large data transfer and always-on GPU cost must be carefully monitored.

Use Case F: Disaster Recovery

Backups or replicated VM images are stored in a separate failure domain. During an outage, infrastructure templates recreate networks and servers, data is restored or promoted, and DNS sends users to the recovery site. Recovery Time Objective (RTO) defines how quickly service must return; Recovery Point Objective (RPO) defines how much recent data loss is acceptable.

Common cloud roles in industry

Role	Main focus
Cloud engineer	Provisioning, networking, identity, compute and storage
DevOps engineer	Automation, CI/CD, infrastructure as code and deployment
Site Reliability Engineer (SRE)	Reliability, observability, incident response and service objectives
Cloud security engineer	IAM, threat detection, compliance and secure architecture
Platform engineer	Reusable internal platforms for developer teams
FinOps practitioner	Cost visibility, allocation, optimization and business accountability

17. Numerical Practice

Use these problems to connect abstract concepts with real capacity, cost and availability decisions.

N1 - Scenario question: Host memory: A server has 64 GB RAM. The hypervisor reserves 8 GB. Six VMs are assigned 8 GB each. How much unassigned RAM remains? Can another 12 GB VM be added without overcommitment?

N2 - Scenario question: CPU ratio: A host has 12 physical CPU cores. Its active VMs have a total of 36 vCPUs. Find the vCPU-to-core overcommit ratio.

N3 - Scenario question: Thin storage: Ten VMs each have a 100 GB thin disk and currently use 25 GB. The datastore capacity is 600 GB. Calculate current use, maximum provisioned capacity and provisioned-to-physical ratio.

N4 - Scenario question: Autoscaling: Traffic is 2,400 requests/s. One instance safely handles 500 requests/s. Add a 20% safety margin and calculate the minimum number of instances.

N5 - Scenario question: Availability: Calculate approximate monthly downtime for 99.9% and 99.99% availability in a 30-day month.

N6 - Scenario question: Compute cost: A test VM costs a hypothetical $0.06/hour. It runs 10 hours per weekday for 22 weekdays. Estimate monthly compute cost and compare with running 730 hours.

N7 - Scenario question: Data transfer time: A 100 GB VM image is copied over an effective 100 Mbit/s link. Ignoring overhead, estimate transfer time in hours.

N8 - Scenario question: Ideal redundancy: Each of two independent servers has 98% availability. Under an ideal failover assumption, calculate the probability that both are unavailable and the combined availability.

N9 - Scenario question: Storage growth: A 500 GB datastore currently stores eight thin VMs using 45 GB each. If each grows by 20 GB, will the datastore remain within capacity?

N10 - Scenario question: Consolidation: A company replaces eight lightly used physical servers with two virtualization hosts. If each old server used 250 W and each new host uses 600 W, compare total power draw, ignoring cooling.

Solutions

N1: Usable for VMs = 64 - 8 = 56 GB. Six VMs use 6 x 8 = 48 GB. Remaining = 8 GB. A 12 GB VM cannot be added without at least 4 GB overcommitment.

N2: 36 / 12 = 3:1.

N3: Current use = 10 x 25 = 250 GB. Maximum provisioned = 10 x 100 = 1,000 GB. Provisioned-to-physical ratio = 1,000/600 = 1.67:1. Current free physical capacity is about 350 GB before overhead.

N4: Base instances = 2,400/500 = 4.8. With 20% margin: 4.8 x 1.2 = 5.76. Round up to 6 instances.

N5: 30 days = 43,200 minutes. 99.9% downtime = 0.001 x 43,200 = 43.2 minutes. 99.99% downtime = 0.0001 x 43,200 = 4.32 minutes.

N6: Scheduled use = 10 x 22 = 220 hours. Cost = 220 x $0.06 = $13.20. Always-on cost = 730 x $0.06 = $43.80. Scheduling saves $30.60 in this simplified example.

N7: 100 GB = approximately 800 gigabits. 800 / 0.1 = 8,000 seconds = 133.3 minutes = about 2.22 hours. Real transfer takes longer due to overhead.

N8: Failure probability per server = 0.02. Both unavailable = 0.02 x 0.02 = 0.0004 = 0.04%. Ideal combined availability = 99.96%.

N9: Current use = 8 x 45 = 360 GB. Growth = 8 x 20 = 160 GB. New total = 520 GB, which exceeds 500 GB by 20 GB before overhead.

N10: Old servers = 8 x 250 = 2,000 W. New hosts = 2 x 600 = 1,200 W. Direct server power reduction = 800 W, or 40%, before considering cooling and storage.

18. Common Misconceptions

Misconception	Correct explanation
“Allocating 50 GB to a VM immediately removes 50 GB from my laptop.”	Only always true for thick reservation. A thin virtual disk may initially use much less and grow as data is written.
“A powered-off VM still consumes its assigned RAM.”	A powered-off VM stores files on disk but does not normally occupy its configured active RAM. It consumes RAM when powered on.
“A VM has its own real CPU cores.”	A VM usually has vCPUs scheduled onto shared physical cores. Dedicated CPU pinning is possible but not the default.
“Virtualization creates unlimited resources.”	It creates flexible abstractions, not extra physical capacity. Overcommitment can cause contention.
“Type 2 means the guest is only simulated.”	Modern Type 2 hypervisors normally use hardware-assisted virtualization and execute much guest code directly.
“One VM on the Internet is a cloud.”	Cloud additionally requires service characteristics such as self-service, pooling, elasticity and measurement.

“The cloud provider handles all security.”	Security is shared. Customers remain responsible for identity, data, configuration and application security.
“SaaS means the user has no responsibility.”	Users still manage accounts, permissions, data sharing, configuration and acceptable use.
“Cloud is always cheaper.”	Cloud can reduce upfront cost and improve agility, but poor sizing, idle resources and data transfer can make it expensive.
“Containers are small VMs.”	Containers share the host kernel, whereas VMs include a guest kernel and virtual hardware.
“High availability means backup is unnecessary.”	Availability keeps service running; backup protects against deletion, corruption, ransomware and retention needs.

Summary :

- Virtualization = abstraction of physical resources into isolated virtual environments.
- VM components = host, guest, hypervisor, vCPU, vRAM, vDisk and vNIC.
- Virtualization triad = safety/control, fidelity/equivalence and efficiency.
- Type 1 runs on hardware; Type 2 runs on a host OS.
- VM vs dual boot: simultaneous shared use vs one native OS at a time.
- Thin disk grows with use; thick disk reserves capacity early.
- Snapshot supports short rollback; backup is an independent recovery copy.
- Live migration transfers a running VM with minimal interruption.
- Cloud = on-demand network service with pooling, elasticity and measured use.
- NIST characteristics = on-demand self-service, broad network access, resource pooling, rapid elasticity and measured service.
- IaaS gives infrastructure control; PaaS gives a managed platform; SaaS gives a complete application.
- Deployment models = public, private, community and hybrid.
- Production cloud architecture uses multiple instances, zones, load balancing, monitoring, backups and secure IAM.
- Provider security does not replace customer responsibility.
- Always evaluate performance, reliability, security, compliance and cost together.

19. Glossary and References

Term	Meaning
Abstraction	A simplified logical view that hides lower-level physical details.
Availability zone	A separate data-center failure domain within a cloud region.
Autoscaling	Automatic addition or removal of capacity based on policy or demand.
Bare metal	Software running directly on physical hardware without a host OS layer.
Checkpoint	A stored state used to resume or roll back a VM or computation.
Cloud computing	On-demand network delivery of pooled configurable computing services.
Container	An isolated application environment that shares the host kernel.
Guest OS	The operating system running inside a VM.
Host	The physical system, and for Type 2 the main OS, that supports VMs.
Hypervisor / VMM	The control layer that creates and manages virtual machines.
IaaS	Cloud model providing compute, storage and networking infrastructure.
Image	A packaged disk or template used to create a VM or container.
Live migration	Movement of a running VM between hosts with minimal interruption.
Load balancer	Service that distributes traffic across multiple backend instances.
Measured service	Monitoring and recording of cloud resource usage.
Multi-cloud	Use of services from multiple cloud providers.
PaaS	Cloud model providing a managed application platform.
Public cloud	Provider-operated cloud shared across customers with logical isolation.
Region	A geographic cloud infrastructure area.
Resource pooling	Sharing provider resources among multiple consumers.
SaaS	Complete software delivered as a network service.

Snapshot	Point-in-time VM state or storage change record used for rollback.
Thin provisioning	Allocating physical storage gradually as virtual storage is used.
Thick provisioning	Reserving the selected physical storage capacity in advance.
vCPU	A virtual processor scheduled on physical CPU resources.
vDisk	A file or logical volume presented to a guest as a disk.
vNIC	A virtual network interface connected through a virtual switch.
VM sprawl	Uncontrolled growth of unused or poorly managed virtual machines.

References and Further Reading

Lecture slide: Virtualization, VM & Cloud Computing, CSE-307

Textbook: A. Silberschatz, P. B. Galvin and G. Gagne, Operating System Concepts, 9th Edition, Chapter 16: Virtual Machines.

Textbook: A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th Edition, Chapter 7: Virtualization and the Cloud.

NIST: The NIST Definition of Cloud Computing, Special Publication 800-145.

AWS: Types of Cloud Computing; What Is Cloud Computing?; Shared Responsibility Model.

Microsoft Learn: Azure Virtual Machine Scale Sets; Azure Load Balancer; Azure reliability guidance.

Google Cloud: Compute Engine overview; Live migration process; App Engine documentation.

NIST SP 800-145

AWS: What is cloud computing?

AWS: Types of cloud computing

AWS shared responsibility model

Azure VM Scale Sets overview

Azure Load Balancer overview

Google Compute Engine overview

Google Cloud live migration

Google App Engine

Virtualization teaches how one physical machine can safely provide several logical computers. Cloud computing extends this idea with networking, automation, self-service, elasticity, metering and managed services. In industry, systems are judged not only by whether they run, but also by security, reliability, performance, recoverability, maintainability and cost.

See the slides for understanding the basics and concept. *

Already told in class what type of questions can come, so those are very important for the incoming MID, CT-3 and Term final *

CSE-307: Operating System

Memory Management in Operating Systems

Study Notes and Numerical Practice Problems

Prepared by: Lec. Khaled Hasan, Department of CSE, MIST

Personal Website: https://khaledhasanirfan.github.io/ (Under construction, More sections to come, keep an eye and open for constructive criticism for myself, feedback will be appreciated, for potential collaboration come to my office or mail: (khaled@cse.mist.ac.bd) THANKS IN ADVANCE

Section A: Study Notes

A.1 Memory Hierarchy

A computer does not store all its data in one type of memory. Instead, memory is arranged in layers, ordered by distance from the CPU. The layers, from closest to farthest, are: CPU registers, cache (L1, L2, L3), main memory (RAM), and secondary storage (SSD, HDD). As we move down this list, speed decreases, capacity increases, and cost per byte decreases.

Registers sit directly inside the CPU and hold the operands the CPU is working on right now; they are the fastest and smallest. Cache stores recently used data so the CPU does not have to wait for slower RAM every time. RAM holds the programs that are currently running. Secondary storage (SSD or HDD) holds programs and files permanently, even when the power is off, but it is far slower to access.

Key idea: a program's binary file lives on disk. To actually run, it must first be loaded into RAM, since the CPU can only execute instructions that are available in main memory. RAM acts as the bridge between slow, large disk storage and fast, tiny CPU processing.

Volatile vs non-volatile memory:

- Volatile memory loses its contents when power is removed. Examples: registers, cache, RAM. Used as temporary working space.
- Non-volatile memory keeps its contents after shutdown. Examples: ROM, SSD, HDD, USB drives. Used for permanent storage.

A simple test: turn the power off. Whatever disappears was volatile; whatever survives was non-volatile.

A.2 Multiprogramming

In single-programming, only one process resides in memory at a time. Whenever that process waits for input or output, the CPU has nothing else to do and sits idle. This wastes CPU time, since most programs spend a large fraction of their life waiting for I/O rather than computing.

Multiprogramming solves this by keeping several processes in RAM at the same time. When one process is waiting for I/O, the CPU simply switches to another resident

process instead of idling. The goal of multiprogramming is not just to fill up RAM; the real goal is to reduce CPU idle time.

Degree of multiprogramming (n) is the number of processes that are resident in main memory at the same time, available for execution. A higher degree of multiprogramming generally means the CPU is less likely to run out of work.

Numerical model of CPU utilization:

Let p be the probability that a single process is waiting for I/O at any given moment, and let n be the degree of multiprogramming. Assuming each process's I/O-wait behaviour is independent, the probability that every single resident process is waiting for I/O at the same time is p raised to the power n (that is, p multiplied by itself n times). CPU utilization is then:

$$\mathbf{U = 1 - p^n}$$

In plain words: CPU utilization is 1 minus the chance that everyone is stuck waiting at once. As n grows, p^n shrinks rapidly (since p is a fraction less than 1), so utilization climbs toward 100 percent.

Worked example: suppose p = 0.80 (each process is idle on I/O 80 percent of the time), the OS itself occupies 2 GB, and each process needs 2 GB.

Total RAM	Processes (n)	Calculation	CPU Utilization
4 GB	1	$1 - 0.8^1$	20.0%
8 GB	3	$1 - 0.8^3$	48.8%
16 GB	7	$1 - 0.8^7$	≈ 79.0%

Key idea: adding more RAM allows a higher degree of multiprogramming, which raises CPU utilization. Note that this does not make the CPU itself run faster; it simply keeps the CPU fed with more available work, so less time is wasted idling.

A.3 Contiguous vs Non-contiguous Allocation

Memory allocation strategies fall into two broad categories, based on how a process's memory is physically placed.

Contiguous allocation:

The entire process must occupy one single, unbroken block of physical memory. This is simple to understand and manage, but it means fragmentation can prevent a process from running even when enough total free memory exists somewhere in the system. Fixed partitioning and dynamic (variable) partitioning both belong to this category.

Non-contiguous allocation:

Different pieces of the same process may be placed in separate, scattered physical locations. This is more flexible and largely avoids the fragmentation problem, but it requires the OS to maintain a mapping (such as a page table) so the CPU can still find every piece correctly. Paging, segmentation, and segmentation with paging belong to this category.

A.4 Fixed (Static) Partitioning

In fixed partitioning, main memory is divided into a fixed number of partitions before any process arrives, typically at system boot. Each partition has a size decided in advance and does not change afterward. This scheme is also called MFT (Multiprogramming with a Fixed number of Tasks).

Core mechanism:

- The OS reserves one partition for itself; the rest of RAM is split into partitions beforehand.
- Normally, one partition holds exactly one process.
- A partition becomes free again once its process finishes and exits.

Partitions can be equal-size (simple to manage, but a small process wastes a large partition, and a very large process may not fit anywhere) or unequal-size (fits a wider range of process sizes, but is still fixed and inflexible once chosen).

Internal fragmentation:

Internal fragmentation is wasted space trapped inside an allocated partition, because the partition is larger than the process placed inside it. This space cannot be used by any other process, even though it is technically "free" within that partition.

Example: Partition (32 KB) − Process (18 KB) = Internal fragmentation (14 KB).

Partition	Process	Internal Fragmentation
100 MB	80 MB	20 MB
200 MB	150 MB	50 MB
300 MB	270 MB	30 MB
400 MB	350 MB	50 MB

Total waste in this example = 150 MB.

Spanning vs non-spanning:

- Non-spanning: a process must fit completely inside a single partition. If no single partition is large enough, the process fails, even if two smaller partitions together would have been enough.
- Spanning: a process may be split across more than one partition. This is possible in principle but adds real complexity, so most simple fixed-partition systems use non-spanning.

Advantages and limitations of fixed partitioning:

- Advantages: simple to understand, fast allocation and deallocation, low management overhead, predictable memory layout, easy to implement.
- Limitations: internal fragmentation, a fixed maximum number of resident processes, large processes may not fit at all, partition sizes cannot adapt, generally poor RAM utilization.

A.5 Dynamic (Variable) Partitioning

In dynamic partitioning, a partition is created at the exact moment a process is loaded, sized exactly to match that process. There is no fixed boundary decided in advance. This scheme is also called MVT (Multiprogramming with a Variable number of Tasks).

Core mechanism:

- Memory starts as one large free block.
- Each arriving process carves out a partition of exactly its own size.
- When a process exits, its space becomes a hole: a contiguous block of free, unallocated memory available for future use.

External fragmentation:

Over time, as processes of different sizes come and go, memory becomes broken up into many small, scattered holes. External fragmentation means enough total free memory exists somewhere in the system, but not as one single contiguous block big enough for the next request.

Example: total free memory is 240 MB, split as three holes of 100 MB, 60 MB, and 80 MB, scattered between allocated processes. An incoming request for 180 MB fails, even though 240 MB is technically free, because the largest single hole is only 100 MB.

Advantages and limitations of dynamic partitioning:

- Advantages: partition size matches the process exactly, little to no internal fragmentation, more flexible than fixed partitioning, no fixed limit on partition count, generally better initial RAM utilization.
- Limitations: external fragmentation appears over time, allocation requires searching a free-hole list, that free-list must be actively managed, compaction may become necessary, and the system tends to slow down and grow more complex as fragmentation accumulates.

One-line engineering summary: fixed partitions waste memory internally (the box is too big for the process inside it); dynamic partitions shatter memory externally (the empty space itself gets broken into unusable pieces). Placement algorithms exist to slow down exactly this shattering.

A.6 Placement Algorithms for Dynamic Partitioning

When several holes are available and a new process needs memory, the OS must decide which hole to use. This decision is made by a placement algorithm. No single algorithm is always the best; performance depends on the specific sizes and arrival order of the processes involved.

First Fit:

Scan the hole list from the beginning, and allocate the first hole that is large enough. This is the fastest algorithm, since it stops searching as soon as a suitable hole is found.

Next Fit:

Similar to First Fit, but the search resumes from the point where the previous search left off, instead of always restarting from the beginning. This spreads allocations more evenly

across memory, but it can skip over a perfectly good hole located earlier in the list until the search wraps around.

Best Fit:

Search the entire hole list, and allocate the smallest hole that is still large enough for the request. This tends to preserve large holes for future large requests, but it also tends to leave behind many tiny, mostly useless leftover holes.

Worst Fit:

Search the entire hole list, and allocate the largest hole available. The idea is that the leftover remainder stays usably large. In practice, this strategy tends to consume the biggest holes quickly, which can hurt later when a genuinely large request arrives.

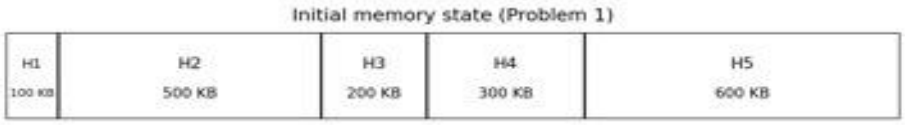
Algorithm	Search Idea	Strength	Limitation
First Fit	First sufficient hole	Fast and simple	Small holes accumulate near the start
Next Fit	Resume from last pointer	Spreads out allocations	May miss a suitable earlier hole
Best Fit	Smallest sufficient hole	Preserves large holes	Produces many tiny, unusable holes
Worst Fit	Largest available hole	Leaves larger remainders	Quickly consumes the biggest holes

So, properly have an idea about each of the important terms, concepts, and know why they occur and in which scenario? What are the alternatives? Try to compare different things, scenes, processes, methods? And most importantly, try to understand everything with examples as I love examples and you will expect it in your questions for sure.

Section B: Numerical Practice Problems *****

The following five problems cover First Fit, Next Fit, Best Fit, and Worst Fit placement, along with one combined problem that adds process completion time. Each problem begins with a diagram of the starting memory layout. Work through each one by hand before checking the answer key at the end of this document.

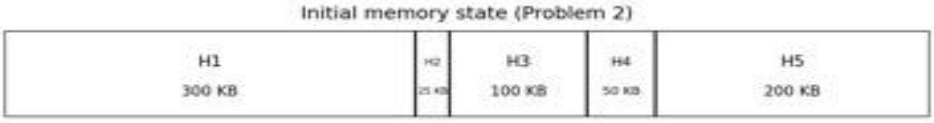
Problem 1: First Fit



Processes arrive in this order and must be placed as they arrive: 212 KB, 417 KB, 112 KB, 426 KB.

Task: Apply the First Fit algorithm. Show which hole each process is placed into, state which request (if any) fails, and report the final memory state (the size of every remaining hole) after all requests have been attempted.

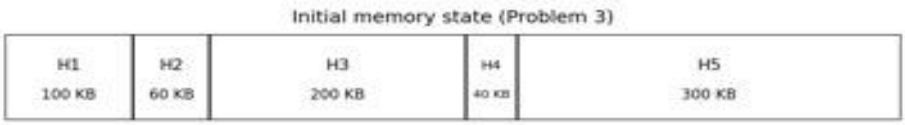
Problem 2: Best Fit vs Worst Fit



Processes arrive in this order: 115 KB, 45 KB, 90 KB.

Task: Apply Best Fit and Worst Fit separately to the same starting memory state and the same process sequence. For each algorithm, show the step-by-step trace and the final list of remaining holes. Then state which of the two algorithms leaves less total wasted (fragmented) space in this particular case.

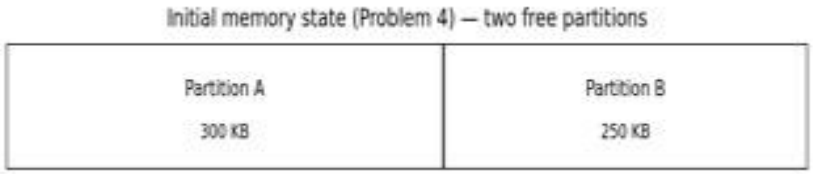
Problem 3: Next Fit



Processes arrive in this order: 80 KB, 30 KB, 150 KB. Assume the search pointer starts at the first hole (H1) before the first request arrives.

Task: Apply Next Fit. For each process, state which hole the search pointer lands on, and where the pointer moves to afterward. Report the final memory state once all three processes are placed.

Problem 4: Placement Combined with Completion Time



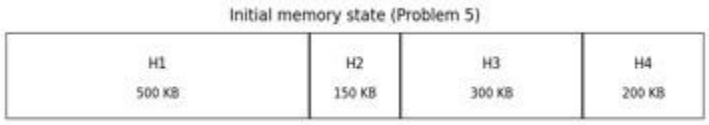
Five processes arrive together at time 0, in this order, each shown as (size, CPU burst time):

Process	Size	Burst Time
P1	200 KB	4 ms
P2	150 KB	3 ms
P3	280 KB	5 ms
P4	100 KB	2 ms
P5	220 KB	3 ms

Rules for this system: there is only one CPU, so only one process can execute at a time. A process must first be loaded into a free partition before it can run, using First Fit (check Partition A, then Partition B). Whenever a partition becomes free, the next process still waiting in the original arrival order that fits into an available partition is loaded immediately. The CPU serves loaded processes strictly in the order they were loaded (FCFS).

Task: Trace which partition each process is loaded into and when. Then determine the CPU execution order and compute the completion (finish) time of process P5.

Problem 5: Worst Fit



Processes arrive in this order: 120 KB, 180 KB, 90 KB.

Task: Apply Worst Fit step by step. Report the final memory state and the total remaining free memory after all three processes are placed.

Answer Key (try to do it on your own first)

Problem 1: First Fit

Starting holes: H1=100, H2=500, H3=200, H4=300, H5=600 (KB).

- 212 KB → first hole ≥ 212 is H2 (500). Allocate: H2 becomes 288.
- 417 KB → scanning from the start, H2 is now only 288 (too small), H3=200 (too small), H4=300 (too small), H5=600 fits. Allocate: H5 becomes 183.
- 112 KB → scanning from the start, H1=100 (too small), H2=288 fits. Allocate: H2 becomes 176.
- 426 KB → scanning all holes (100, 176, 200, 300, 183), none is ≥ 426. This request FAILS.

Final memory state: H1=100, H2=176, H3=200, H4=300, H5=183 (KB). Total free memory remaining = 959 KB, but the request for 426 KB still failed, since no single hole was large enough.

Problem 2: Best Fit vs Worst Fit

Starting holes: H1=300, H2=25, H3=100, H4=50, H5=200 (KB).

Best Fit trace:

- 115 KB → smallest hole that still fits is H5 (200). Allocate: H5 becomes 85.
- 45 KB → smallest sufficient hole is now H4 (50). Allocate: H4 becomes 5.
- 90 KB → smallest sufficient hole is now H3 (100). Allocate: H3 becomes 10.

Final holes (Best Fit): H1=300, H2=25, H3=10, H4=5, H5=85. Total free = 425 KB.

Worst Fit trace:

- 115 KB → largest hole is H1 (300). Allocate: H1 becomes 185.
- 45 KB → largest hole is now H5 (200). Allocate: H5 becomes 155.
- 90 KB → largest hole is now H1 (185). Allocate: H1 becomes 95.

Final holes (Worst Fit): H1=95, H2=25, H3=100, H4=50, H5=155. Total free = 425 KB.

Comparison: both algorithms leave the same total free memory (425 KB), since the same three processes were placed either way. However, Best Fit leaves several very small, mostly unusable holes (25, 10, 5 KB), while Worst Fit keeps the free space in fewer,

larger, more usable chunks (95, 100, 155 KB). In this particular case, Worst Fit leaves memory in a more usable shape for a future medium-sized request.

Problem 3: Next Fit

Starting holes: H1=100, H2=60, H3=200, H4=40, H5=300 (KB). Pointer starts at H1.

- 80 KB → pointer starts at H1 (100), which fits. Allocate: H1 becomes 20. Pointer stays at H1.
- 30 KB → search continues from H1 onward; H1=20 is too small, H2=60 fits. Allocate: H2 becomes 30. Pointer moves to H2.
- 150 KB → search continues from H2 onward; H2=30 too small, H3=200 fits. Allocate: H3 becomes 50. Pointer moves to H3.

Final memory state: H1=20, H2=30, H3=50, H4=40, H5=300 (KB). Total free = 440 KB.

Problem 4: Placement Combined with Completion Time

Loading and execution trace:

- t = 0: Partition A (300 KB) and Partition B (250 KB) are both free. P1 (200 KB) loads into A. P2 (150 KB) loads into B.
- t = 0 to 4: CPU runs P1 (loaded first).
- t = 4: P1 finishes and releases Partition A. Next waiting process is P3 (280 KB); it fits into A (300 KB), so P3 loads into A.
- t = 4 to 7: CPU runs P2 (next in load order).
- t = 7: P2 finishes and releases Partition B. Next waiting process is P4 (100 KB); it fits into B (250 KB), so P4 loads into B.
- t = 7 to 12: CPU runs P3 (next in load order).
- t = 12: P3 finishes and releases Partition A. The only process still waiting is P5 (220 KB); it fits into A (300 KB), so P5 loads into A.
- t = 12 to 14: CPU runs P4 (next in load order).
- t = 14 to 17: CPU runs P5 (last in load order).

Process P5 finishes at t = 17 ms.

Summary of finish times: P1 = 4 ms, P2 = 7 ms, P3 = 12 ms, P4 = 14 ms, P5 = 17 ms.

Problem 5: Worst Fit

Starting holes: H1=500, H2=150, H3=300, H4=200 (KB).

- 120 KB → largest hole is H1 (500). Allocate: H1 becomes 380.
- 180 KB → largest hole is now H1 (380). Allocate: H1 becomes 200.
- 90 KB → largest hole is now H3 (300) (tied candidates H1=200 and H4=200 are smaller than H3). Allocate: H3 becomes 210.

Final memory state: H1=200, H2=150, H3=210, H4=200 (KB). Total free memory remaining = 760 KB.

COURSE: OPERATING SYSTEM

CODE: CSE-307

TOPIC: VIRTUALIZATION, VIRTUAL MACHINE AND CLOUD COMPUTING BASICS

Lec Khaled Hasan, CSE, MIST

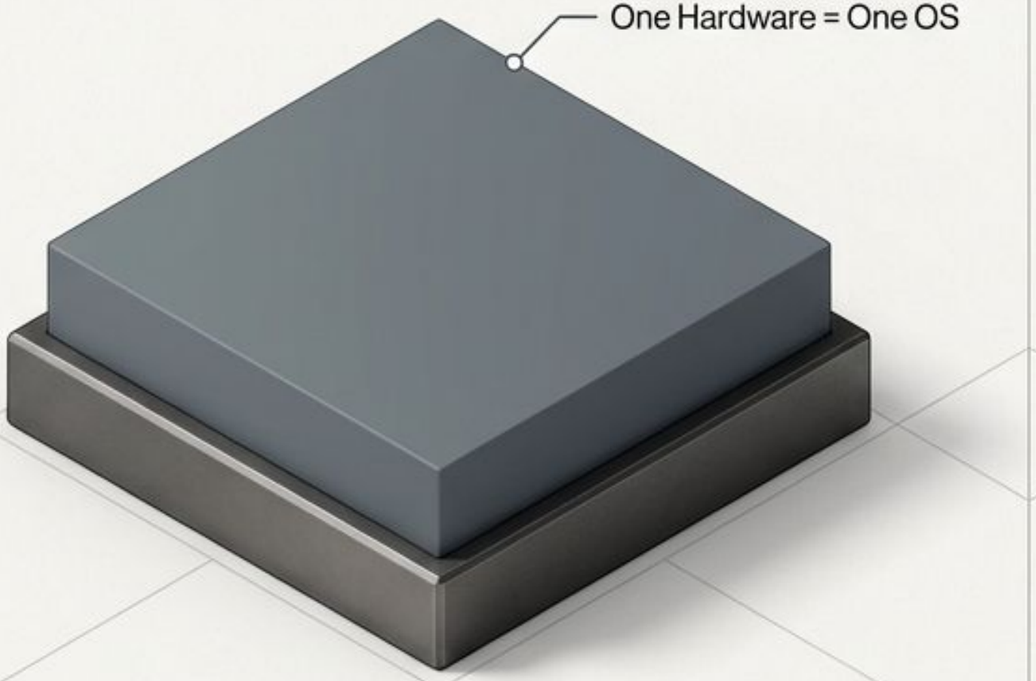
Personal Website : https://khaledhasanirfan.github.io/ (Would love to join and connect with you guys)

NotebookLM

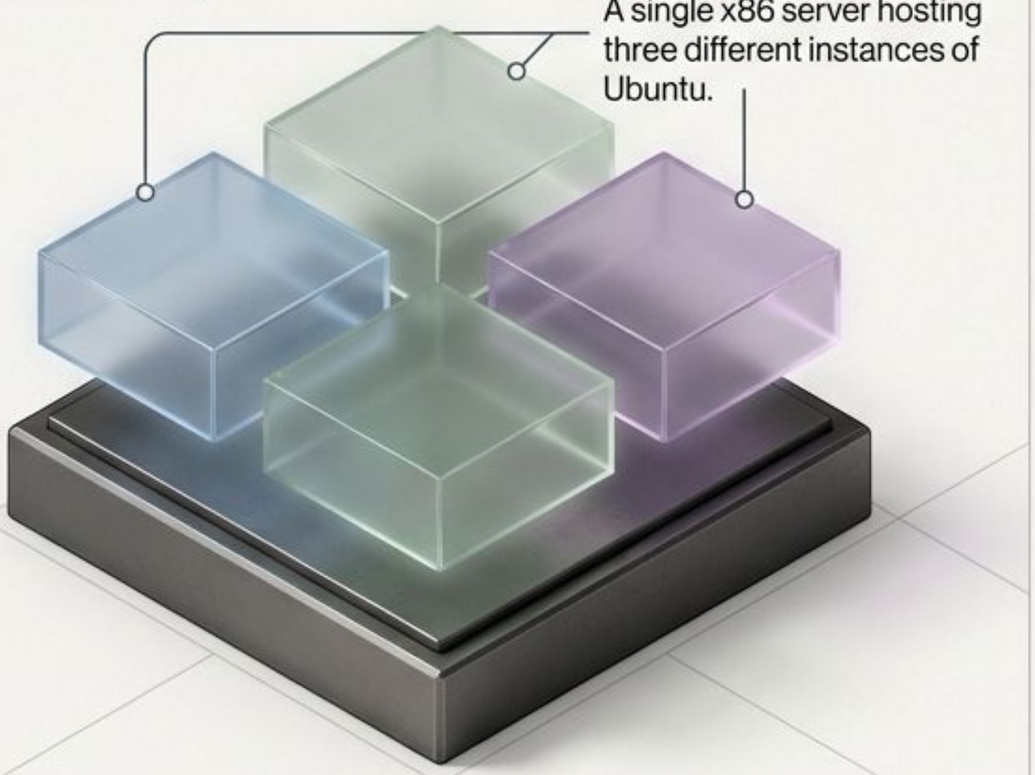
The Illusion of Multiplicity

Virtualization is the software-defined abstraction of physical hardware into multiple independent execution environments. It allows one physical machine to behave exactly like many.

The Past

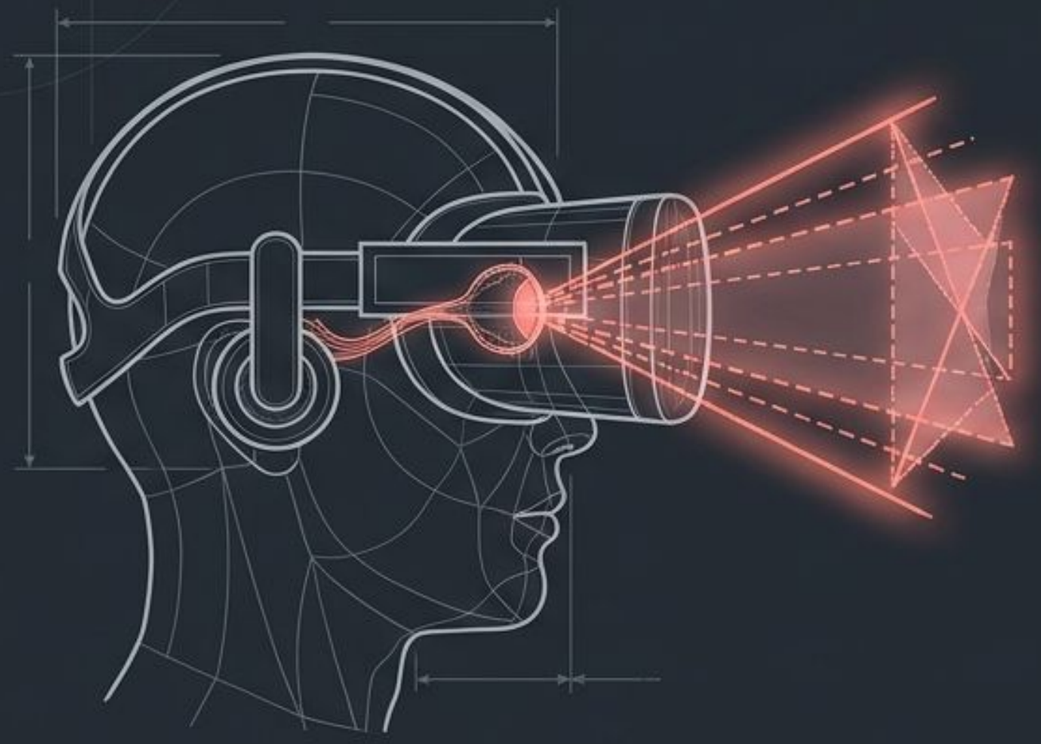


The Virtualized Present



NotebookLM

THE ULTIMATE SOFTWARE ILLUSION





Virtual Reality (The Human Senses):
VR creates the **illusion** of a **physical, three-dimensional world** for the human eye.

Virtualization (The Guest OS):
Virtualization creates the **illusion** of **physical, bare-metal hardware** (CPU, RAM, Disk, I/O) for an **isolated Guest Operating System.**

NotebookLM

The Virtualization Triad

A hypervisor must balance three essential characteristics to provide a viable virtual environment.

Safety

Maintains absolute control over all virtualized resources and isolation.

Efficiency

The vast majority of the code inside the VM executes natively on the CPU without hypervisor intervention.



Fidelity

Program behavior inside the VM is completely identical to its behavior on native bare hardware.

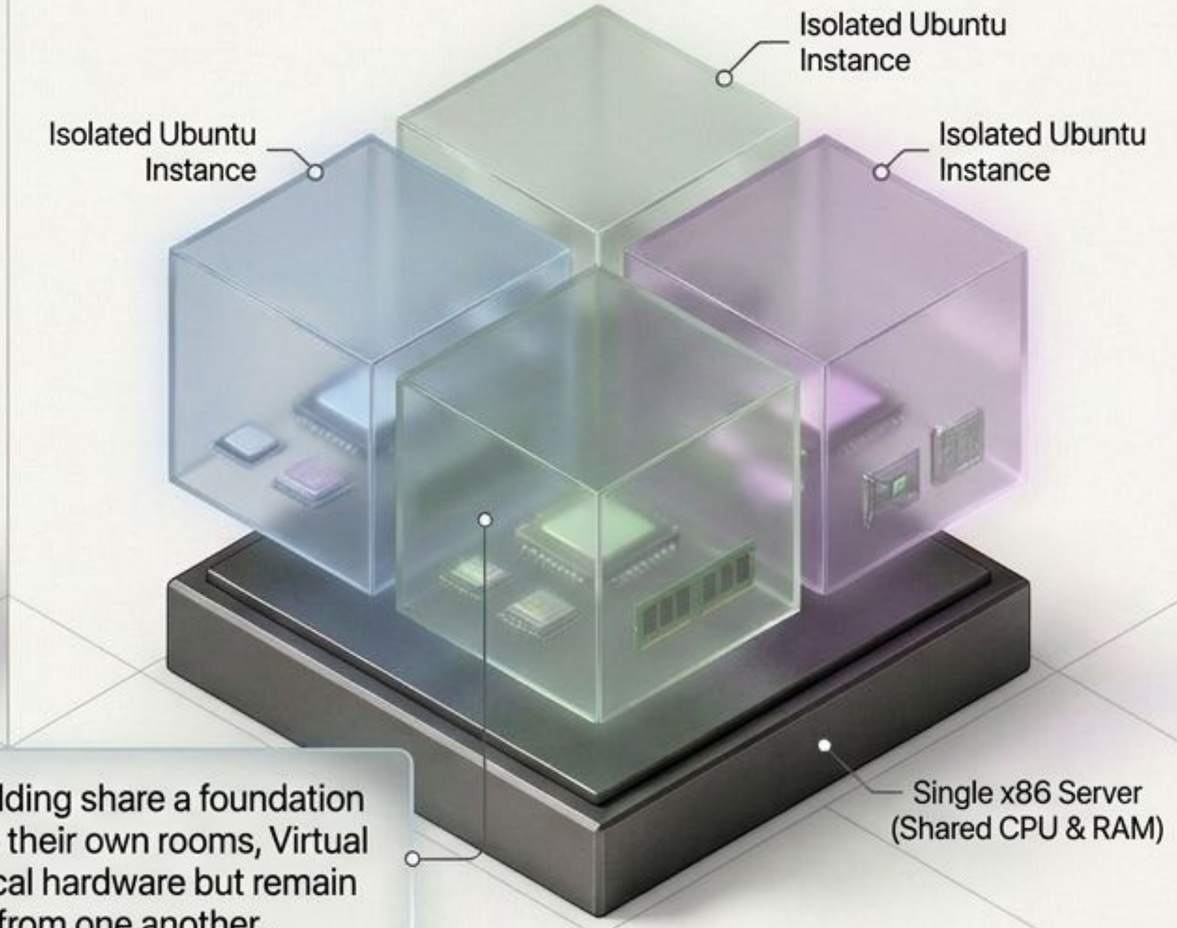
NotebookLM

Shared Infrastructure, Private Environments

The Analogy



The Technical Reality



Just as tenants in a building share a foundation but hold private keys to their own rooms, Virtual Machines share physical hardware but remain entirely isolated from one another.

NotebookLM

The Strategic Trade-Offs of Virtualization

Strategic Advantages

Server Consolidation

Maximizing hardware usage by combining lightly loaded servers onto single physical hosts.

Absolute Isolation

Secure containment of environments, preventing a failure or virus in one VM from affecting others.

Safe R&D

Creating risk-free, isolated sandboxes for rapid development and testing.



Structural Costs

Performance Overhead

Inevitable loss of processing speed and efficiency due to the hypervisor translation layer.

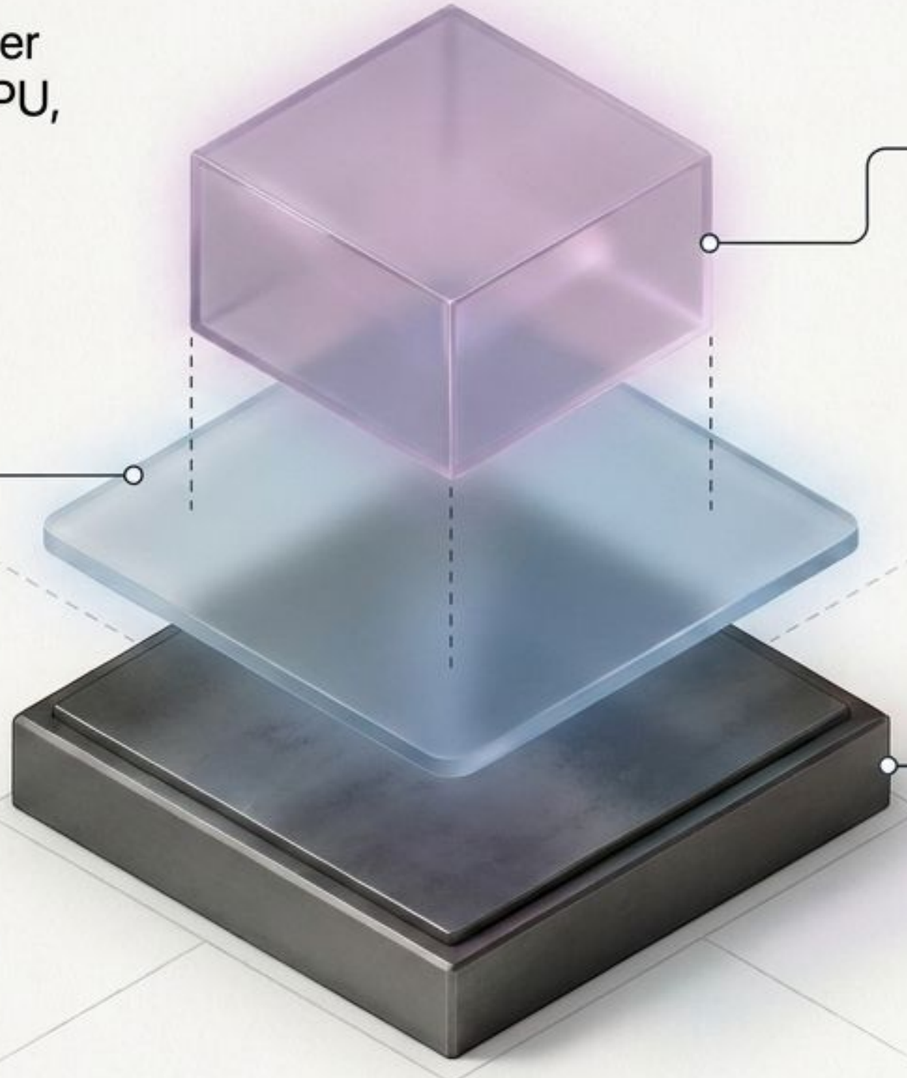
Single Point of Failure

If the underlying physical host or hypervisor crashes, every VM hosted on it instantly fails.

NotebookLM

Anatomy of a Virtual Machine

A VM is a software-defined computer equipped with its own virtualized CPU, RAU, RAM, disk, and I/O devices.



The Guest

The operating system and applications running inside the encapsulated virtual environment.

The Hypervisor / VMM

The critical software layer that creates, allocates, and safely runs the VMs.

The Host

The underlying physical hardware (CPU, RAM, Disk) and its native environment.

NotebookLM

The Physical Constraints of Virtual Spaces

Virtual resources are abstractions, but they are bounded by literal physical hardware constraints.



4GB Remaining Free RAM

Available for Host OS and other tasks

4GB Allocated to Guest OS

Active Windows Virtual Machine

Allocating 4GB of RAM and a 50GB virtual disk to a VM physically removes those resources from the host OS's available pool for as long as the VM is active.

NotebookLM

THE SPACE ILLUSION: RESOURCE PROVISIONING

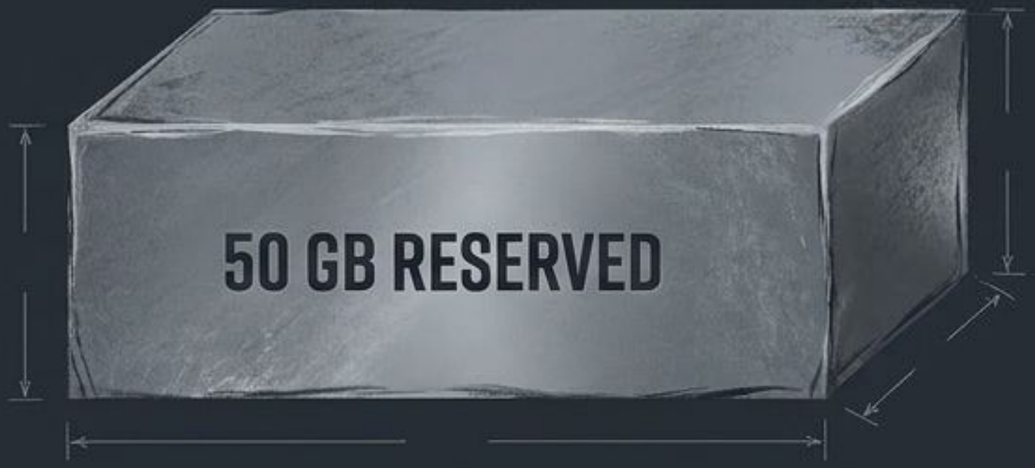
THIN PROVISIONING (DEFAULT)



The virtual disk starts small and **expands dynamically** as the Guest OS uses space.

Example: VMware allocates a 50 GB max .vmdk file, but if Ubuntu only uses 5 GB, your host disk **only loses 5 GB**. **Optimizes for host space.**

THICK PROVISIONING



The **entire allocated disk size is reserved** on the host hardware **immediately** upon creation. **Optimizes for performance stability.**

NotebookLM

Real-World Scenarios for Virtual Machines



Legacy Compatibility

Running outdated legacy applications on old OS versions without requiring obsolete physical hardware.



Cross-Platform Development

Seamlessly testing software across Windows, Linux, and macOS simultaneously on a single developer machine.



Security Sandboxing

Creating a quarantined environment to safely execute and analyze malware without risking the host system.

NotebookLM

Execution Architectures: VM vs. Dual Boot

Dual Boot Architecture



Requires exclusive hardware access; only one OS runs at a time.

Virtual Machine Architecture



Hypervisor shares hardware resources; multiple OSs execute simultaneously.

NotebookLM

THE MAGICIAN: UNDERSTANDING THE HYPERVISOR



The **Virtual Machine Monitor (VMM) or hypervisor** is the **highest-privileged** software layer.

Its **singular role** is to **emulate multiple copies** of the underlying **bare metal**.

Because it performs a **singular, focused task**, it operates with **100x fewer lines of code** than a standard Operating System, drastically reducing **bugs** and **potential vulnerabilities**.

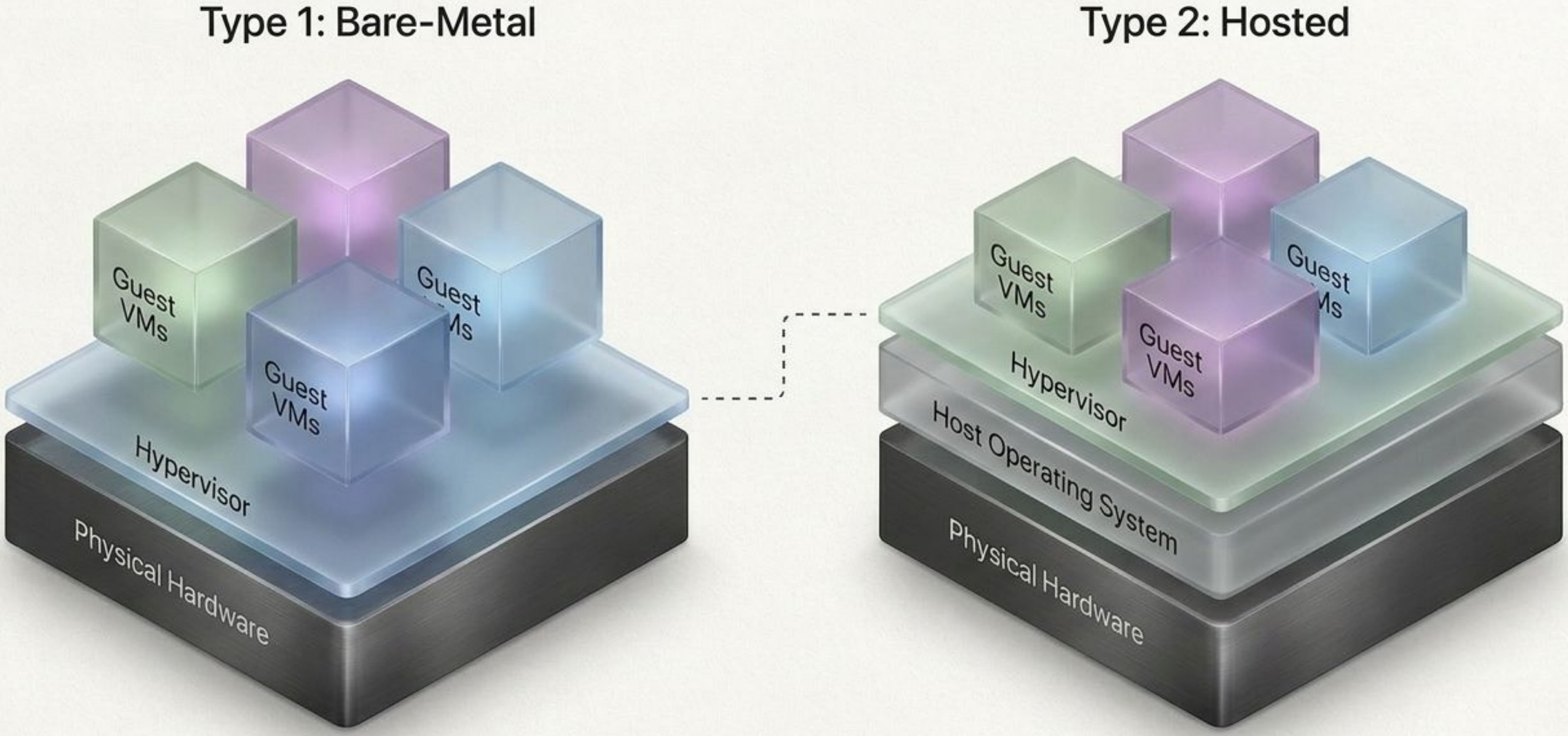
NotebookLM

DIAGNOSTIC COMPARISON: HYPERVISOR VS. OPERATING SYSTEM

	HYPERVISOR	OPERATING SYSTEM
PRIMARY ROLE	**Emulates** hardware architecture.	Provides **services** for applications.
RESOURCE MANAGEMENT	**Shares** physical hardware among multiple Guest OSes.	**Shares** physical hardware among multiple **processes.**
COMPLEXITY & SCOPE	**Minimal, isolated**, highly specific.	**Massive, intertwined, broad** (handles Scheduler, Memory Manager, File System, I/O).

NotebookLM

The Hypervisor Divide: Type 1 vs. Type 2



NotebookLM

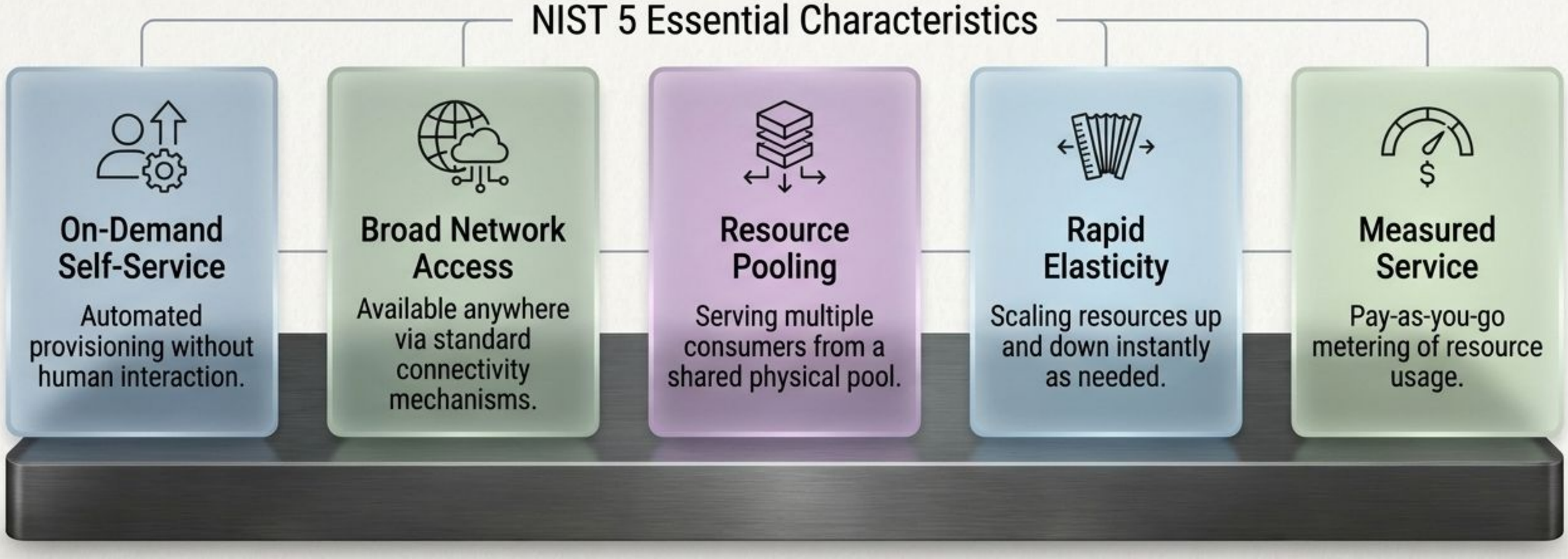
Hypervisor Comparison Matrix

	Type 1 (Bare-Metal)	Type 2 (Hosted)
Architecture	Runs directly on hardware	Runs as an application on a host OS
Performance	Minimal overhead (highest performance)	Moderate overhead (resource sharing with host)
Setup Complexity	Requires dedicated hardware and IT expertise	Installs easily like standard software
Primary Audience	Enterprise Data Centers and Cloud Infrastructure	Personal Labs, Testing, and Development
Industry Examples	VMware ESXi, Microsoft Hyper-V, Xen	VMware Workstation, Oracle VirtualBox

NotebookLM

Cloud Computing: Virtualization at Scale

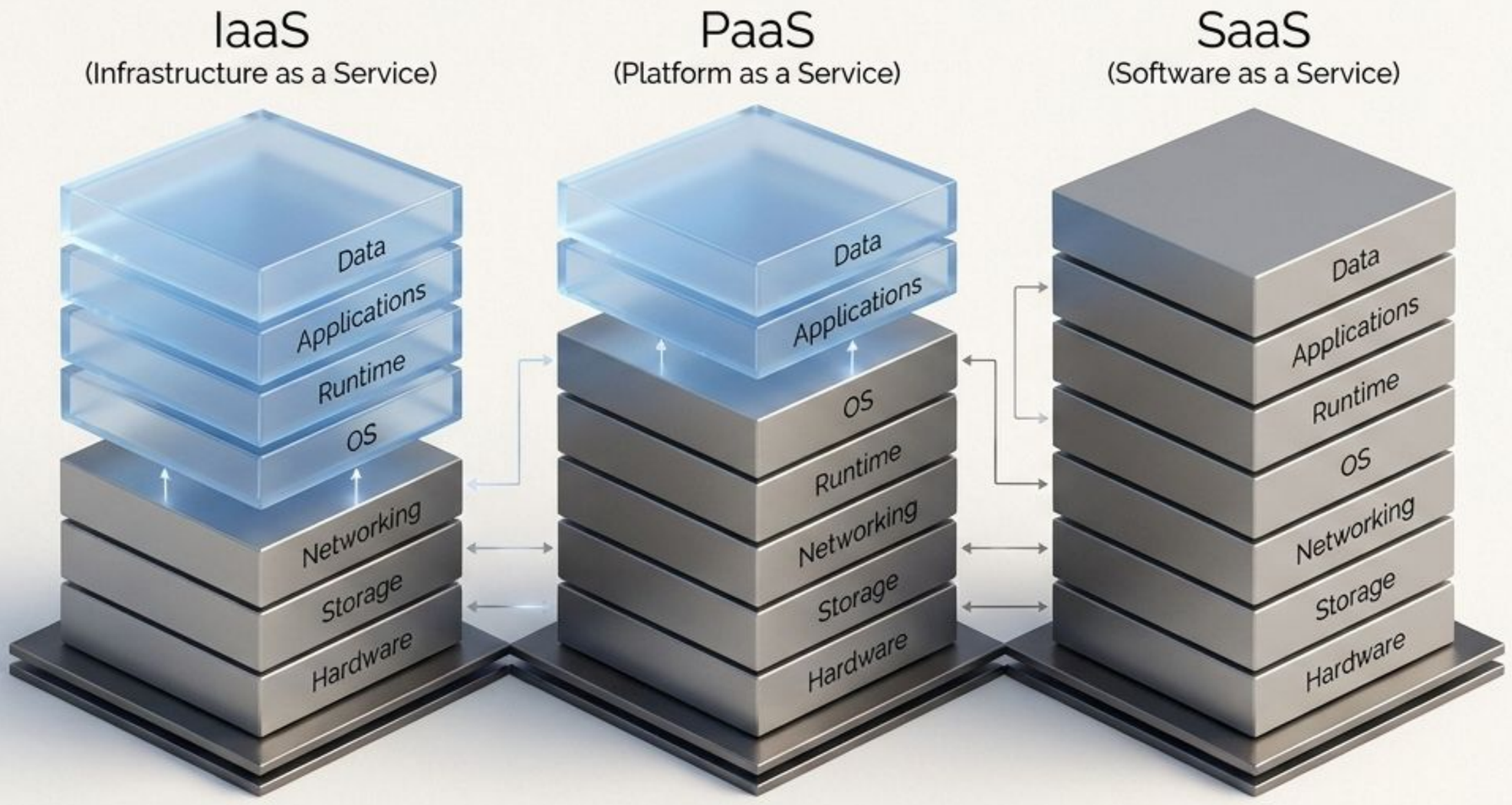
The delivery of computing, storage, and applications as a service over a network. Cloud computing is the ultimate realization of virtualized infrastructure.



NotebookLM

The Cloud Service Stack

Gray = Provider Responsibility
Blue Glass = User Responsibility



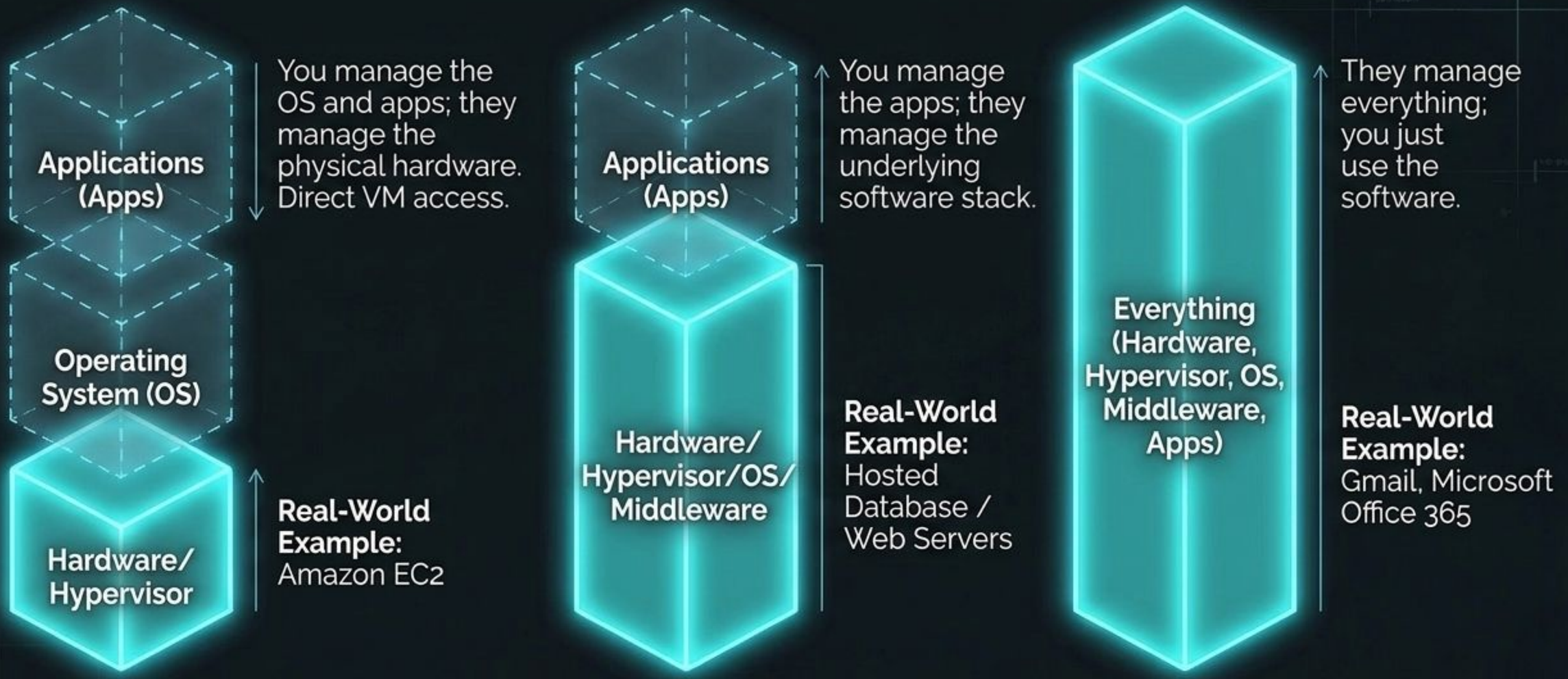
Direct access to VMs
(e.g., Amazon EC2)

A ready software stack
(e.g., database servers)

Specific accessible software
(e.g., Google Apps)

NotebookLM

THE CLOUD STACK: SERVICE MODELS



NotebookLM

The Reality of Cloud Infrastructure

Advantages



Economy of Scale

Massive cost efficiency achieved through shared global data centers.



Reduced Maintenance

Hardware failures are handled invisibly by the cloud provider.



Infinite Scalability

Instantly grow architecture from a single VM to a thousand.

Risks & Disadvantages



Vendor Lock-In

Systemic difficulty migrating architecture between competing cloud platforms.



Privacy Concerns

Inherent risk of entrusting sensitive organizational data to a third-party server.



Jurisdictional Issues

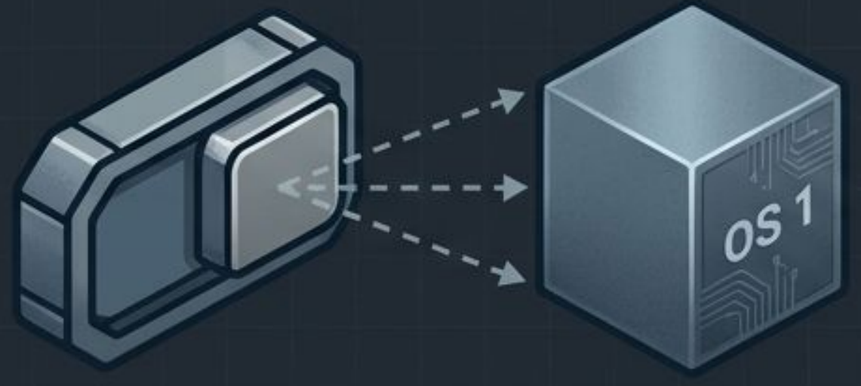
Complex legal compliance dictated by the physical location of the cloud hardware.

NotebookLM

Scenario 1: The Developer's Dilemma

The Problem: A developer needs to test an application across **10 different Operating Systems** by the end of the day.

The Question: Should they utilize a **Dual Boot** setup or **Virtual Machines**?



DUAL BOOT

Dual boot grants **exclusive control** to one OS at a time, **wasting hours** on constant reboots.



VIRTUAL MACHINES

VMs **share the CPU and RAM** simultaneously, allowing **instant, concurrent testing** across all 10 isolated environments.

The Engineering Justification: Virtual Machines win.

NotebookLM

Important Links and Docs related to this slide

Detailed and more information are in the docx with some problems, scenario type problem, some more details dive in cloud computing with examples.

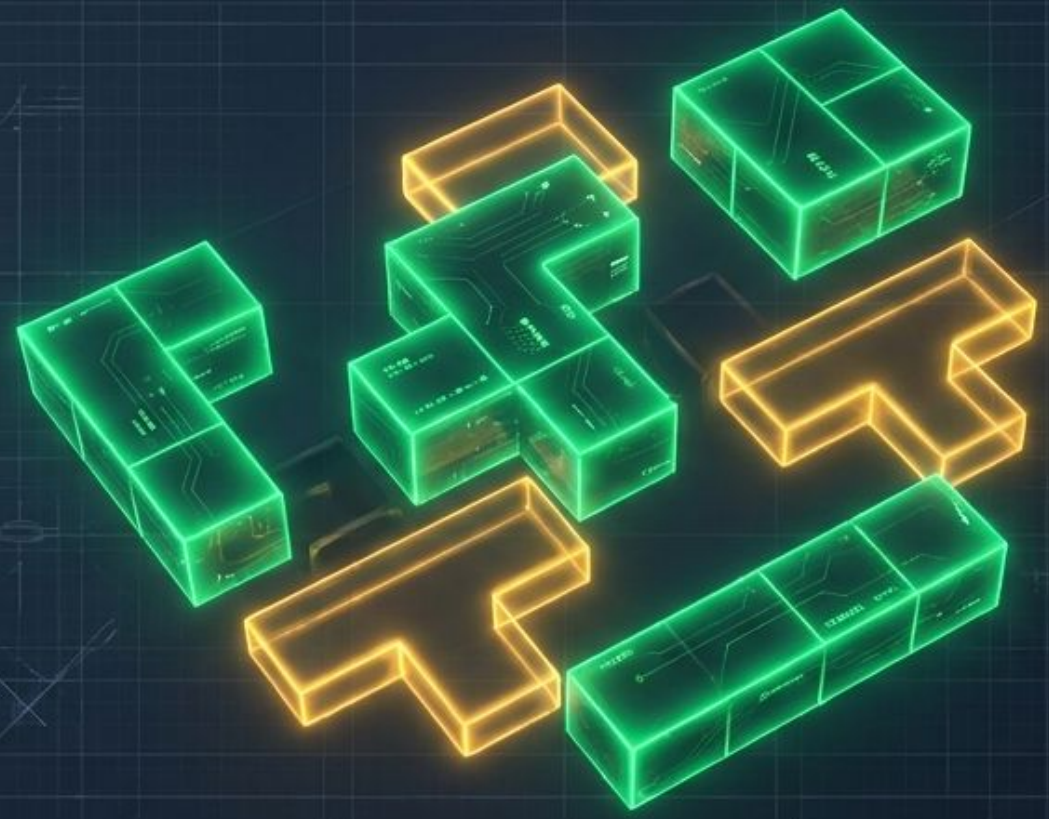
Doc link :
https://docs.google.com/document/d/1nLbOwCQ0kAQvTxDeXUtgIGi4lWczKiivQSpNiuoi26g/edit?tab=t.4u0b1dw3syj4

Make your first cloud server and start working with it :
https://youtu.be/YH_DVenJHII?si=rnjM4-tzRsSrbuFK


THANK
YOU

CSE-307 Spring 2026
Operating System
Topic : Memory Management in Operating Systems Part-1

- Lec Khaled Hasan



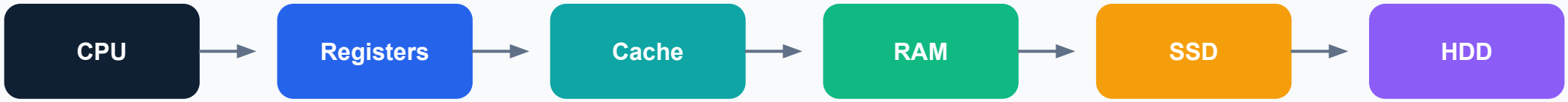
Operating System Memory Management

The Architecture of Spatial Allocation

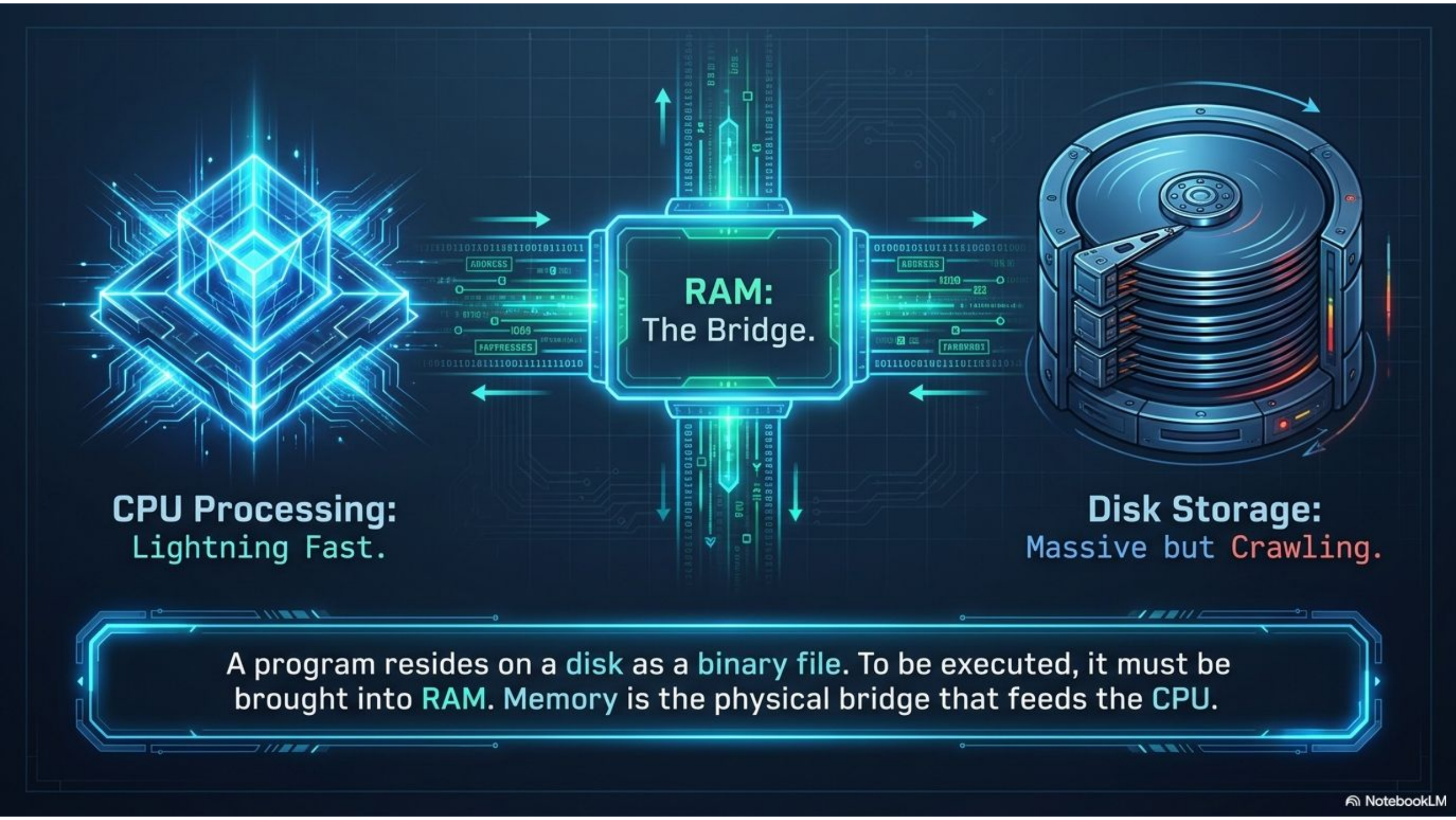
NotebookLM

Main contents :

- Memory hierarchy
- Multiprogramming
- Contiguous allocation
- Partitioning
- Fragmentation
- Placement algorithms



Key idea: The OS keeps the right data in the right memory at the right time.


RAM:
The Bridge.
CPU Processing:
Lightning Fast.
Disk Storage:
Massive but Crawling.
A program resides on a disk as a binary file. To be executed, it must be brought into RAM. Memory is the physical bridge that feeds the CPU.

NotebookLM

A scenario: many apps, limited RAM

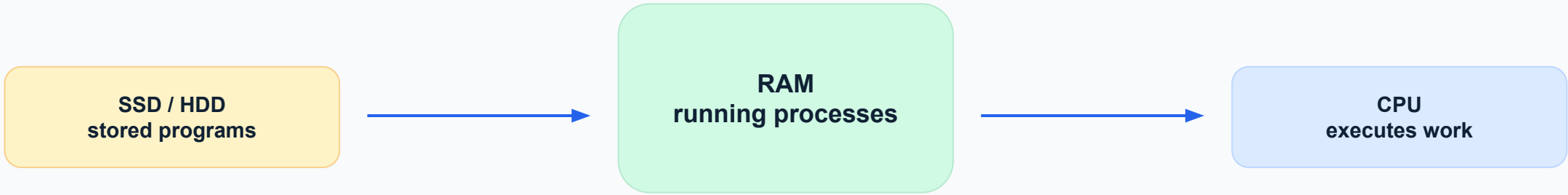
A normal PC is always negotiating between storage, RAM, and CPU work.

Chrome
Many tabs, scripts, cache

VS Code
Editor, extensions, terminal

Spotify
Streaming + buffers

Game
Textures + engine data

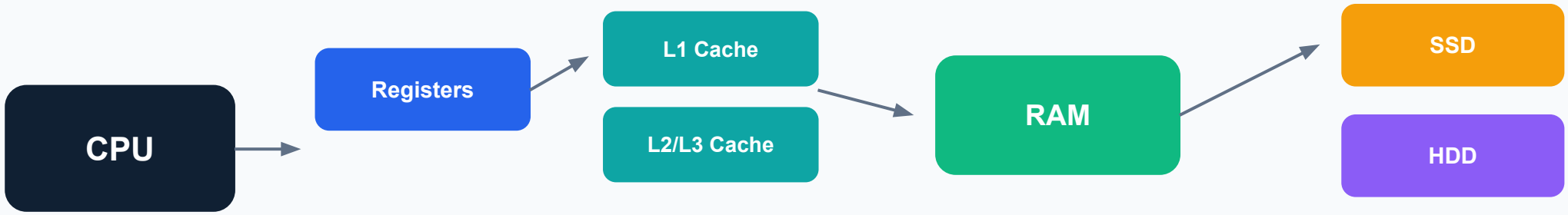


Key idea: Programs stay on disk until the OS loads their needed parts into main memory.

Architecture

CPU–memory architecture positioning

Memory is arranged around distance from the CPU and frequency of access.



Closest
Registers are inside the CPU and fastest.

Near
Cache stores recently used data.

Middle
RAM holds running processes.

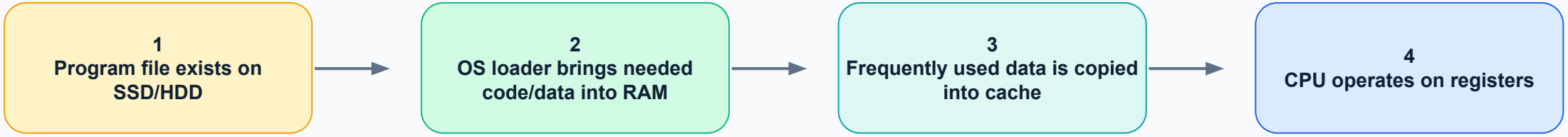
Far
Disk stores data persistently.

Key idea: The closer memory is to the CPU, the faster and usually smaller it is.

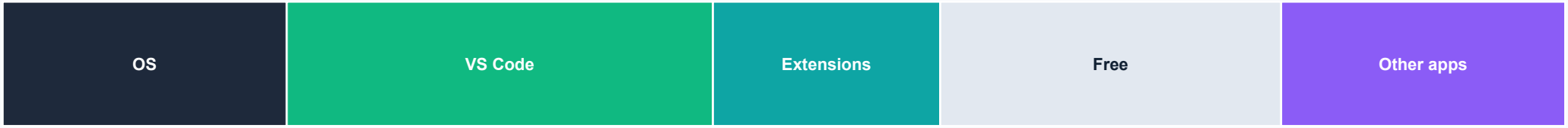
Architecture

How opening an app works

Example: launching VS Code from storage to CPU execution.



RAM after launching VS Code



Key idea: CPU execution depends on instructions and data being available through memory.

Memory Types

Types of memory present in a PC

Registers
fastest • very small
CPU operands

Cache
very fast • small
recent data

RAM
fast • GBs
running apps

ROM/Firmware
stable • small
boot code

Virtual memory
slower • disk-backed
RAM illusion

SSD
medium • large
apps/files

HDD
slower • very large
archives

USB/External
varies • portable
transfer/backup

Each memory type solves a different part of the speed–capacity–cost problem.

Memory Types

Volatile vs non-volatile memory

A simple power-off test separates temporary working memory from persistent storage.

Volatile memory

Data is lost when power is removed.
Examples: registers, cache, RAM.
Use: temporary working area for running programs.

Non-volatile memory

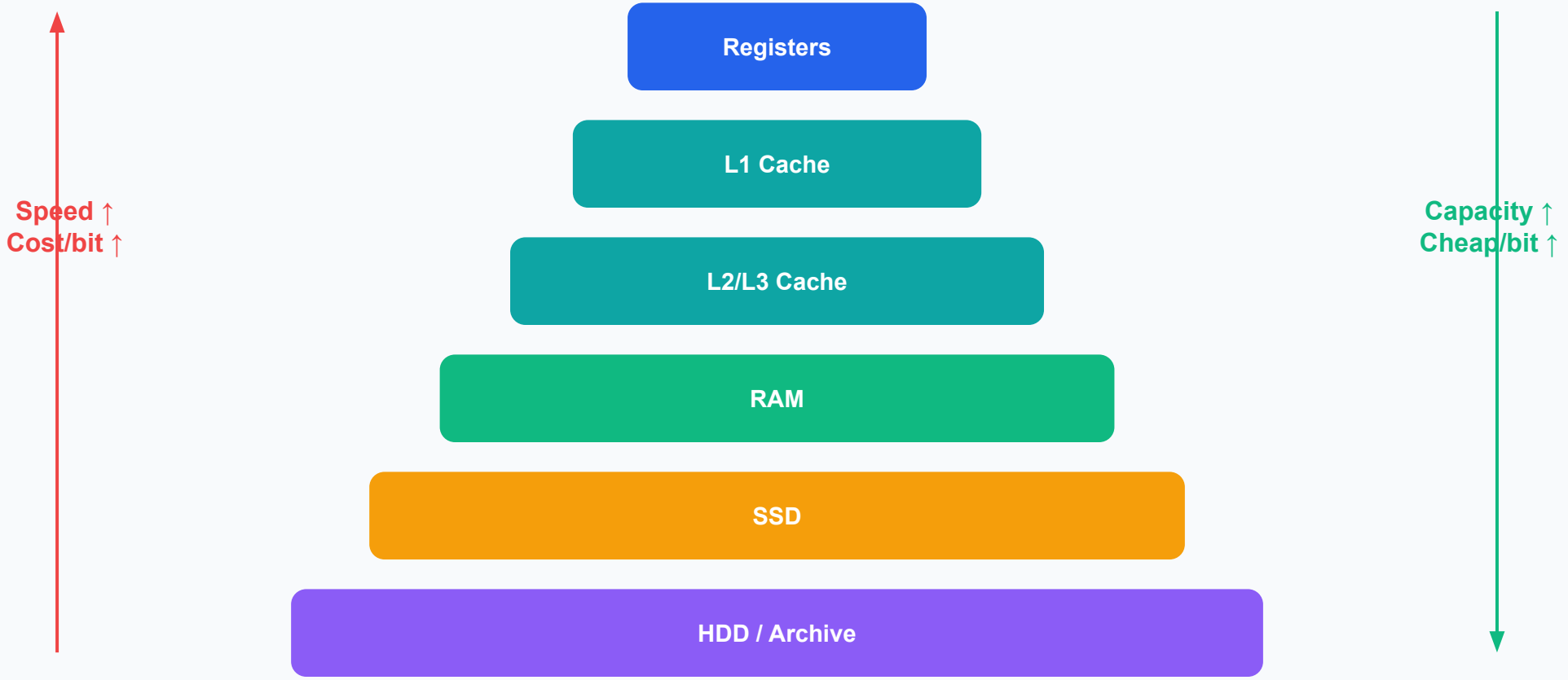
Data remains after shutdown.
Examples: ROM, SSD, HDD, USB.
Use: permanent program and file storage.

Power off test: RAM contents vanish, but saved files remain on SSD/HDD.

Hierarchy

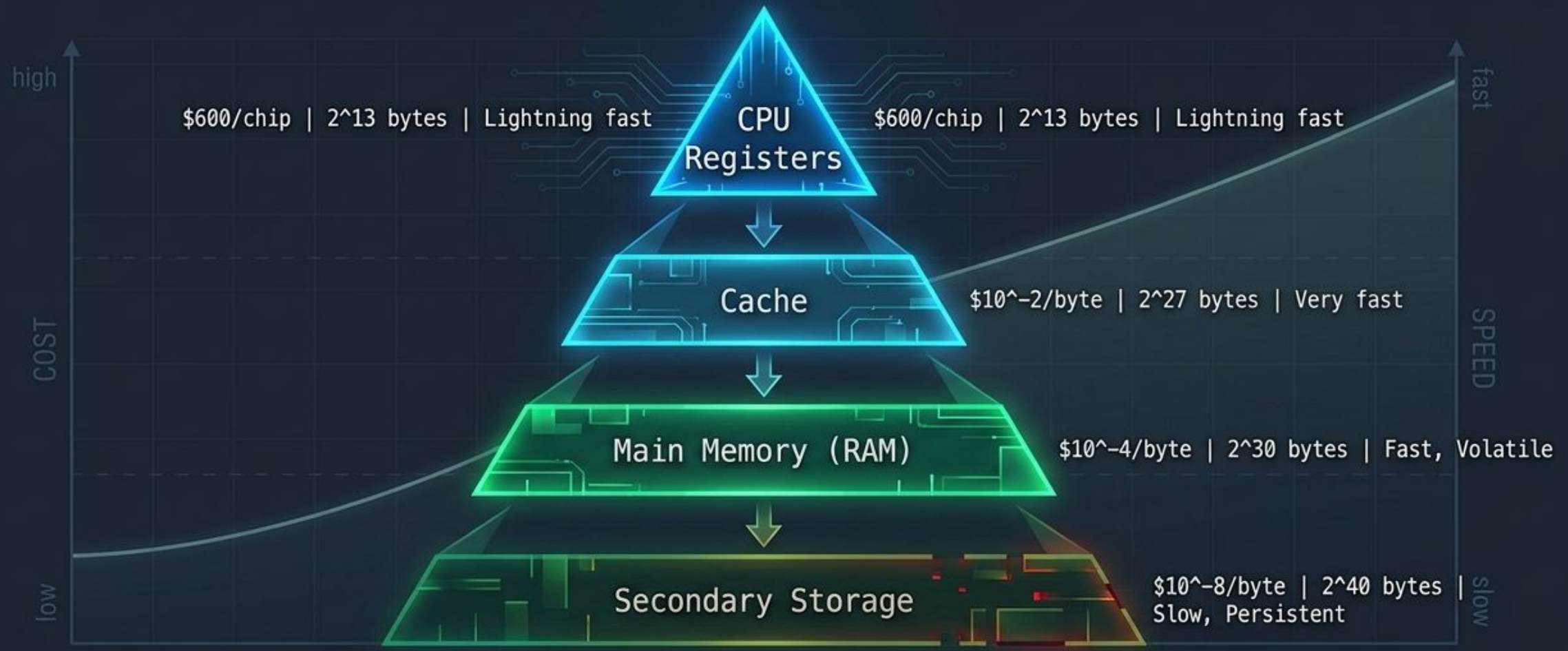
Memory hierarchy

Computer memory is arranged as a hierarchy, not as one perfect device.



Key idea: Higher levels are faster and smaller; lower levels are larger and cheaper.

THE MEMORY HIERARCHY: ARCHITECTURAL ABSTRACTION



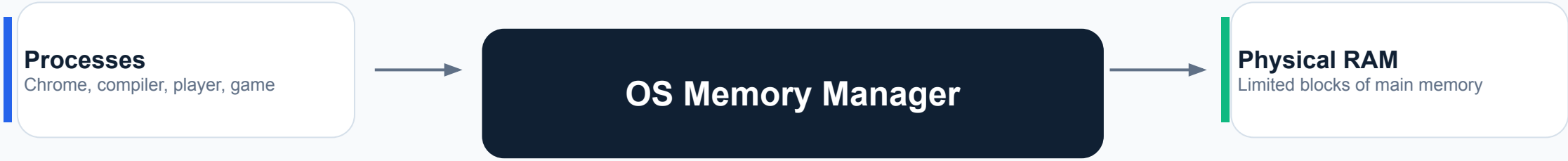
Programmers want memory that is private, large, fast, non-volatile, and cheap.
Physics says no. The OS must abstract this hierarchy into a usable model.

NotebookLM

OS Role

Goal of memory management in OS

The OS must make limited RAM feel safe, fair, and useful.



- Maximize RAM use
- Maximize CPU use
- Protect processes
- Reduce waste
- Support relocation
- Controlled sharing

Key idea: The OS allocates memory so running programs can progress without harming each other.

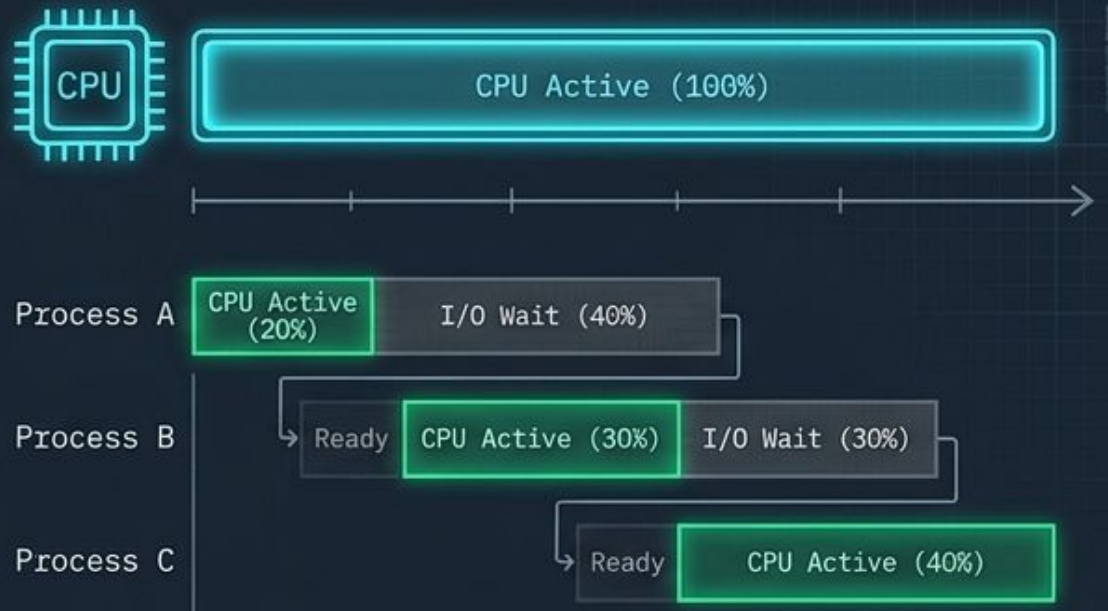
SINGLE-PROGRAMMING VS. MULTIPROGRAMMING

Single-Programming



One program means a starved CPU.

Multiprogramming



Degree of Multiprogramming (n)

The number of processes loaded into memory simultaneously to ensure the CPU is always fed.

NotebookLM

OS Role

Memory-management functions

The memory manager performs several jobs behind every running program.

Track
free vs used memory

Allocate
give RAM to processes

Deallocate
return memory after exit

Translate
logical → physical address

Relocate
move process if needed

Protect
isolate process memory

Share
controlled common memory

Swap
move between RAM and disk

Reduce waste
handle fragmentation

THE MEMORY MANAGER'S MANDATE

ALLOCATION

Assign RAM to processes as they arrive.



TRACKING

Monitor used space vs. free holes.



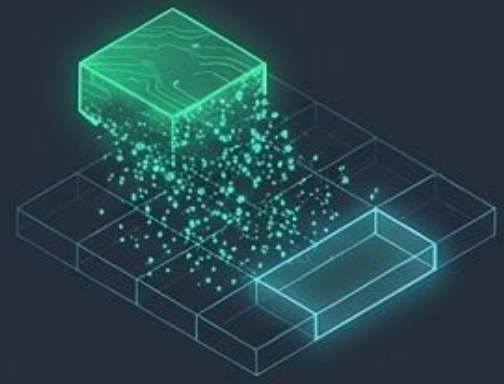
PROTECTION

Prevent processes from reading/writing outside their domain.



DEALLOCATION

Reclaim space when processes terminate.

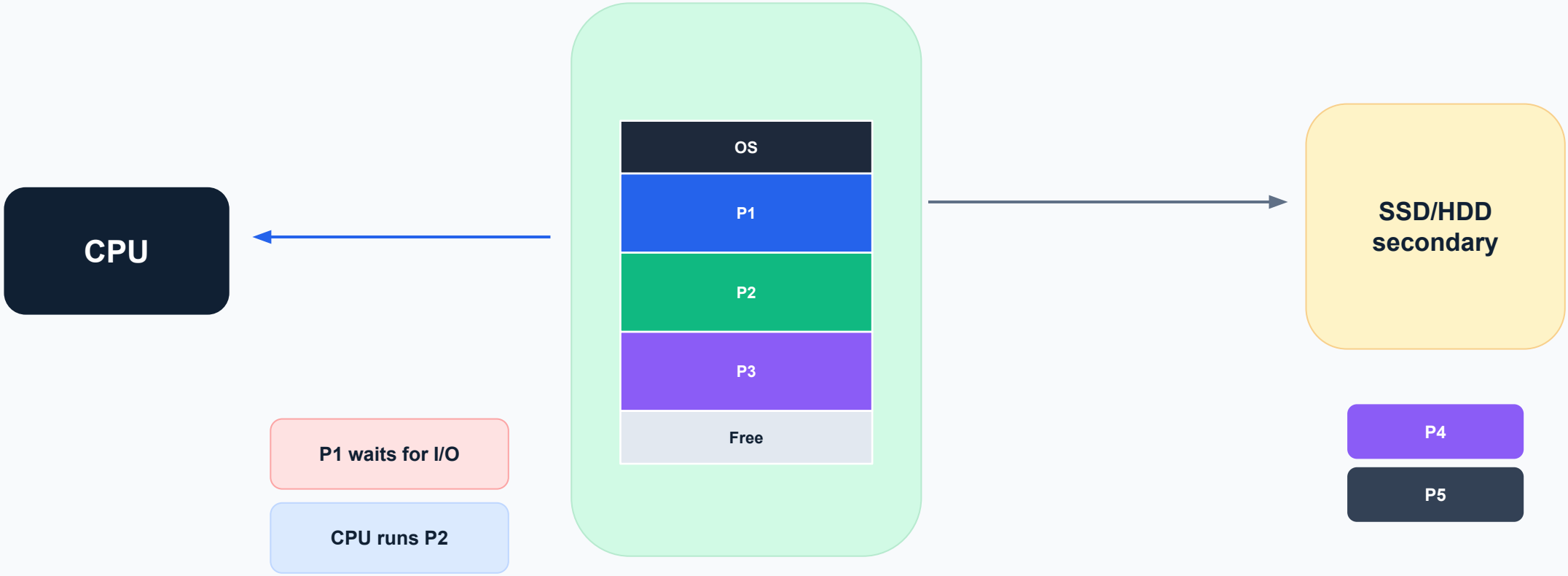


NotebookLM

Multiprogramming

Multiprogramming concept

Keep several processes in RAM so the CPU has useful work while one process waits.



Key idea: The goal is not only to fill RAM; the goal is to reduce CPU idle time.

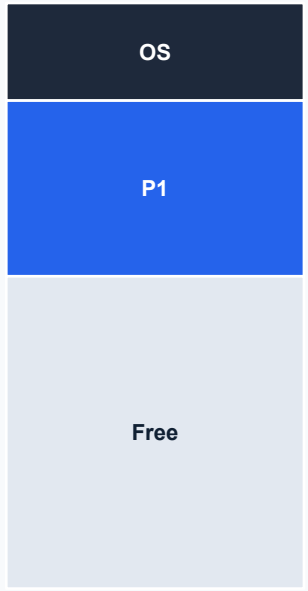
Multiprogramming

Degree of multiprogramming

A simple number: how many processes are resident in main memory.

Degree 1

RAM



Degree 2

RAM



Degree 4

RAM



Degree of multiprogramming = number of resident processes in RAM available for execution

Multiprogramming

Numerical model: CPU utilization

A simplified formula explains why more resident processes can help.

$$U = 1 - p^n$$

U
CPU utilization

p
Probability one process waits for I/O

n
Degree of multiprogramming

Assumption: I/O-wait events are independent and equally likely for each process.

Key idea: More resident processes reduce the chance that all processes are waiting at the same time.

Multiprogramming

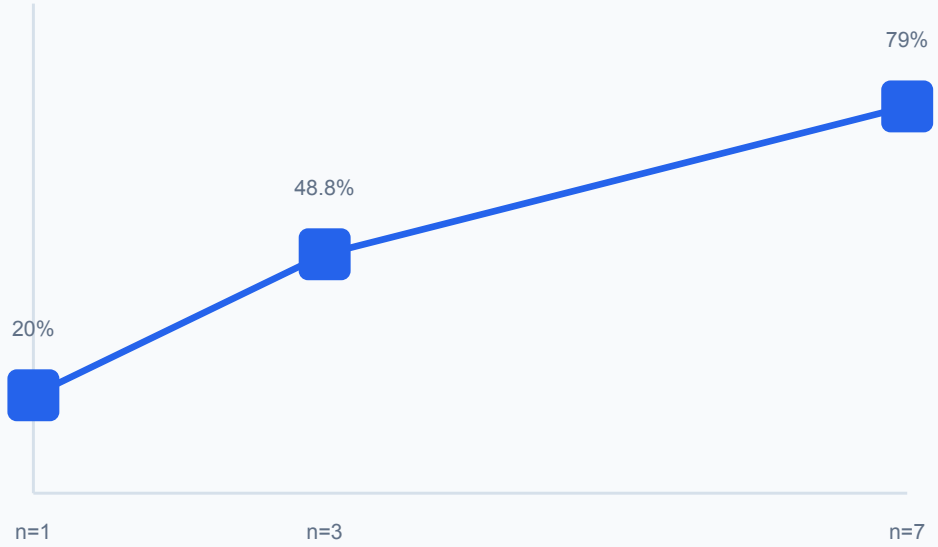
How increasing RAM may improve CPU utilization

More RAM can allow a higher degree of multiprogramming.

Given: p = 0.80, OS = 2 GB, each process = 2 GB

Total RAM	Processes n	Calculation	CPU Utilization
4 GB	1	$1 - 0.8^1$	20%
8 GB	3	$1 - 0.8^3$	48.8%
16 GB	7	$1 - 0.8^7$	≈ 79.0%

CPU utilization trend

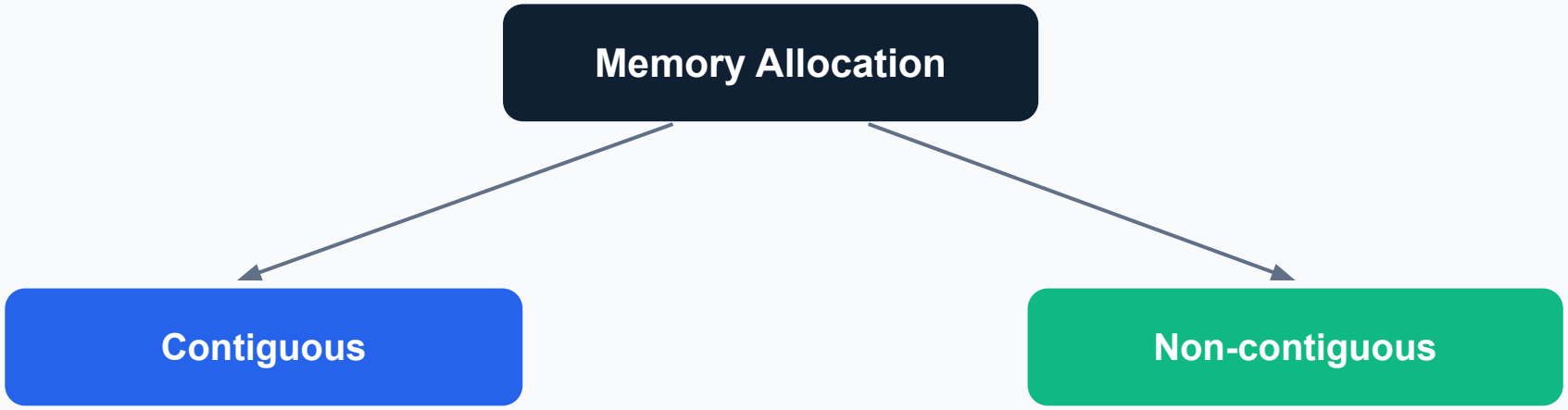


Key idea: More RAM may improve utilization, but it does not make the CPU clock faster.

Allocation Categories

Main categories of memory allocation

The first major division is based on physical placement of a process.



One continuous physical block
Fixed partitioning and dynamic partitioning

Separated physical blocks
Paging, segmentation, segmentation with paging

Allocation Categories

Contiguous vs non-contiguous placement

The same process can be placed as one block or divided into parts.

Contiguous allocation

Physical RAM



Contiguous basis

A whole process must fit in one continuous block. Easy to understand, but fragmentation matters.

Non-contiguous allocation

Physical RAM



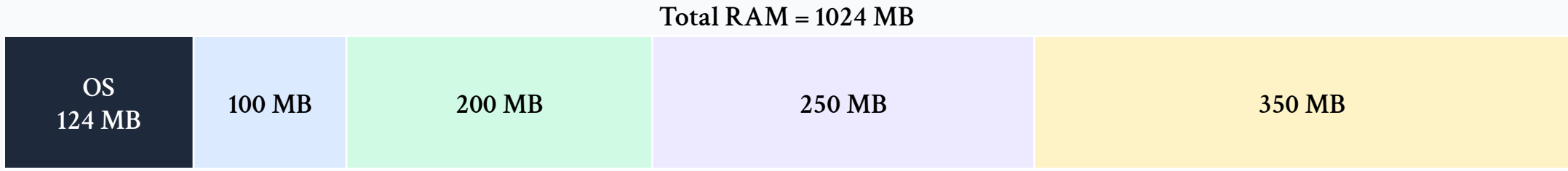
Non-contiguous basis

Different pieces of a process may be placed in separate memory locations. More flexible, but needs mapping.

Fixed Partitioning

Fixed partitioning

Main memory is divided into fixed partitions before processes arrive.



1
OS keeps its reserved area

2
Remaining RAM is split beforehand

3
One process normally goes into one partition

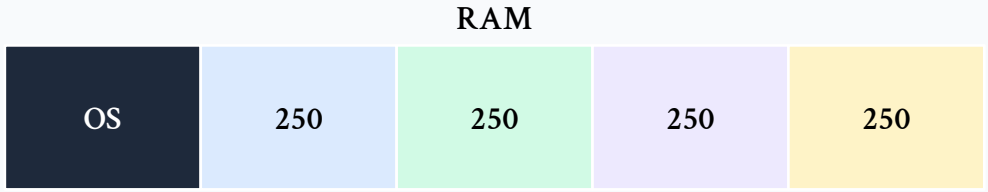
4
Partition becomes free when process exits

Key idea: Fixed partition sizes are chosen before allocation, so flexibility is limited.

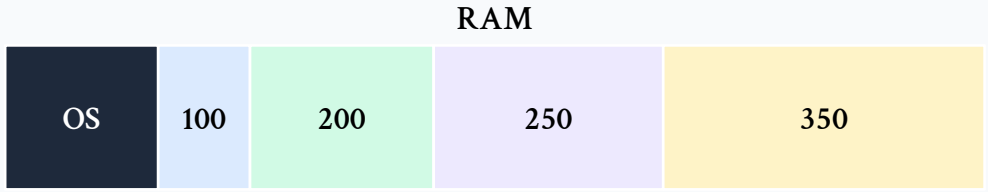
Equal-size and unequal-size fixed partitions

Fixed partitioning can be simple, but matching process sizes is difficult.

Equal-size partitions



Unequal-size partitions



Equal-size

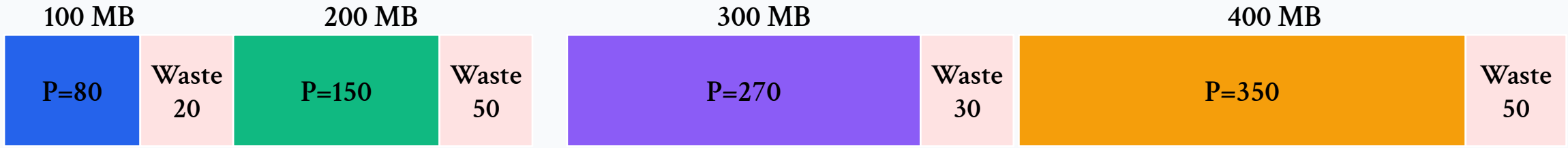
Easy to manage, but small processes waste space and large processes may not fit.

Unequal-size

Can fit different process sizes better, but still fixed and inflexible.

Internal fragmentation

Waste inside an allocated partition.



Partition	Process	Internal fragmentation
100 MB	80 MB	20 MB
200 MB	150 MB	50 MB
300 MB	270 MB	30 MB
400 MB	350 MB	50 MB

Total waste = 150 MB

Key idea: Internal fragmentation exists inside allocated memory blocks.

Spanning and non-spanning

Can one process cross partition boundaries?

Non-spanning

A process must fit completely inside one partition.

Two free partitions



350 MB process fails

Spanning

A process is split across more than one partition.

Conceptual split



Possible, but complex

Fixed Partitioning

Fixed partitioning: pros and cons

Simple management, but poor flexibility.

Advantages

- Simple to understand
- Fast allocation and deallocation
- Low management overhead
- Predictable memory layout
- Easy implementation

Limitations

- Internal fragmentation
- Fixed number of resident processes
- Large processes may not fit
- Partition sizes are inflexible
- Poor RAM utilization

Dynamic Partitioning

Dynamic / variable partitioning

Partitions are created according to the sizes of arriving processes.



Key idea: Dynamic partitioning reduces fixed-size waste, but holes appear over time.

Dynamic Partitioning

Concept of a hole

A hole is a free contiguous memory block available for allocation.

RAM with holes



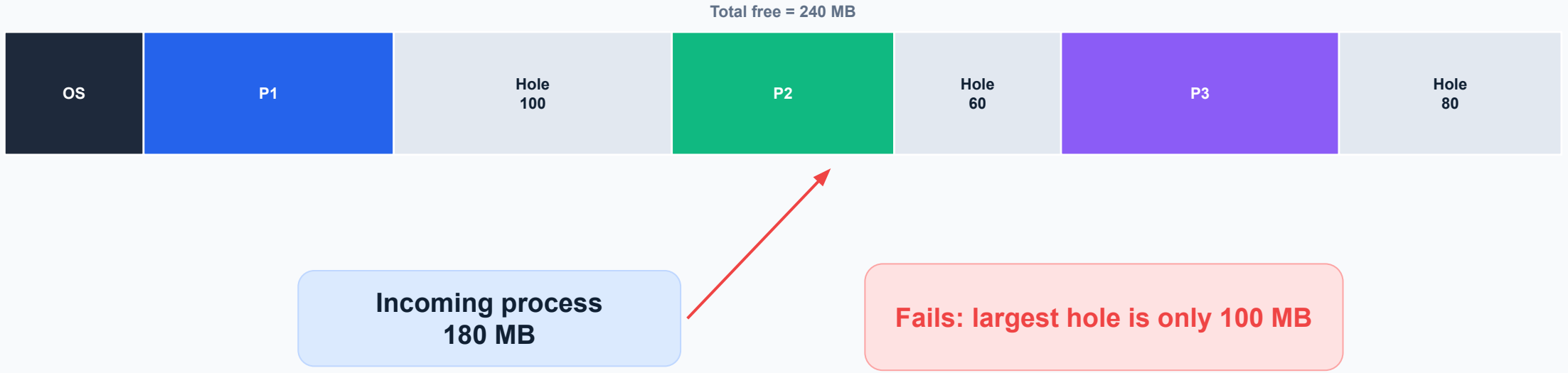
Hole list	Size
H1	120 MB
H2	80 MB
H3	60 MB

Key idea: The OS searches the hole list when a new process requests memory.

Dynamic Partitioning

External fragmentation

Enough total free memory exists, but not as one large contiguous hole.



Internal fragmentation
Waste inside allocated blocks.

External fragmentation
Free space scattered between allocated blocks.

Dynamic Partitioning

Dynamic partitioning: pros and cons

More flexible than fixed partitioning, but fragmentation shifts outside.

Advantages

- Partition size matches process size
- Less internal fragmentation
- More flexible than fixed partitioning
- No fixed partition count
- Better initial RAM utilization

Limitations

- External fragmentation
- Allocation search overhead
- Free-list management required
- Compaction may be needed
- Slower and more complex over time

Placement Algorithms

Placement algorithms for dynamic partitioning (VERY IMPORTANT ******

When several holes are available, which one should the OS choose?

First Fit
First hole large enough

Next Fit
Continue from last search point

Best Fit
Smallest hole large enough

Worst Fit
Largest available hole

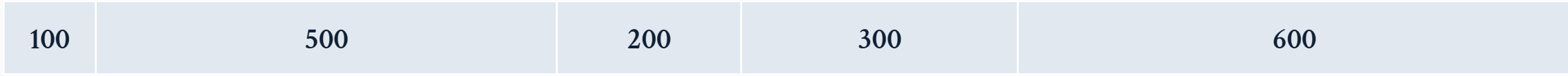
No algorithm is always best; performance depends on workload and request order.

Placement Algorithms

Numerical example: First Fit

Holes: 100, 500, 200, 300, 600 KB. Processes: 212, 417, 112, 426 KB.

Initial holes



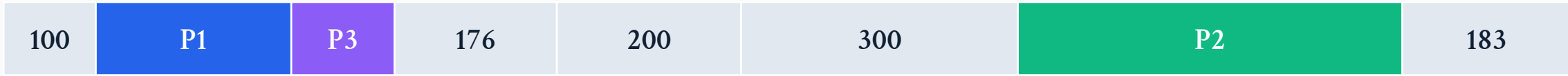
P1=212
500 → P1, rem 288

P2=417
600 → P2, rem 183

P3=112
288 → P3, rem 176

P4=426
Fails: no hole ≥ 426

After First Fit



Key idea: First Fit restarts scanning from the beginning for each request.

Placement Algorithms

Numerical example: Next Fit

Same holes and processes, but searching resumes from the previous allocation point.

Assumption: the remainder of the last-used hole is considered first on the next request.

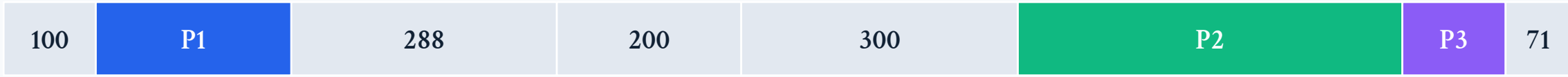
P1=212
500 → P1, rem 288; pointer here

P2=417
Continue → 600, rem 183

P3=112
Continue from 183 → allocate there, rem 71

P4=426
Wrap around; no hole ≥ 426 → fail

After Next Fit



Key idea: Next Fit may skip useful earlier holes until the search wraps around.

Placement Algorithms

Best Fit and Worst Fit

Two opposite strategies: smallest suitable hole vs largest hole.

Best Fit

P1=212
300 → rem 88

P2=417
500 → rem 83

P3=112
200 → rem 88

P4=426
600 → rem 174

After Best Fit



Worst Fit

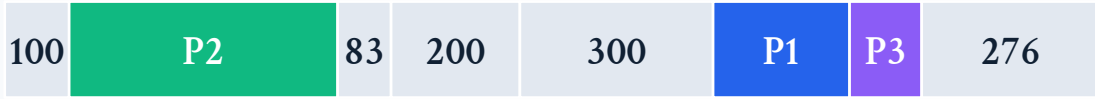
P1=212
600 → rem 388

P2=417
500 → rem 83

P3=112
388 → rem 276

P4=426
Fails

After Worst Fit



Best Fit tendency

Preserves large holes but may create many tiny holes.

Worst Fit tendency

Leaves large remainders, but may waste the biggest useful holes.

Placement Algorithms

Algorithm comparison

The best choice depends on process sizes and request order.

Algorithm	Search idea	Strength	Limitation
First Fit	First sufficient hole	Fast + simple	Small holes near start
Next Fit	Resume from pointer	Spreads searching	May miss earlier holes
Best Fit	Smallest sufficient hole	Preserves large holes	Tiny holes
Worst Fit	Largest hole	Larger remainders	Consumes big holes

Practice

Some practices & Tasks: predict the allocation

Solve manually before showing the answer.

Holes: 120 KB, 400 KB, 250 KB, 600 KB

Processes: 115 KB, 230 KB, 390 KB, 200 KB

Task 1

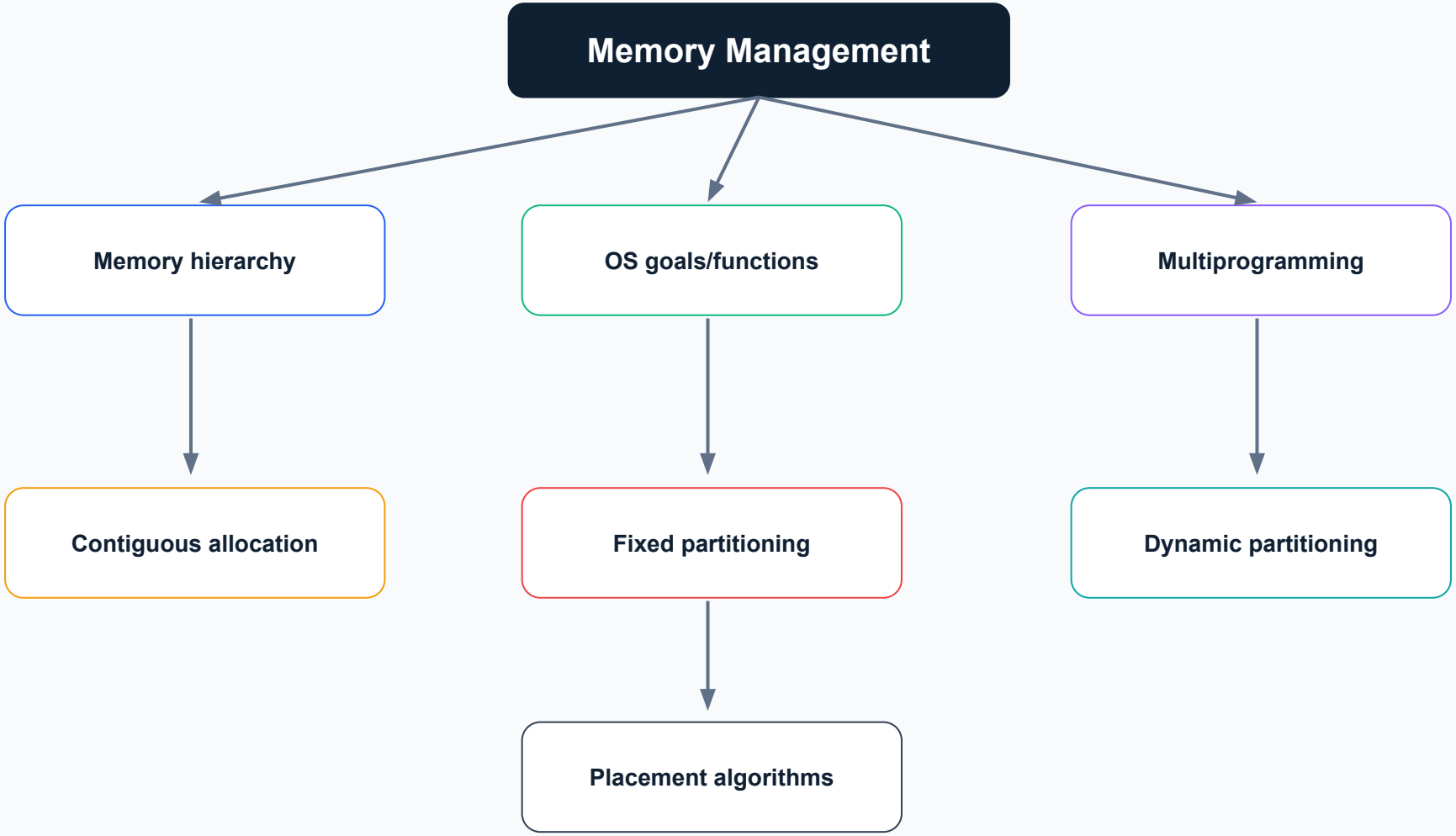
Apply First Fit step by step.

Task 2

Apply Best Fit step by step.

Wrap-up

Summary



Fixed Partitioning (MFT - Multiprogramming with a Fixed number of Tasks)

CORE MECHANISM

- The OS divides memory into **static**, **rigid blocks** at system boot.
- One **partition** = **One process**.
- Requires a simple **integer queue** to drop processes into the next available **slot**.

VISUAL METAPHOR

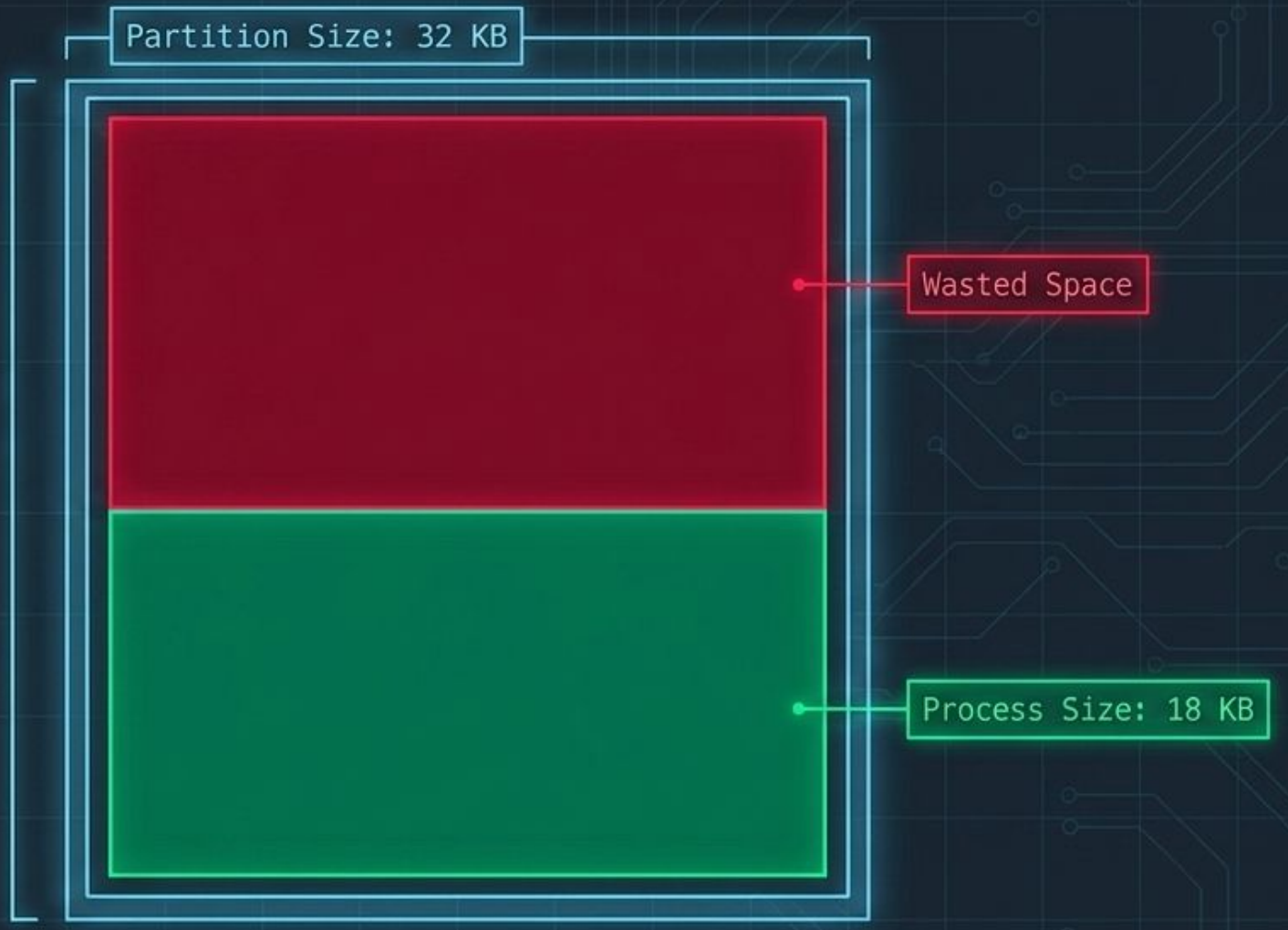
The Hotel: Every room is exactly the same size, regardless of whether a single traveler or a family checks in.



NotebookLM

The Flaw of MFT: Internal Fragmentation

Calculation:
Partition (32) – Process (18)
= Internal Fragmentation (14)



Space wasted inside an allocated partition because the partition is larger than the process. It is trapped and cannot be used by any other program.

NotebookLM

Dynamic Partitioning (MVT - Multiprogramming with a Variable number of Tasks)



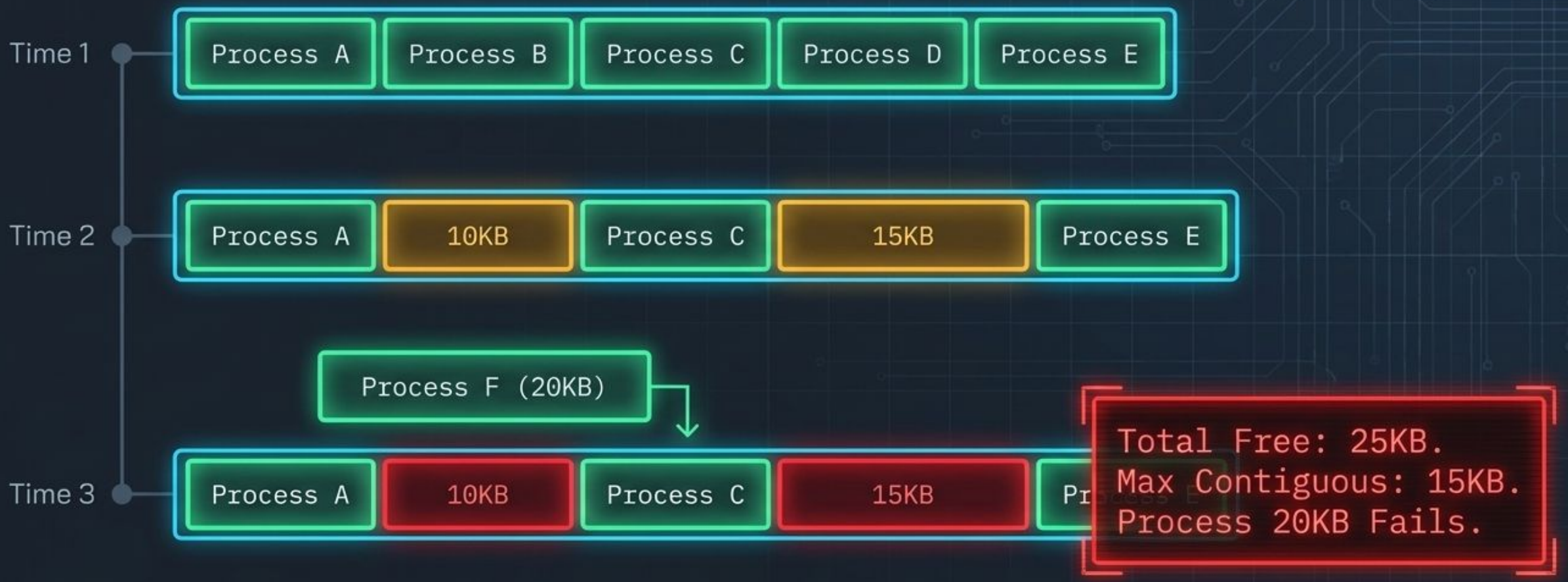
CORE MECHANISM

- Partitions are created dynamically to be the exact size of the arriving process.
- When a process leaves,it creates a Hole.

Hole - A contiguous block of free, unallocated memory available to house new processes.

NotebookLM

The Flaw of MVT: External Fragmentation



Total memory space exists to satisfy a request, but it is not contiguous.
The free space is shattered into unusable fragments between active processes.

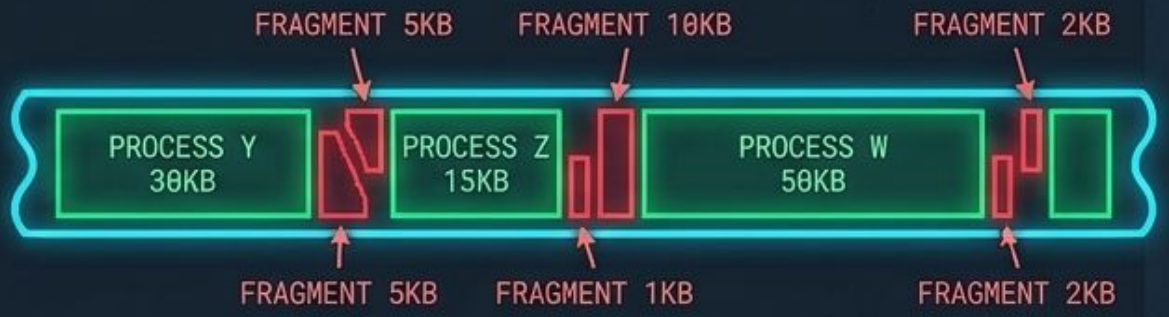
NotebookLM

MFT & Internal Fragmentation



The box is too big.

MVT & External Fragmentation



The empty space is shattered.

Engineering Truth: Fixed partitions waste memory internally. Dynamic partitions shatter memory externally. How do we slow down the shattering? Better placement algorithms.

NotebookLM

The Placement Algorithms

1. First Fit

Allocate the first hole that is big enough.

(Fastest, minimal searching)

2. Next Fit

First Fit, but starts scanning from the location of the last placement.

(Distributes wear and tear across memory)

3. Best Fit

Allocate the smallest hole that is big enough.

(Produces the smallest leftover hole, but requires searching the whole list)

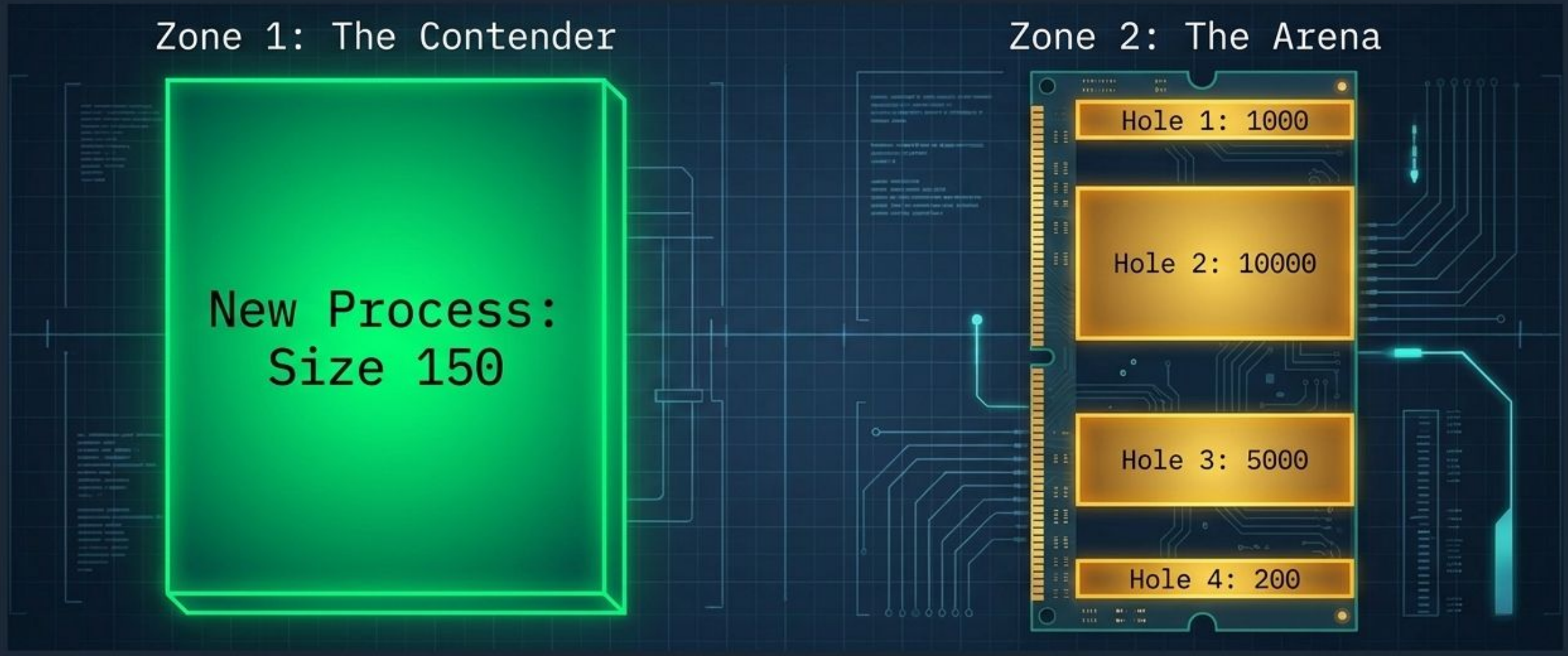
4. Worst Fit

Allocate the largest hole available.

(Produces the largest leftover hole, which might be useful for another process)

NotebookLM

The Simulation Setup

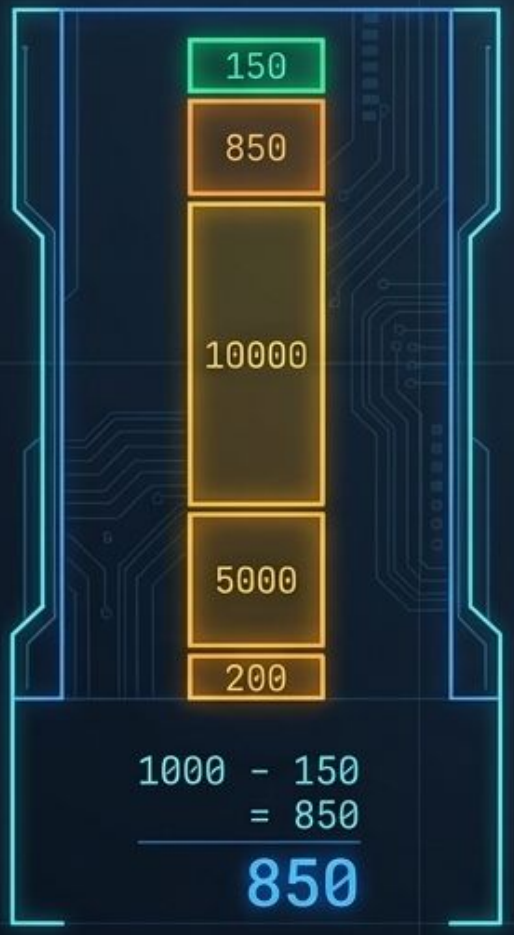


The Mission: Which hole does each algorithm select, and how much fragmented space is left behind?

NotebookLM

The Execution Trace

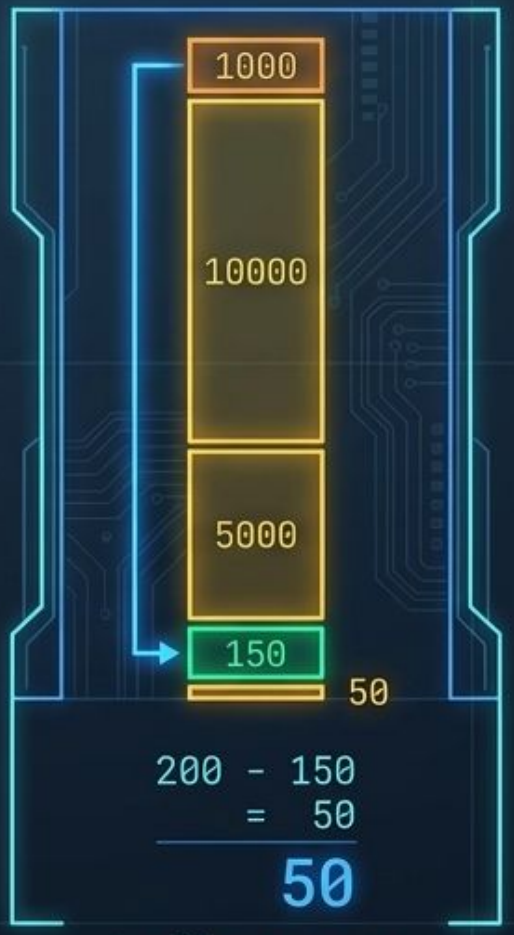
First Fit



Next Fit



Best Fit



Smallest waste

Worst Fit



Largest remaining usable block

NotebookLM

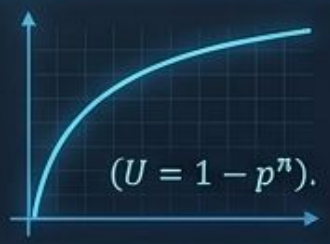
THE BOTTLENECK

We needed RAM to bridge the speed gap between CPU and Disk.



THE IMPERATIVE

We needed to tightly pack RAM to satisfy the multiprogramming curve $(U = 1 - p^n)$.



THE ARCHITECTURE

We moved from Fixed Partitions (Internal Waste) to Dynamic Partitions.



THE TACTICS

We deploy Search Algorithms (Best/Worst/First Fit) to slow down External Fragmentation.



Memory management is not just storage—it is the continuous algorithmic optimization of space and time.

NotebookLM

Important Links and Docs related to this slide

Detailed and more information are in the docx with some problems, scenario type problem, some more details dive in memory management in OS.

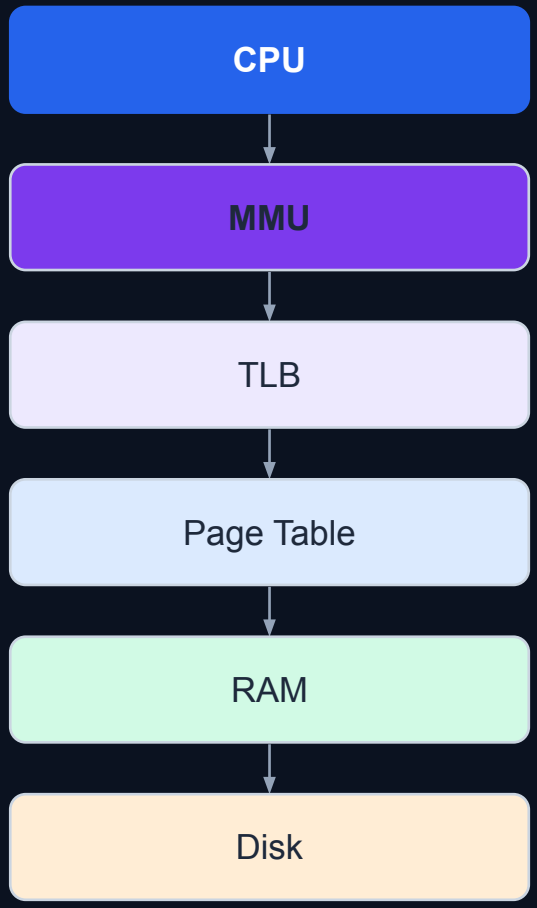
Doc link :
https://docs.google.com/document/d/1nLbOwCQ0kAQvTxDeXUtgIGi4lWczKiivQSpNiuoi26g/edit?tab=t.hcemc32waizh


THANK
YOU

CSE-307 Operating System

Memory Management Part 2

Non contiguous allocation

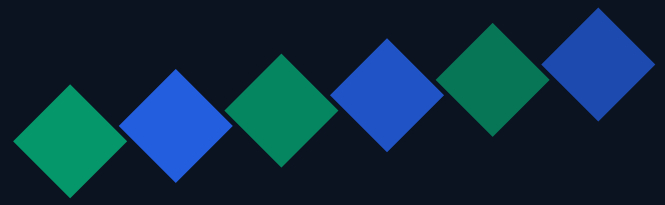


Lec Khaled Hasan · Spring 2026

PART 2

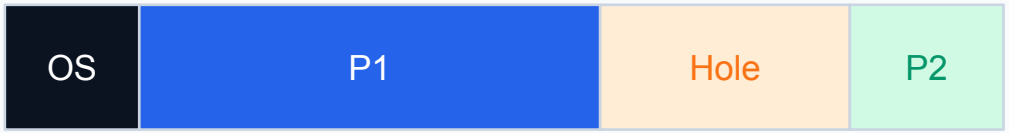
Non-Contiguous Memory Allocation

From scattered free memory to address translation



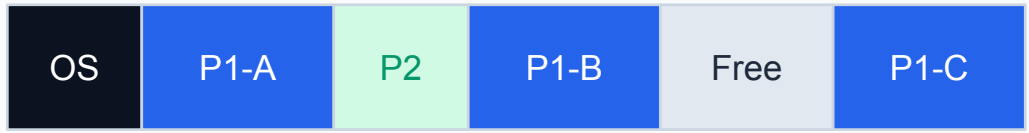
Where Part 1 ended

Contiguous allocation



One process needs one continuous physical block

Non-contiguous allocation



Pieces of a process may be placed separately, using mapping

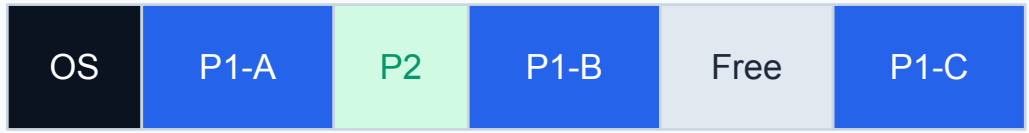
Key idea: Contiguous allocation fails when a process cannot find one large continuous hole.

What is non-contiguous allocation?

Definition

A process is divided into smaller parts.
The parts do not need adjacent physical locations.
The OS keeps a mapping table to find each part.

Non-contiguous placement



P1 appears continuous logically, but its pieces are scattered physically.

Key idea: Flexibility increases, but address mapping becomes essential.

Why it matters today

Large apps
Only active parts need RAM

Many processes
RAM is shared flexibly

Protection
Each process gets its own view

Efficiency
Scattered frames can still be useful

Key idea: Modern memory management is largely mapping-based, not placement-only.

Two major non-contiguous ideas



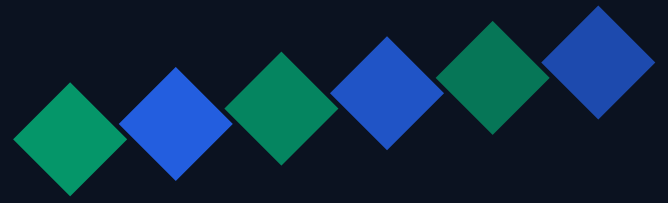
Both need tables to translate CPU-generated logical addresses.

Key idea: Non-contiguous allocation changes the problem from “where can it fit?” to “how do we map it?”

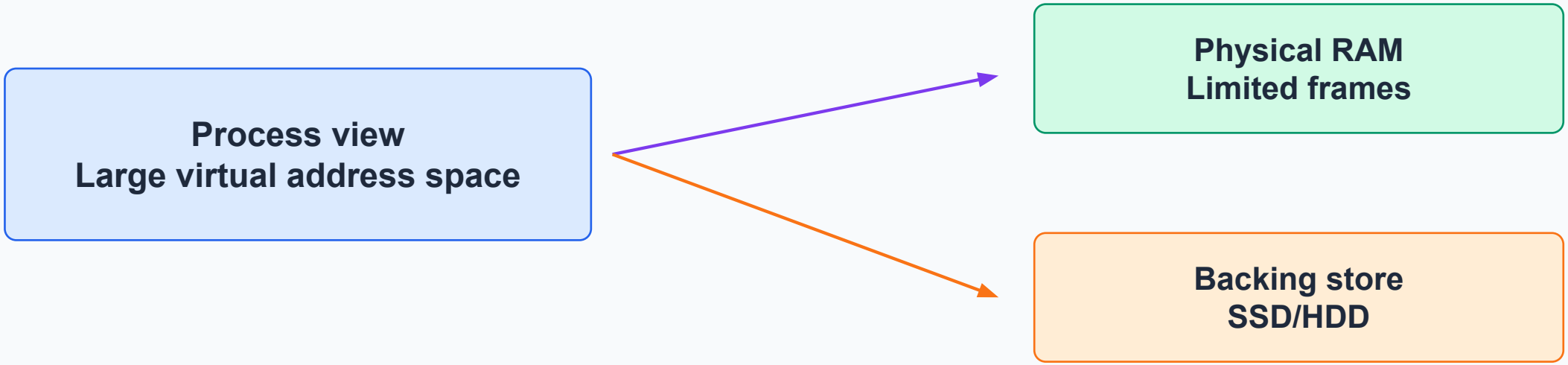
SECTION A

Virtual Memory

A large private memory illusion built over limited RAM



Virtual memory: the basic idea

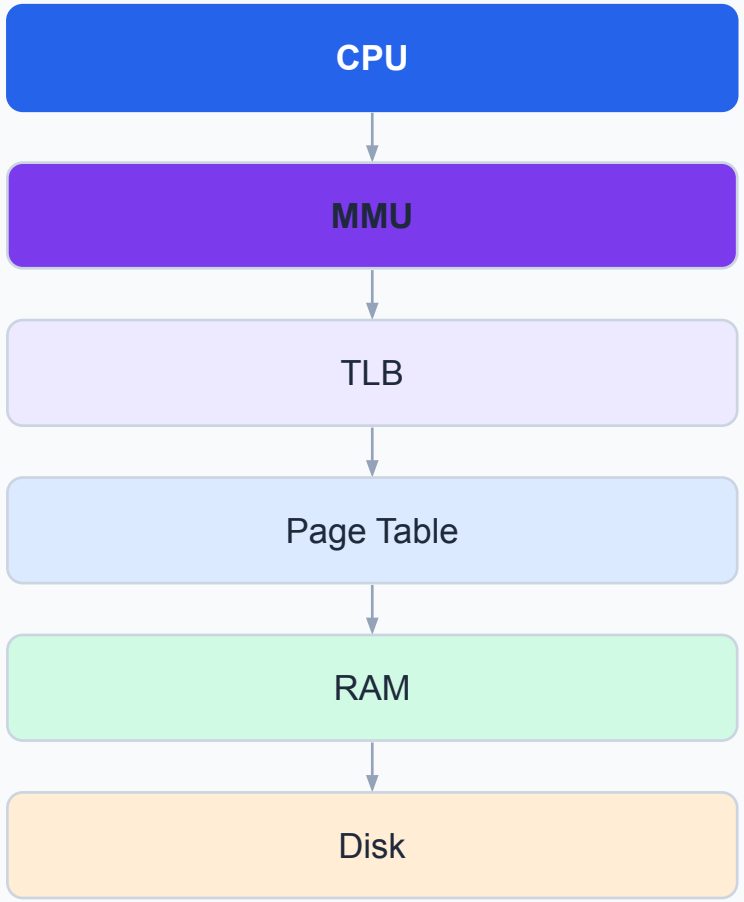


OS + MMU decide where each part actually resides.

Key idea: Virtual memory separates the address a process uses from the physical location used by RAM.

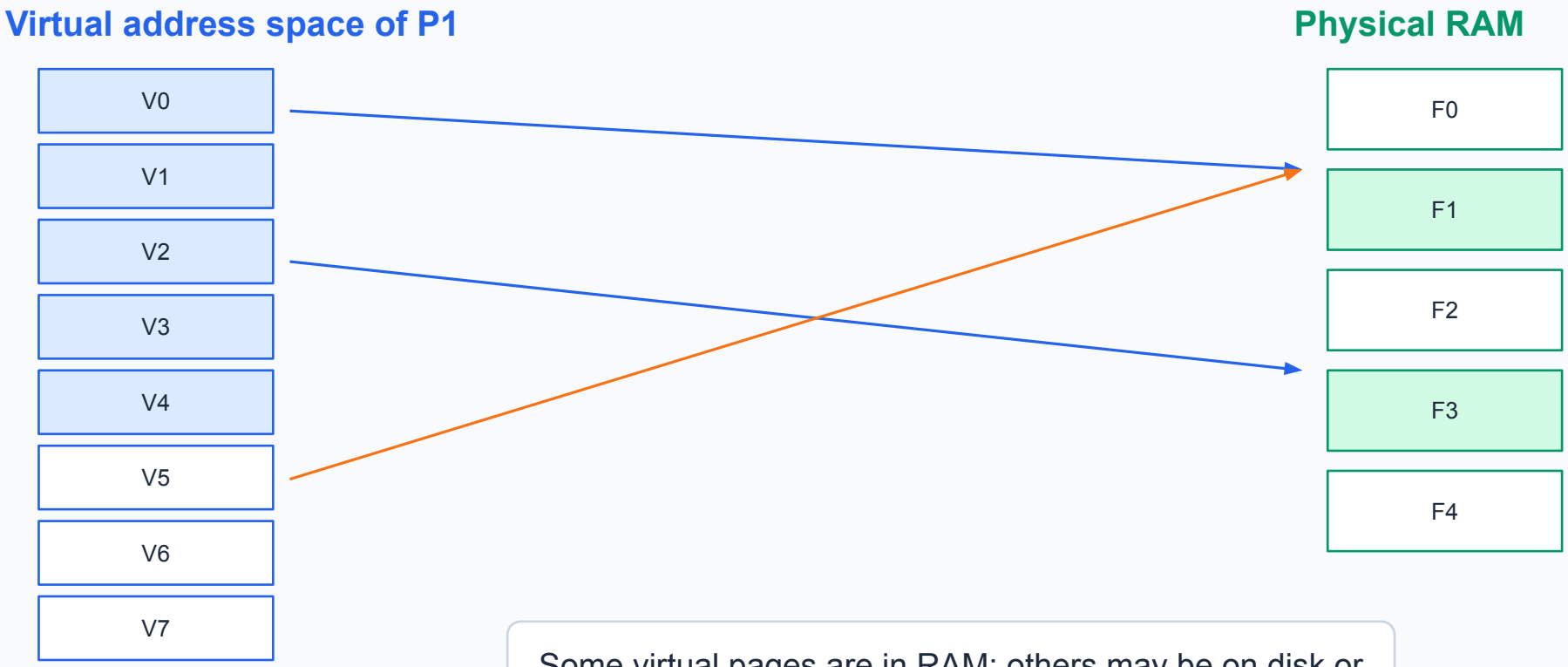
Why do we need virtual memory?

Programs can be larger than available physical RAM.
Only actively used portions need to be resident.
Several processes can share RAM safely.
Each process can believe it has a private address space.
Inactive pages can stay on disk until needed.



Key idea: Virtual memory improves convenience, protection, and RAM utilization.

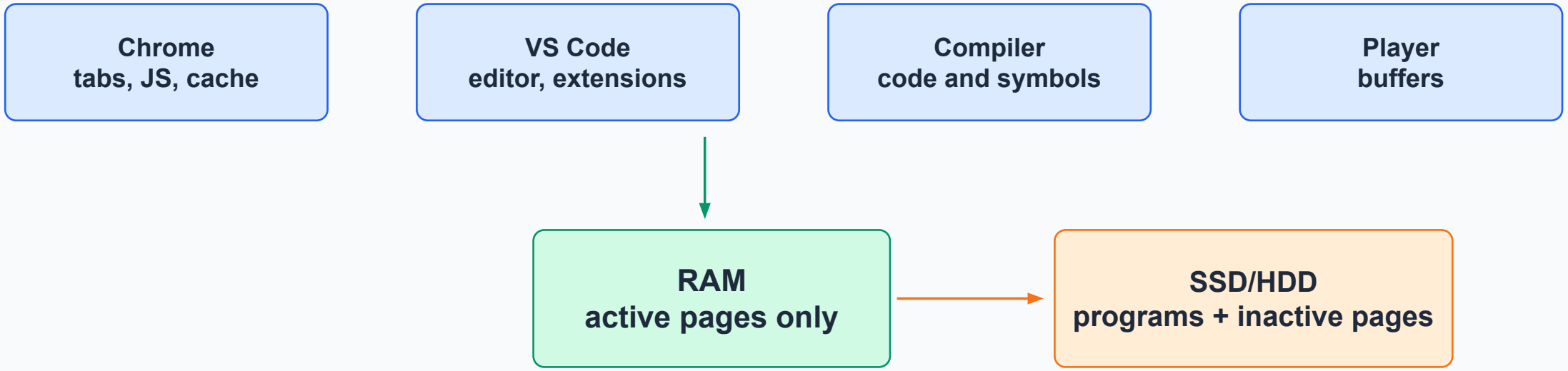
Virtual address space vs physical memory



Some virtual pages are in RAM; others may be on disk or not yet loaded.

Key idea: Virtual memory is a mapping abstraction, not a promise that everything is in RAM.

Scenario: opening multiple applications



Key idea: A normal PC works because the OS keeps the needed pieces close to the CPU.

Small numerical intuition

Physical RAM = 8 GB

Running processes request 20 GB virtual space

Only active working portions need RAM now

Example assumption:

Chrome active pages: 1.2 GB
VS Code active pages: 0.8 GB
Compiler active pages: 1.0 GB
OS + services: 2.0 GB
Total active ≈ 5.0 GB < 8 GB

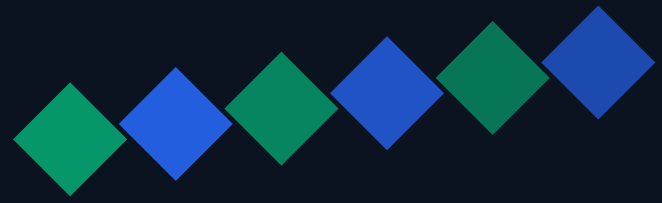
Result
The system can run because virtual size and resident size are different.

Key idea: Virtual memory is practical because most programs show locality.

SECTION B

Paging

Fixed-size mapping from pages to frames



Paging: definition

Process is divided into equal-size pages



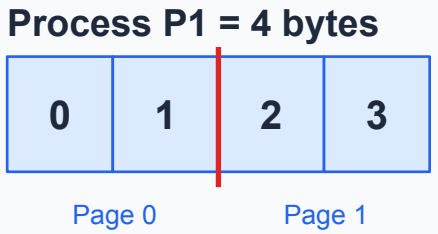
Physical memory is divided into equal-size frames

Page size = Frame size

Key idea: A page can fit exactly into any frame.

Why paging helps

No need to find one large contiguous hole.
Scattered free frames can be used.
Address translation preserves the process view.
External fragmentation is avoided.
The last page may still waste some space internally.



Physical memory = 16 bytes

0		0	1
1		2	3
2	**P1-P0**	4	5
3		6	7
4	**P1-P1**	8	9
5		10	11
6		12	13
7		14	15

Key idea: Paging trades placement difficulty for translation-table management.

Scenario setup

Process P1 size
4 bytes

Page size
2 bytes

Physical memory
16 bytes

Frame size
2 bytes

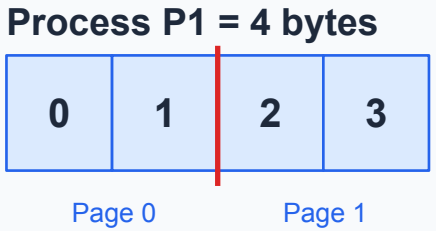
Therefore

Number of pages = 4 / 2 = 2

Number of frames = 16 / 2 = 8

Key idea: Use tiny byte numbers so address translation becomes visible.

Step 1: split P1 into pages



Page 0 holds bytes 0 and 1

Page 1 holds bytes 2 and 3

Key idea: Pages are fixed-size pieces of the process address space.

Step 2: split physical memory into frames

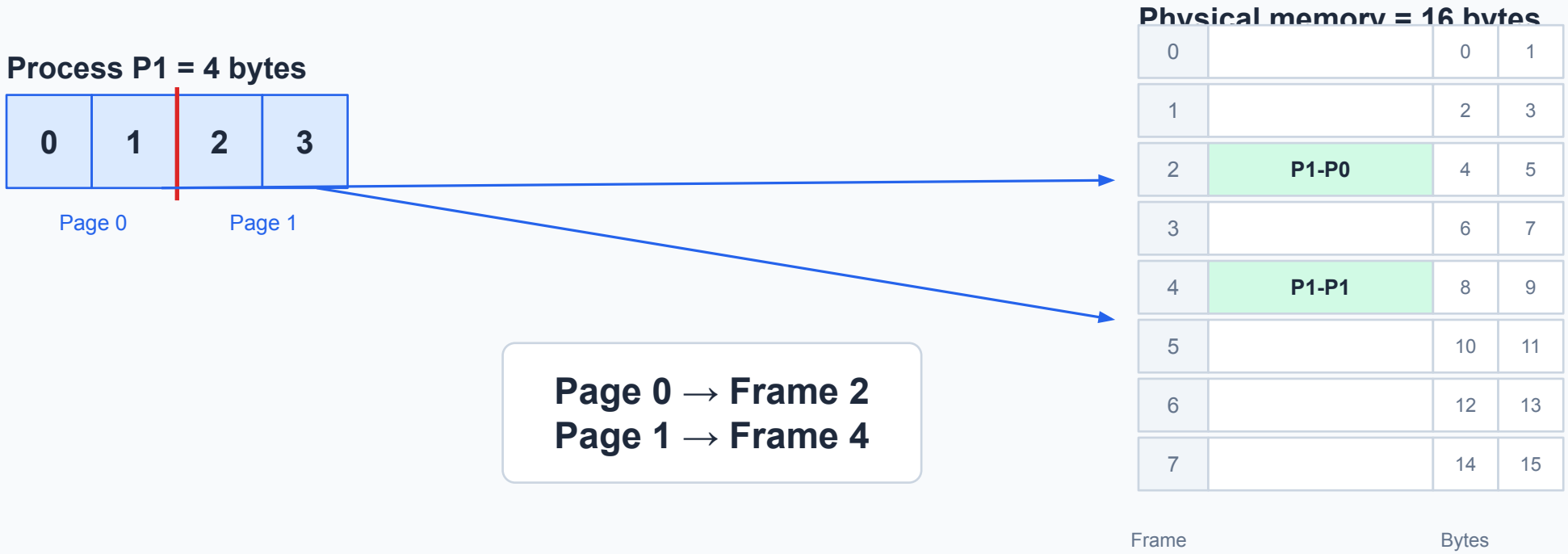
16 bytes / 2 bytes = 8 frames
Frame numbers: 0 to 7

Physical memory = 16 bytes

Frame		Bytes	
0		0	1
1		2	3
2		4	5
3		6	7
4		8	9
5		10	11
6		12	13
7		14	15

Key idea: Frames are fixed-size physical containers in RAM.

Step 3: place pages into non-contiguous frames



Page 0 → Frame 2
Page 1 → Frame 4

Key idea: The physical placement is non-contiguous, but the process still sees bytes 0–3.

Step 4: the CPU asks for logical byte 3

CPU request
logical address = 3

→

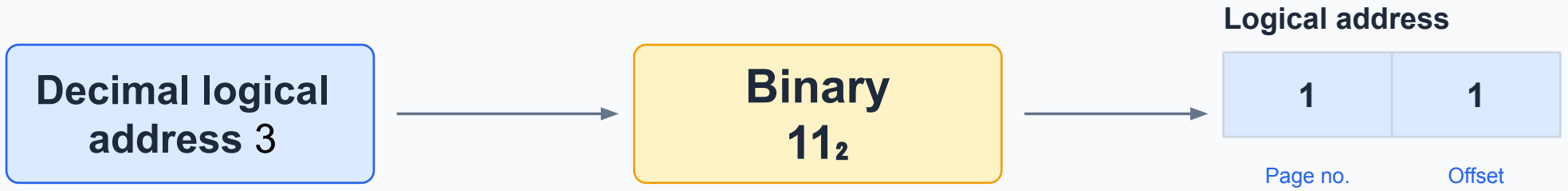
MMU must translate
LA → PA

Physical memory = 16 bytes

Frame		Bytes	
0		0	1
1		2	3
2	P1-P0	4	5
3		6	7
4	P1-P1	8	9
5		10	11
6		12	13
7		14	15

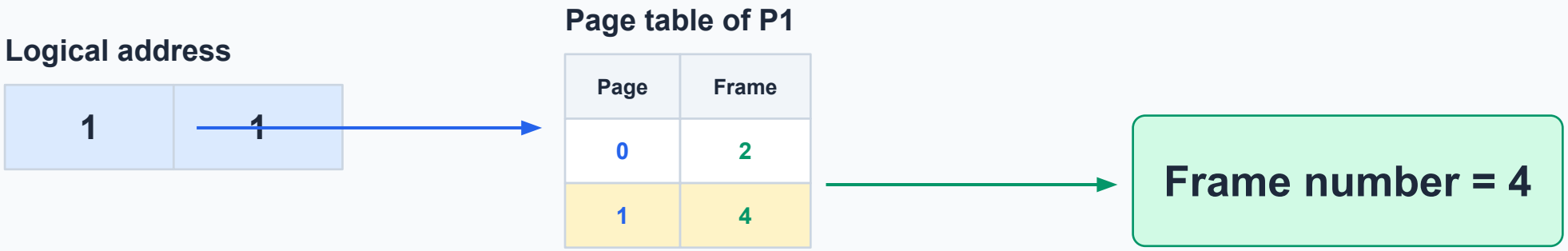
Key idea: The CPU generates a logical address; the MMU maps it to a physical address.

Step 5: convert logical address 3 into fields



Key idea: Logical address = [page number | page offset].

Step 6: page-table lookup



Key idea: The page table stores the frame number for each page.

Step 7: build the physical address



Binary form: 100 | 1 $= 1001_2 = 9_{10}$

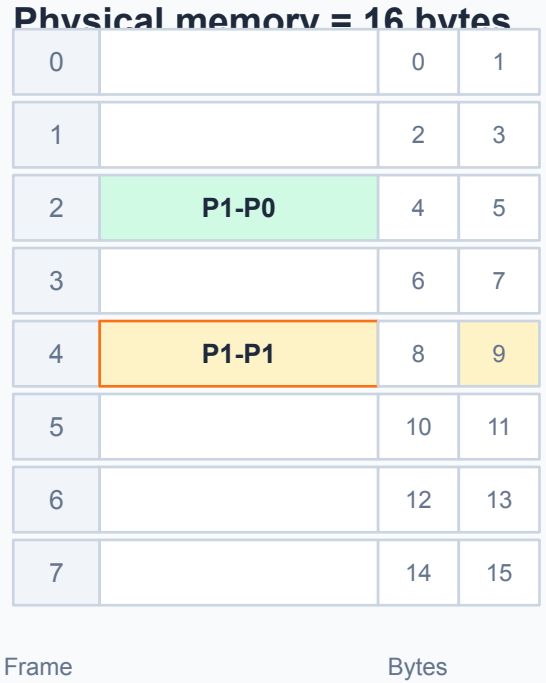
Key idea: Translation changes the page/frame field, not the offset.

Step 8: fetch the byte at physical address 9

CPU asked for LA = 3

→

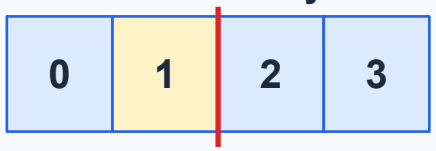
MMU translated to PA = 9



Key idea: The process sees byte 3; RAM serves byte 9.

Quick second example: logical byte 1

Process P1 = 4 bytes



LA = 1 = 01_2

0	1

PA

010	1

$0101_2 = 5_{10}$

Page table of P1

Page	Frame
0	2
1	4

Physical memory = 16 bytes

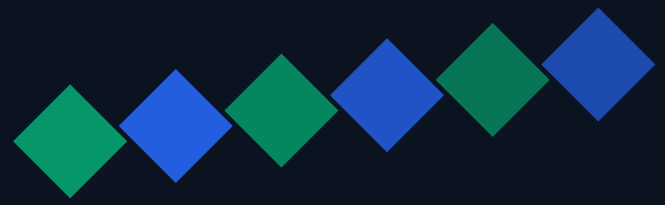
Frame		Bytes	
0		0	1
1		2	3
2	P1-P0	4	5
3		6	7
4	P1-P1	8	9
5		10	11
6		12	13
7		14	15

Key idea: Page 0 maps to Frame 2, so logical byte 1 maps to physical byte 5.

SECTION C

Address Representation

Page number, frame number, and the unchanged offset



Logical and physical address fields

Logical address generated by CPU

Page number	Page offset

Physical address used by RAM

Frame number	Frame offset

Page offset = Frame offset

Only the first field changes:
page number → frame number

Key idea: The offset identifies a position inside the page, and that position remains the same inside the frame.

Address-bit formulas

Number of pages	Process size / Page size
Number of frames	Physical memory size / Frame size
Offset bits	$\log_2$(page size in bytes)
Page-number bits	Logical-address bits − Offset bits
Frame-number bits	Physical-address bits − Offset bits

Key idea: Bit fields are determined by how many alternatives must be represented.

Predict before reveal: paging example 1

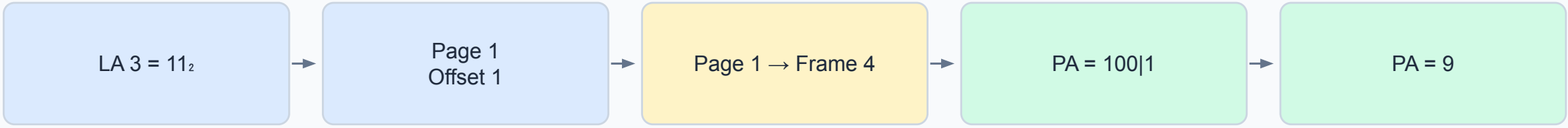
Given

Process P1 = 4 bytes
Page size = 2 bytes
Physical memory = 16 bytes
Frame size = 2 bytes
Page 0 → Frame 2
Page 1 → Frame 4

Question
Where does logical address 3 map?

Key idea: First find page number and offset, then use the page table.

Solution: paging example 1



Frame 4 requires 3 bits because physical memory has 8 frames: 000 to 111.

Key idea: Logical address 3 becomes physical address 9.

Predict before reveal: realistic 4-KB example

Page size = 4 KB = 4096 bytes
Logical address = 13,700
Page table entry: Page 3 → Frame 12

Find:
1) page number
2) offset
3) physical address

Key idea: Divide by page size to get page number; remainder is offset.

Solution: realistic 4-KB example

Page number	⌊13700 / 4096⌋ = 3
Offset	13700 mod 4096 = 1412
Page table	Page 3 → Frame 12
Physical address	12 × 4096 + 1412 = 50,564

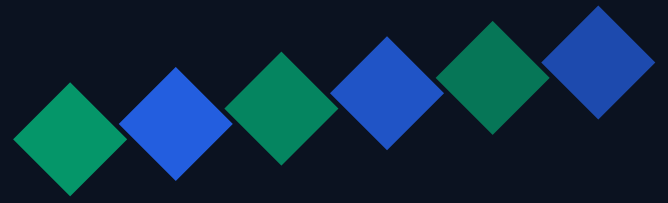
Answer
PA = 50,564

Key idea: Physical address = frame number × frame size + offset.

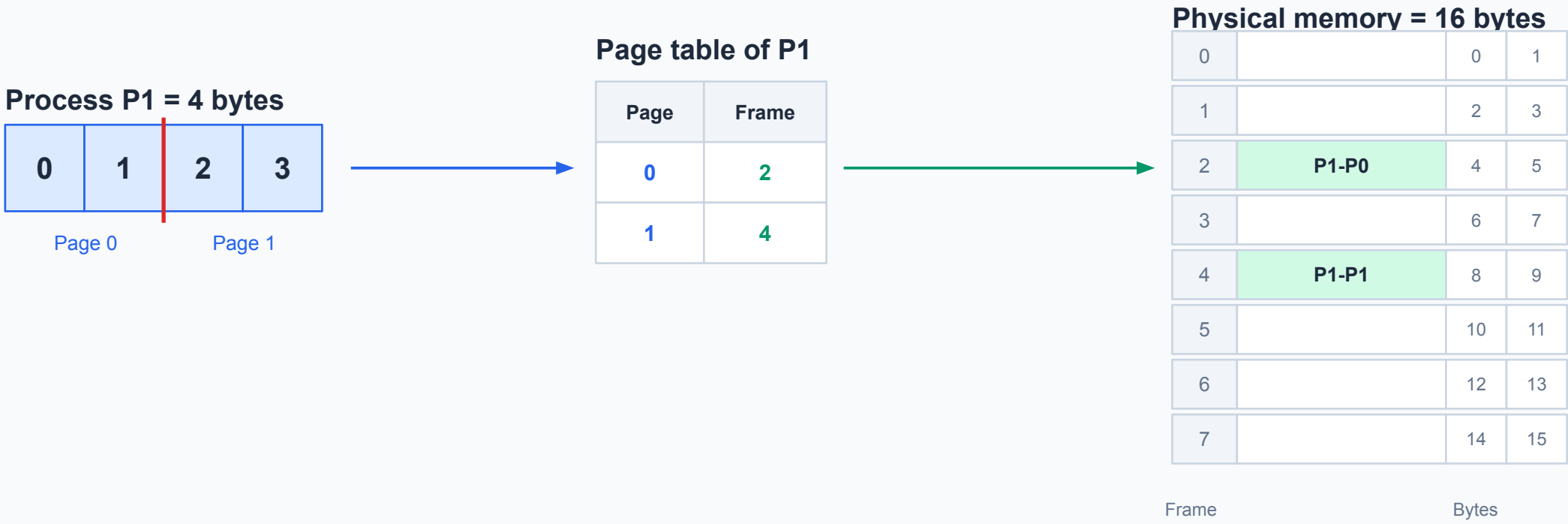
SECTION D

Page Tables

The map that makes paging possible



Why do we need a page table?



Key idea: The page table records where each virtual page currently lives in physical memory.

One page table per process

Page table of P1

Page	Frame
0	2
1	4

P1 page table

Page	Frame
0	6
1	1

P2 page table

Same virtual page number may map differently in different processes.

Key idea: Virtual address spaces are private; page tables define each process’s private mapping.

Page-table entry fields

Frame number	Where the page is in RAM
Present / Valid	Is the page currently resident?
Protection	Read / write / execute permission
Reference bit	Was it recently accessed?
Modified / Dirty	Was it changed?

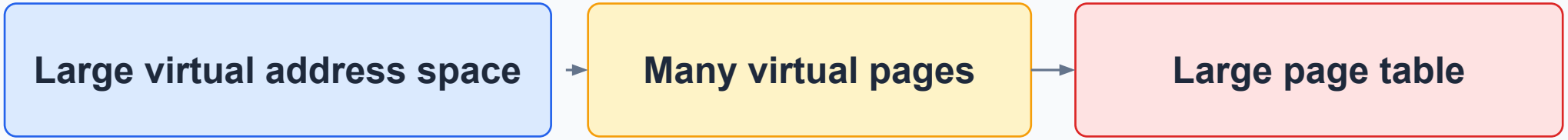
Key idea: A PTE stores more than a frame number; it also controls validity, safety, and replacement decisions.

MMU in translation



Key idea: The MMU performs address translation using the page table selected by the OS.

The hidden cost of page tables



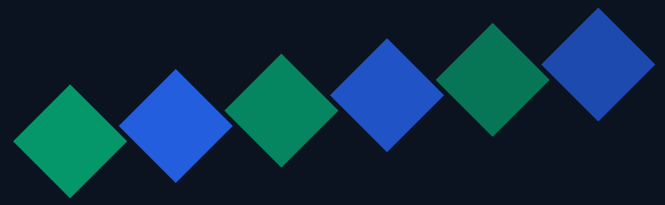
Question: Do we really need to keep one huge flat table?

Key idea: Multi-level paging is introduced to manage large and sparse page tables.

SECTION E

Multi-Level Paging

Paging the page table itself



The problem: a huge single-level page table

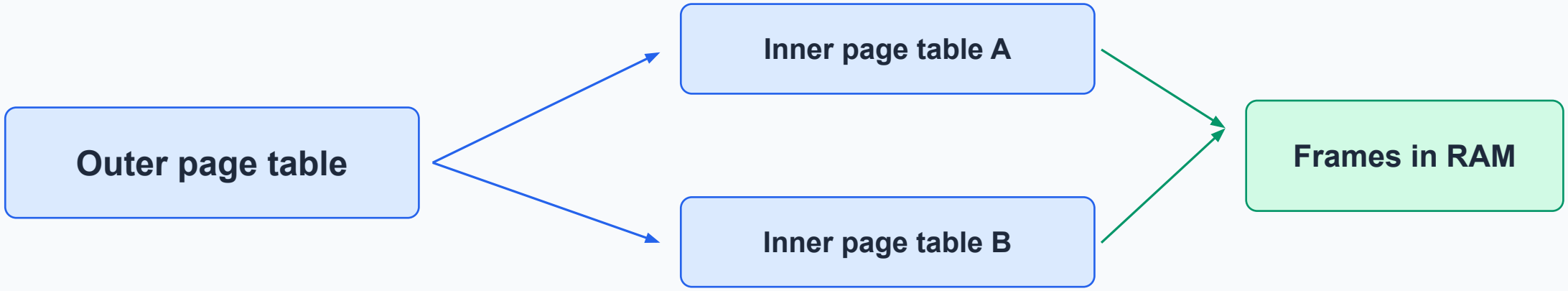
Single flat page table



Many entries may exist only because the address space is large.

Key idea: A large address space can make a flat page table too large to manage efficiently.

Multi-level paging intuition



Key idea: Multi-level paging stores page-table pieces only where they are needed.

Numerical setup for multi-level paging

Logical address space = 4 GB = 2^{32} bytes
Physical memory = 256 MB = 2^{28} bytes
Page/frame size = 4 KB = 2^{12} bytes
Frame number bits = 28 − 12 = 16 bits
Assume PTE stores only frame number = 16 bits = 2 bytes

Goal
Show why a flat page table is too large, then split it.

Key idea: The numerical example uses simplified PTE size so the sizing logic is easy to see.

Single-level page table size

Offset bits	$\log_2(4\ \text{KB}) = 12$ bits
Virtual pages	$2^{32} / 2^{12} = 2^{20}$ pages
PTE size	16 bits = 2 bytes
Flat table size	$2^{20} \times 2$ bytes $= 2^{21}$ bytes = 2 MB

2 MB page table
> one 4-KB frame

Key idea: The page table itself becomes too large to handle as one small unit.

Two-level address split

32-bit logical address

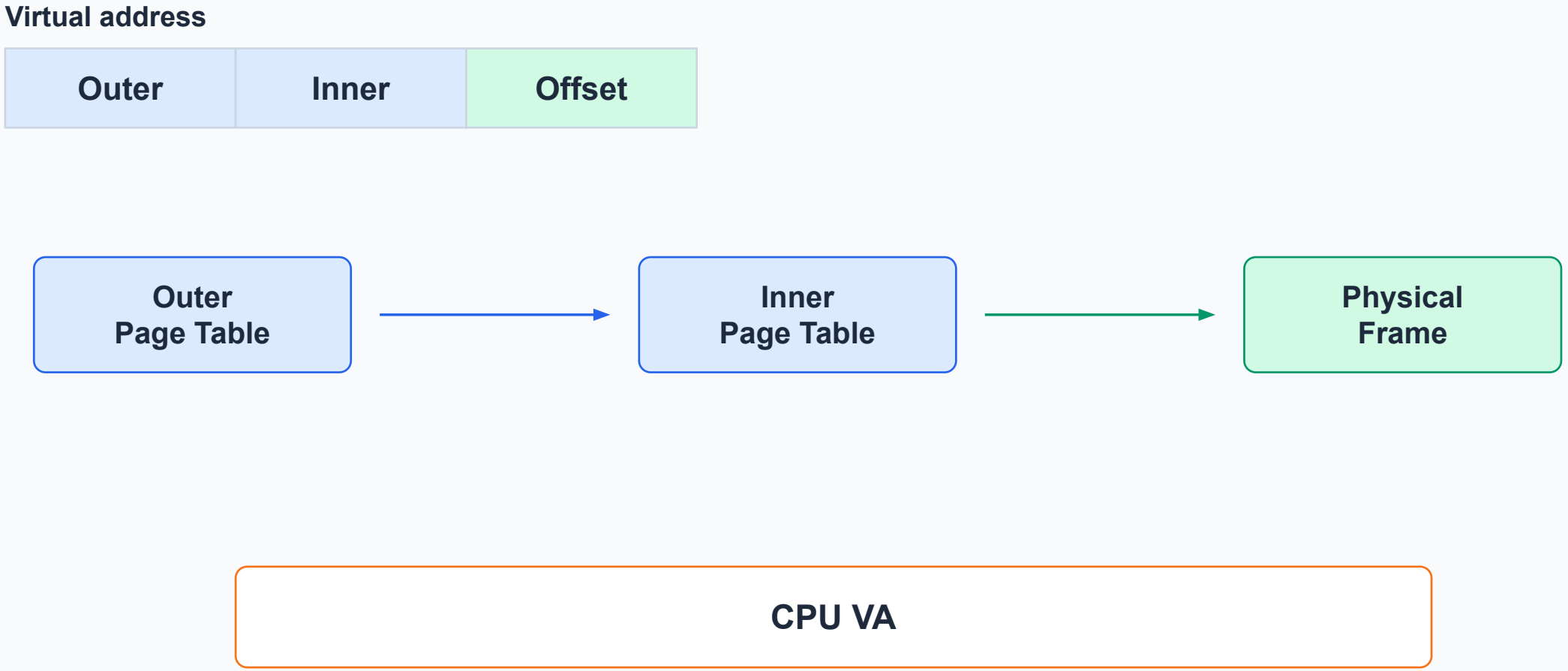
Outer index 9 bits	Inner index 11 bits	Offset 12 bits

9 + 11 + 12 = 32 bits

Inner table size
2^{11} entries × 2 bytes
= 4096 bytes = 4 KB

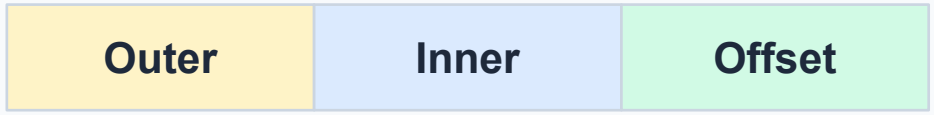
Key idea: Each inner page table now fits exactly inside one 4-KB frame.

Step 1: CPU generates a virtual address



Step 2: use the outer index

Virtual address

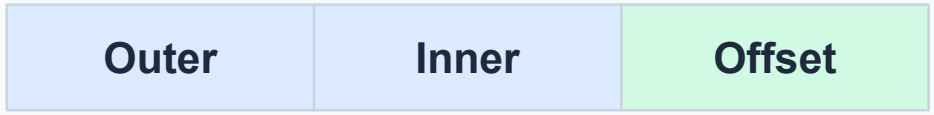




Outer index selects an outer-table entry

Step 3: locate the inner page table

Virtual address

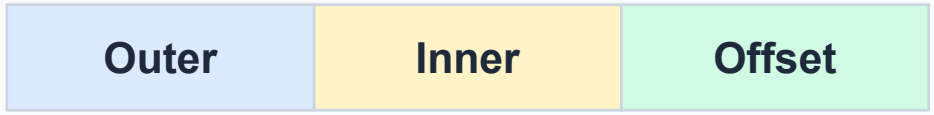




Outer entry points to one inner page table

Step 4: use the inner index

Virtual address



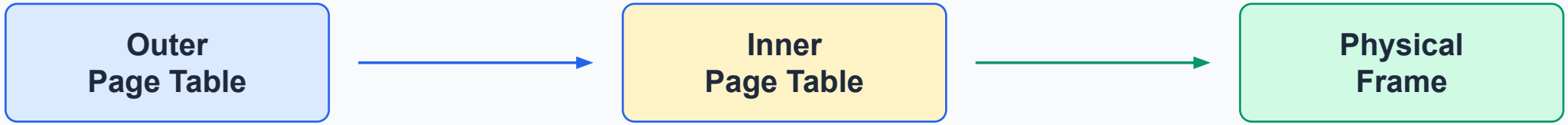


Inner index selects the final PTE

Step 5: obtain frame number

Virtual address

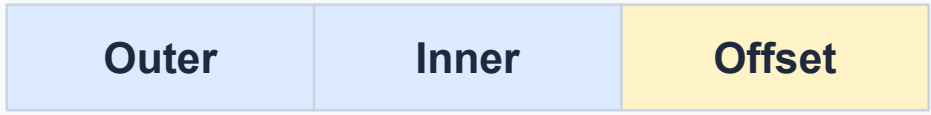
Outer	Inner	Offset



PTE provides the physical frame number

Step 6: append unchanged offset

Virtual address





Physical address = [frame | same offset]

What multi-level paging saves

Flat table

Needs entries for all virtual pages

Multi-level table

Allocates lower-level tables only when a region is used

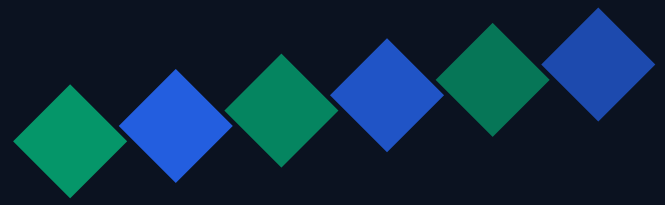
Best for sparse large address spaces

Key idea: Multi-level paging reduces memory overhead by not materializing unused parts of the page table.

SECTION F

Inverted Page Table

One table entry per physical frame



Conventional vs inverted page table

Conventional page table
One entry per virtual page
usually per process

Inverted page table
One entry per physical frame
system-wide idea

Different question:

Normal: where is this virtual page?

Inverted: which virtual page is in this frame?

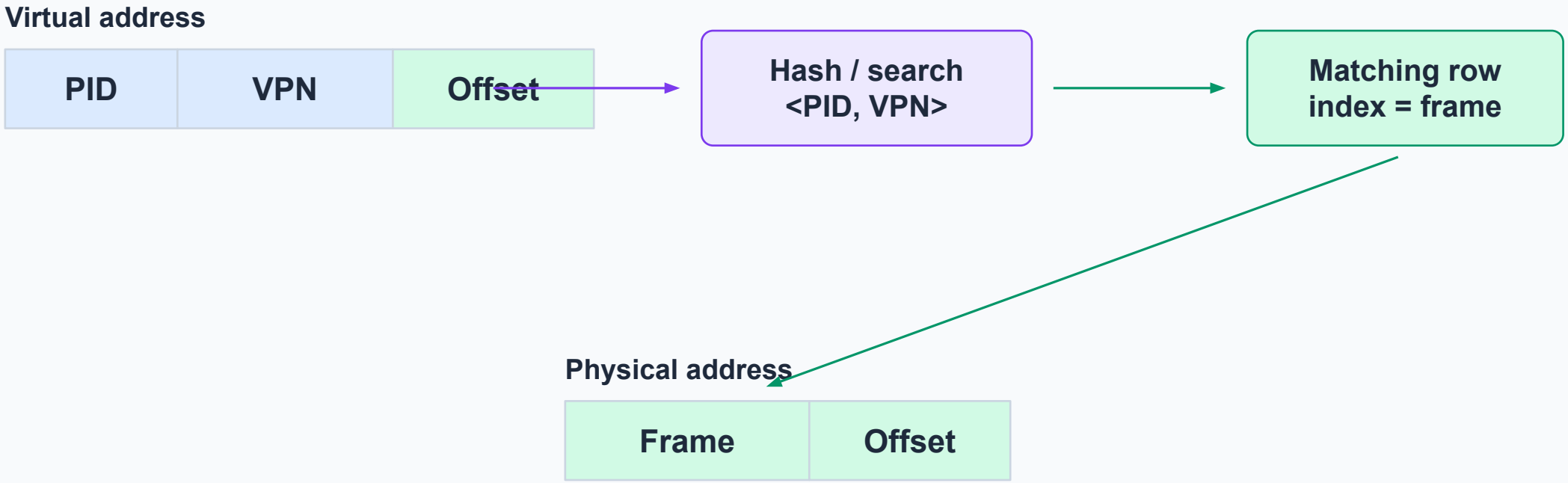
Key idea: An inverted page table scales with physical memory, not directly with total virtual address space.

Inverted table entry

Frame index	Page no	Process No/ID
0	Pg1	P2
1	Pg2	P3

The table index itself gives the physical frame number.

Inverted page-table lookup



Key idea: The offset remains unchanged after the frame is found.

Numerical intuition: inverted table size

Physical RAM = 4 GB = 2^{32} bytes
Page size = 4 KB = 2^{12} bytes
Number of physical frames = $2^{32} / 2^{12} = 2^{20}$
So an inverted table has about 1,048,576 frame entries.

It does not need one entry for every possible virtual page of every process.

Key idea: The table size follows physical memory size.

Inverted page table: trade-offs

Strengths
- smaller for huge virtual spaces
- system-wide physical-frame view
- works well with TLB

Limitations
- lookup is harder
- often needs hashing
- shared pages need careful handling

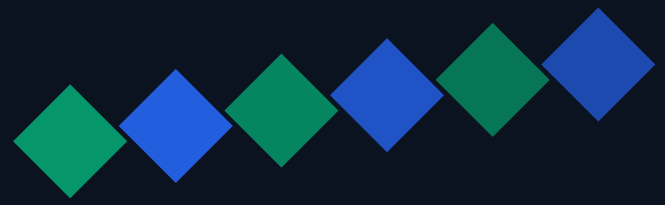
TIME > MEMORY as time is more valuable to us.

Key idea: Inverted tables save space, but make translation logic more complex.

SECTION G

Page Faults & Page Faults Service

Loading pages only when needed



Page fault is not always an error

Programming error?

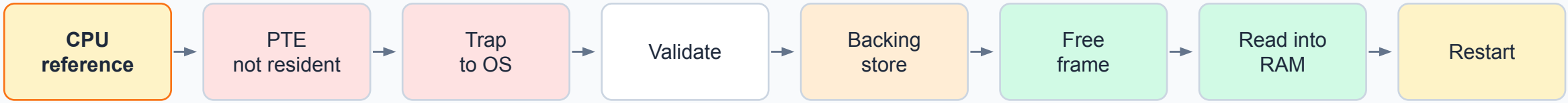
Sometimes yes
invalid access

Normal OS event?

Often yes
valid page not in RAM

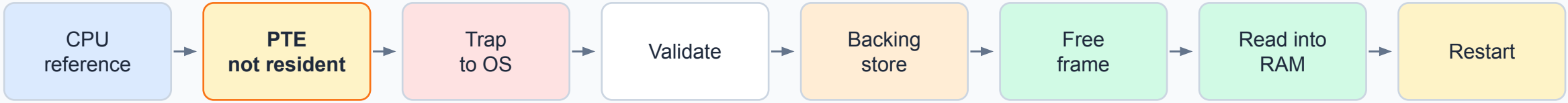
Key idea: Under demand paging, a page fault can simply mean “bring this valid page from disk.”

Step 1: CPU references a page



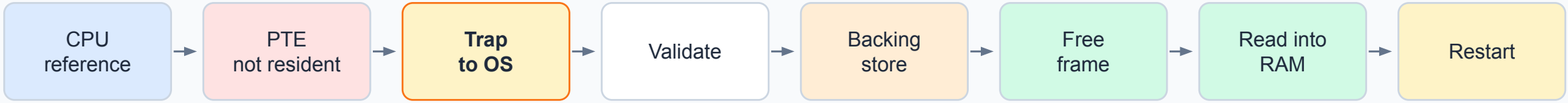
Key idea: Page-fault service is OS + hardware cooperation.

Step 2: page table says not resident



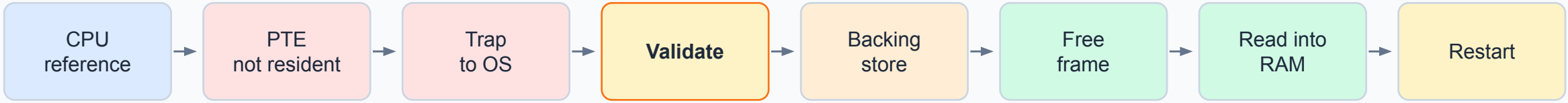
Present/valid bit says: not in RAM

Step 3: hardware traps to OS



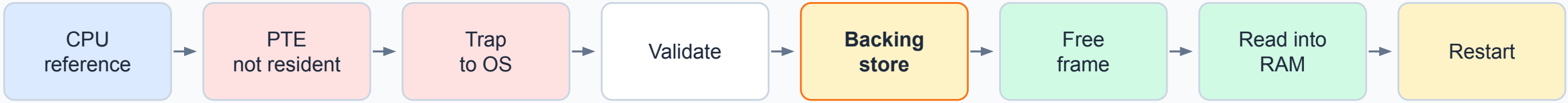
Key idea: Page-fault service is OS + hardware cooperation.

Step 4: OS validates the reference



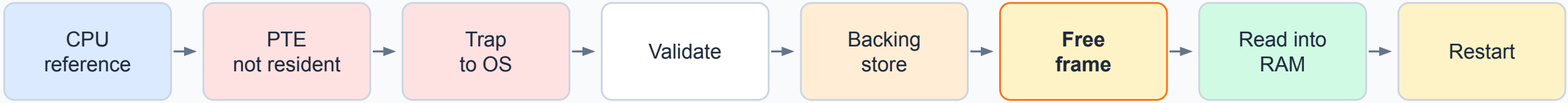
Key idea: Page-fault service is OS + hardware cooperation.

Step 5: locate page on backing store



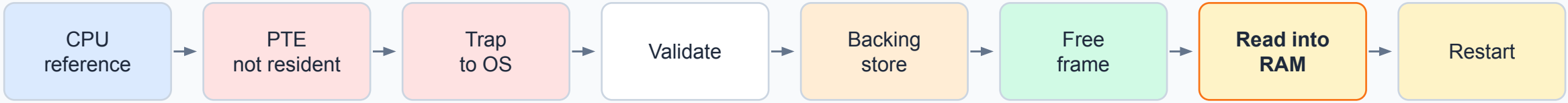
Disk access is slow, so page faults are expensive.

Step 6: find a free frame



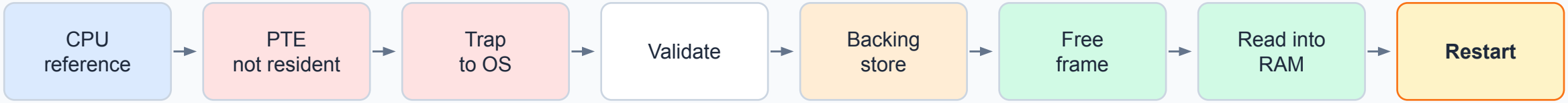
Key idea: Page-fault service is OS + hardware cooperation.

Step 7: read page into RAM



Key idea: Page-fault service is OS + hardware cooperation.

Step 8: update table and restart instruction

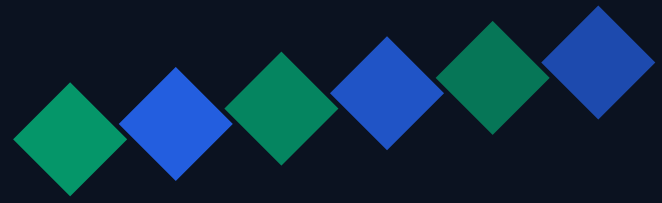


The interrupted instruction is executed again after the mapping is fixed.

SECTION H

Thrashing and Working Set

When paging overwhelms execution



Working set intuition

Process has pages:



Current loop repeatedly uses P2, P3 and P4

Working set ≈ {P2, P3, P4}

Key idea: A process runs smoothly when its active working set has enough frames.

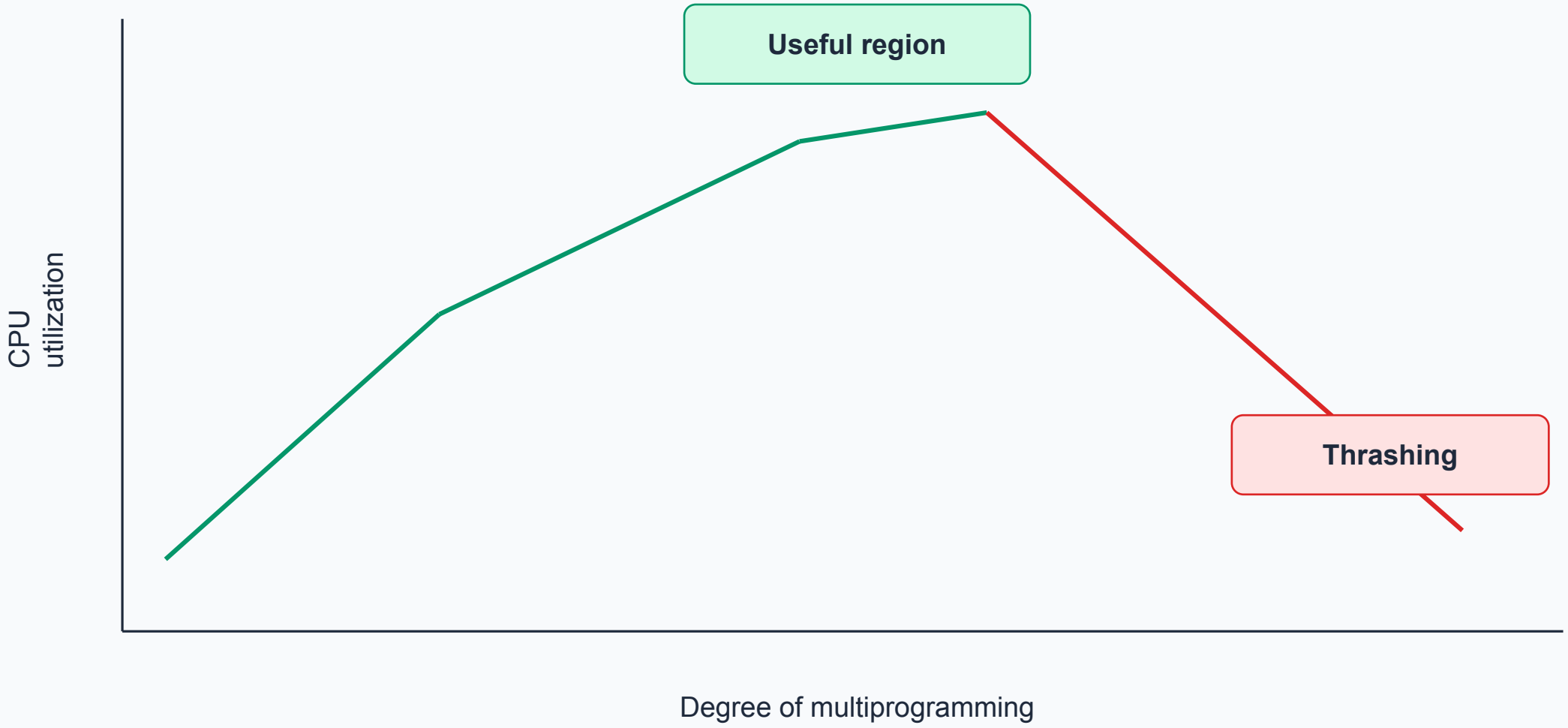
Thrashing: definition



System spends more time moving pages than executing instructions.

Key idea: Thrashing is not just “low RAM”; it is excessive paging activity.

Classic thrashing graph



Key idea: More multiprogramming helps only until memory pressure becomes too high.

Causes of thrashing

Too many active processes in memory
Insufficient frames for each process
Working sets do not fit in RAM
Poor locality or sudden memory pressure
Global replacement may spread page stealing across processes

Symptom
Page faults rise sharply, CPU utilization falls.

Key idea: Thrashing is a system-level imbalance between active memory demand and available frames.

Solutions and LTS role

Give enough frames for active working sets

Reduce degree of multiprogramming

Use working-set or PFF reasoning

Local replacement can contain damage

Add RAM when workload genuinely exceeds capacity

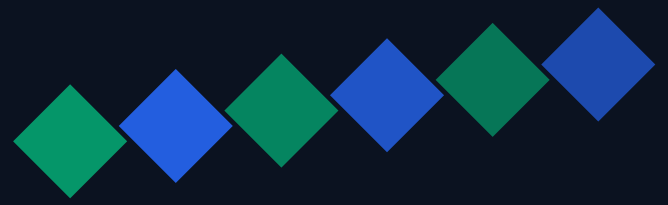
LTS / admission control can stop admitting too many new processes.

Key idea: The long-term scheduler helps by controlling how many processes compete for memory.

SECTION I

Segmentation

Logical variable-size division of a program



Segmentation: the basic idea

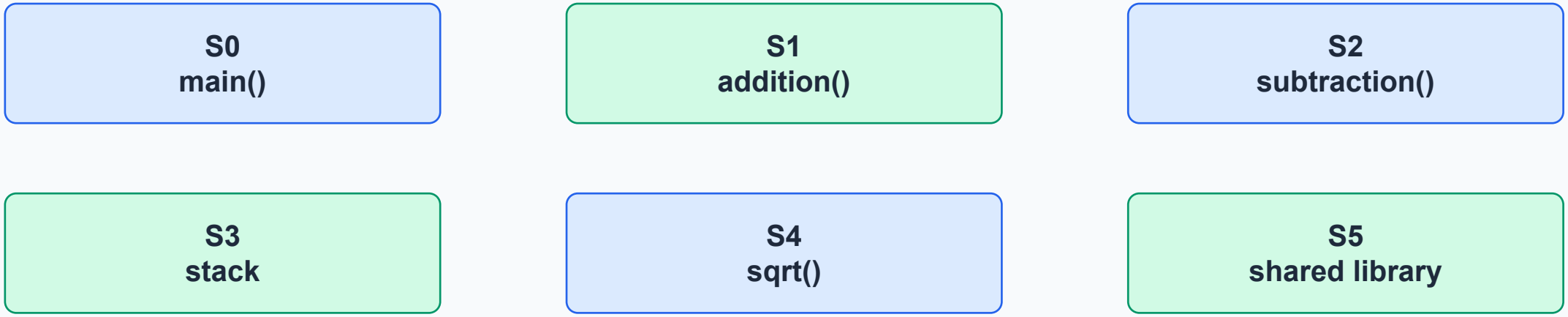
Paging
fixed-size pages
may split logical modules

Segmentation
logical units
variable sizes

Examples of segments: main(), add(), stack, heap, library module

Key idea: Segmentation divides a process according to meaningful program units.

Program modules as segments



Key idea: Segments may have different sizes because program modules naturally differ in length.

Segmentation address format

Logical address

Segment number	Offset

Segment number selects a row in the segment table.

Offset tells how far inside that segment to access.

Key idea: Segmentation uses <segment number, offset>.

Segment table entries

Segment table

Seg	Base	Size
S0	3300	200
S1	1800	400
S2	2700	600
S3	2300	400
S4	2200	100
S5	1500	300

Base
starting physical address

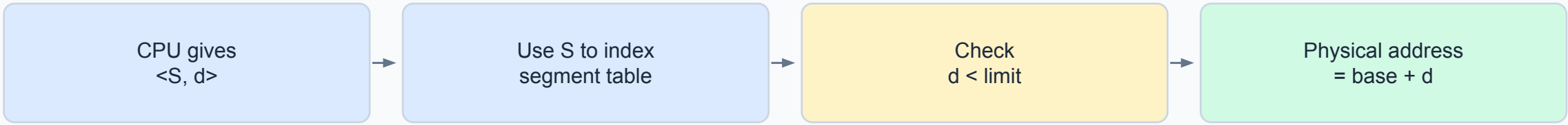
Size / limit
segment length

Rule for zero-based offset:

0 ≤ offset < limit

Key idea: The limit check prevents a process from reading beyond the selected segment.

Segmentation translation mechanism



If offset is invalid → trap

Key idea: Segmentation translates by adding the offset to the segment base, after a bounds check.

Valid example: <S3, 200>

Segment table

Seg	Base	Size
S0	3300	200
S1	1800	400
S2	2700	600
S3	2300	400
S4	2200	100
S5	1500	300

Logical address

S3	200

S3 base = 2300
S3 size = 400

Check: 200 < 400
Valid

PA = 2300 + 200 = 2500

Key idea: The selected segment is S3; the final physical address is 2500.

Invalid example: <S3, 450>

Segment table

Seg	Base	Size
S0	3300	200
S1	1800	400
S2	2700	600
S3	2300	400
S4	2200	100
S5	1500	300

Logical address

S3	450

S3 size = 400

Check: 450 < 400 ?
False

Trap / protection fault

Key idea: Invalid offsets are blocked before physical memory is accessed.

Segmentation memory layout

Segment table

Seg	Base	Size
S0	3300	200
S1	1800	400
S2	2700	600
S3	2300	400
S4	2200	100
S5	1500	300

Physical memory



Segments can be placed at different physical locations while keeping their logical identity.

Key idea: The segment table connects logical segments to their physical base locations.

Protection and sharing intuition

Code segment
read + execute

Data segment
read + write

Shared library
shared mapping

Because segments correspond to logical units, protection can also be meaningful at that level.

Key idea: Segmentation can express programmer-visible structure more naturally than pure paging.

Paging vs segmentation

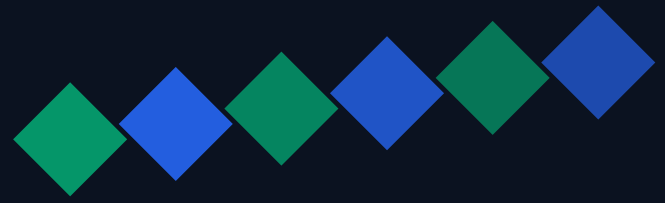
Aspect	Paging	Segmentation
Basis	Fixed-size blocks	Logical variable-size units
Address	<page, offset>	<segment, offset>
Table	Page table	Segment table
Fragmentation	Possible internal	Possible external
Visibility	Mostly invisible to programmer	Closer to programmer view
Strength	Simple allocation	Logical protection/sharing

Key idea: Paging is physical-management friendly; segmentation is logical-structure friendly.

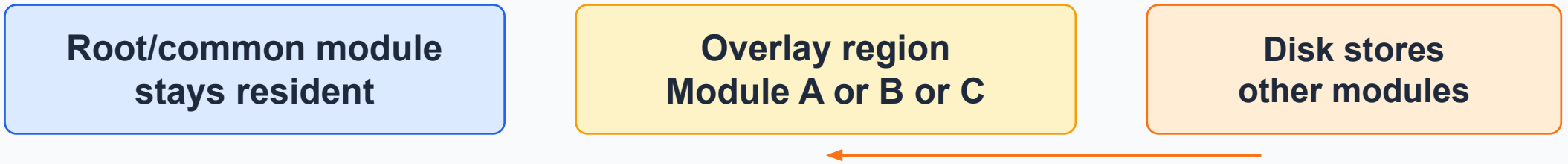
SECTION J

Overlay

A manual predecessor to virtual memory ideas



Overlay: basic idea



Only one mutually exclusive module occupies the overlay area at a time.

Key idea: Overlays let large programs run in small memory, but the programmer manages the swapping plan.

Overlay vs virtual memory

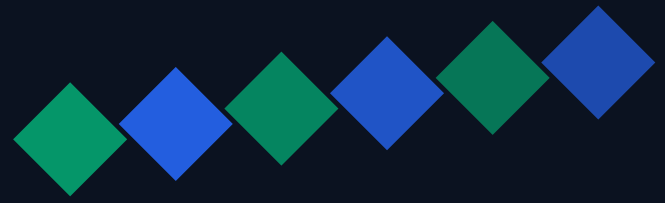
Aspect	Overlay	Virtual Memory
Managed by	Programmer / compiler	OS + MMU
Unit	Modules / routines	Pages or segments
Transparency	Manual	Mostly transparent
Hardware support	Not necessarily	Usually required
Modern use	Limited/special cases	Core OS mechanism

Key idea: Virtual memory automates what overlays forced programmers to plan manually.

SECTION K

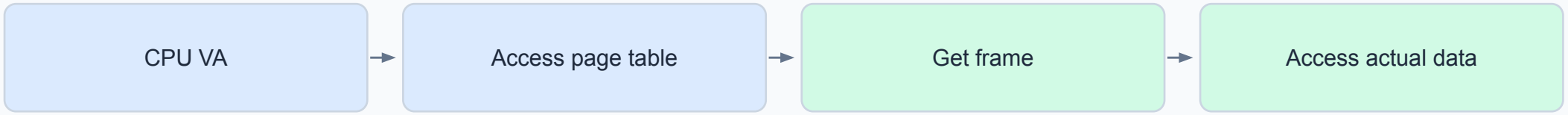
TLB

Making address translation fast



The motivation for TLB

Without TLB



A simple data access may require two RAM accesses.

Key idea: Paging needs translation; TLB reduces the translation overhead.

What is a TLB?

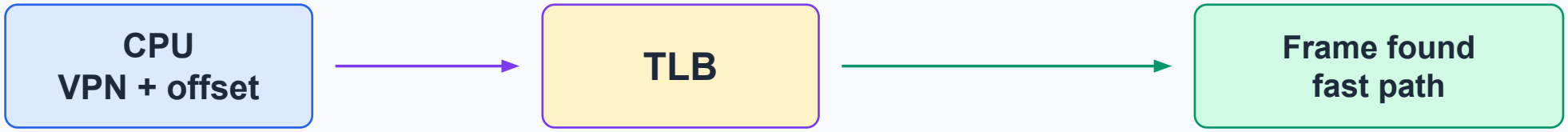
TLB
Translation Lookaside Buffer

Small, fast associative cache of recent page-table translations

Stores mappings like: VPN → Frame number

Key idea: The TLB caches translations, not the actual program data.

TLB hit



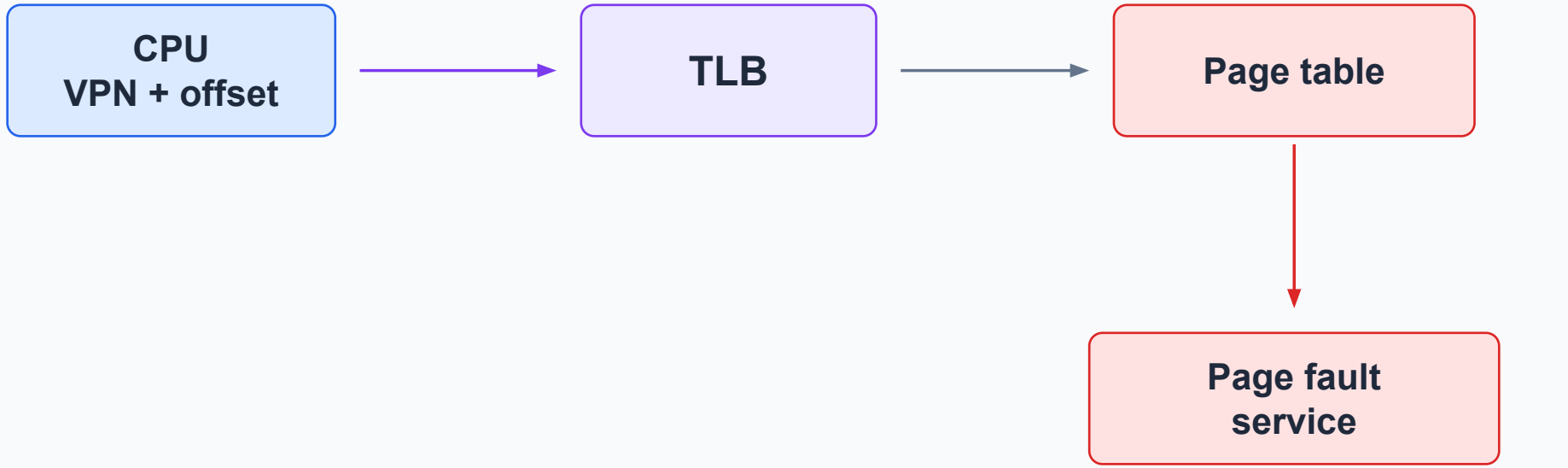
VPN is found directly in the TLB

TLB miss, page is resident



Translation not in TLB, but page table has it

TLB miss + page fault



Translation absent because page is not in RAM

Important distinction

TLB MISS ≠ PAGE FAULT

TLB miss
translation cache does not contain the mapping

Page fault
needed page is not currently in RAM

Key idea: A TLB miss may be solved by consulting the page table; a page fault needs OS service and usually disk access.

Predict before reveal: TLB EAT

Memory access time = 100 ns
TLB lookup treated as negligible/same pipeline
TLB hit ratio = 80%
On TLB hit: one memory access
On TLB miss with resident page: page table access + data access

Find the effective access time.

Effective access time , EAT = Hit * t + (1-Hit) * t (generalized formula)

You can build your own formula and do maths for TLB

Key idea: Separate hit cost and miss cost first.

Solution: TLB hit ratio = 80%

Hit cost	100 ns
Miss cost	100 ns + 100 ns = 200 ns
EAT	0.80 × 100 + 0.20 × 200
Answer	120 ns

Key idea: Even a good hit ratio still matters because misses require an extra memory access.

Why high TLB hit ratio matters

80% hit ratio
T = 120 ns

99% hit ratio
T = 101 ns

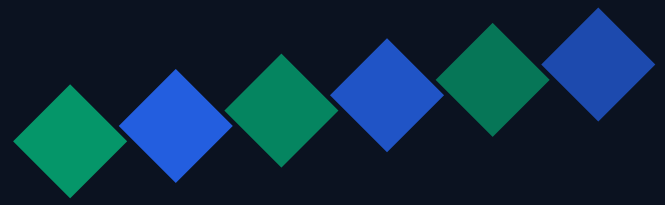
$$EAT = 0.99 \times 100 + 0.01 \times 200 = 101 \text{ ns}$$

Key idea: TLB performance depends heavily on locality and hit ratio.

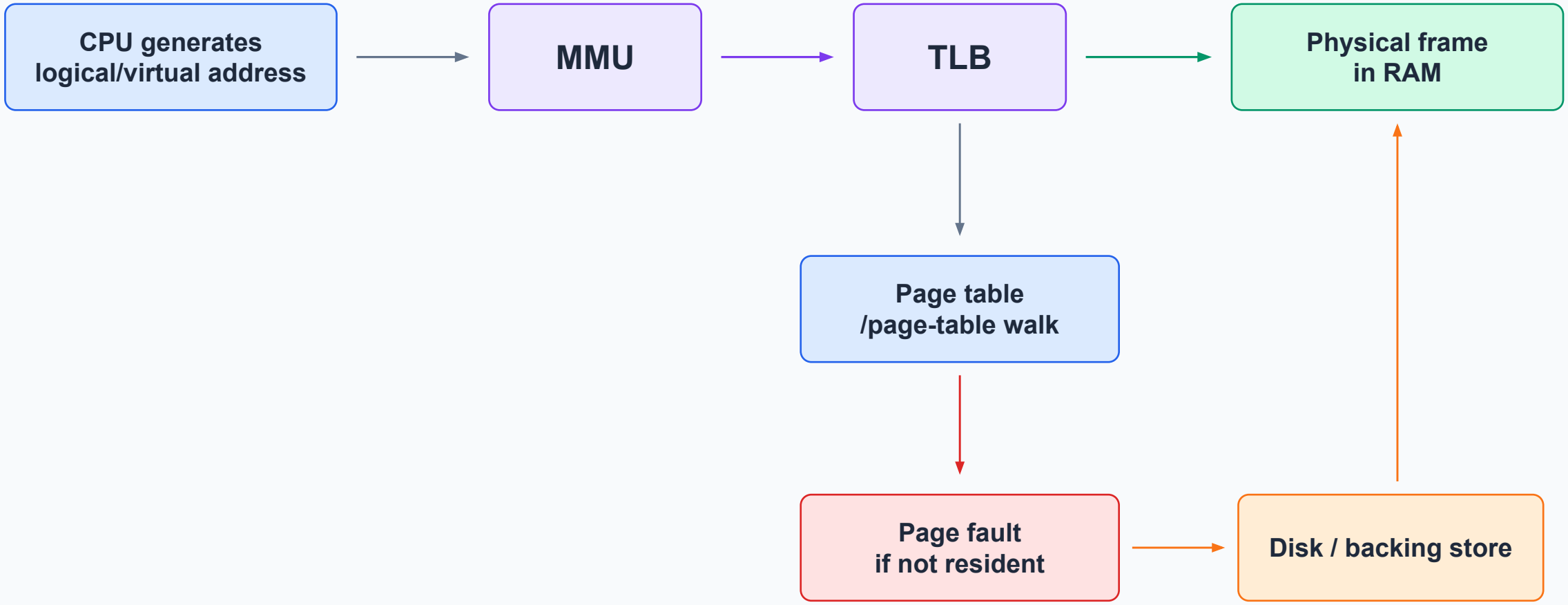
FINAL

Putting Everything Together

One map for translation, paging, faults, and speed



Unified memory-translation flow



Key idea: TLB speeds translation; page tables define mapping; page faults bring missing pages into RAM.

In-class activity: predict the mapping

Page size = 4 bytes
Logical address = 23
Page table: P5 → F9
Question 1: page number?
Question 2: offset?
Question 3: physical address?

Some sample theory questions

1. Why must page size and frame size be equal?
2. Why does the offset remain unchanged?
3. Why is TLB miss not the same as page fault?
4. How does segmentation differ from paging?

Key idea: If students can answer these, the lecture objective is met.

Disk Scheduling: Architecture & Algorithms

Goal: reduce unnecessary disk-head movement

Disk architecture: the storage hierarchy

Sequence matters for both capacity calculation and access-time analysis.

Platter

Circular magnetic disk. A hard disk may contain multiple platters.

Surface / Head

Each platter can have upper and lower usable surfaces. Usually one head per usable surface.

Track → Sector → Data

Each surface contains tracks. Each track is divided into sectors. Data is stored inside sectors.

sequence

Platters × usable surfaces × tracks × sectors × bytes/sector.

Physical structure: platter, spindle and R/W head

The disk rotates; the read/write head mainly moves inward or outward across tracks.

Spindle

Rotates all platters together in one direction.

Actuator arm

Moves the read/write heads radially across tracks.

R/W head

Reads or writes data when the required sector rotates under the head.

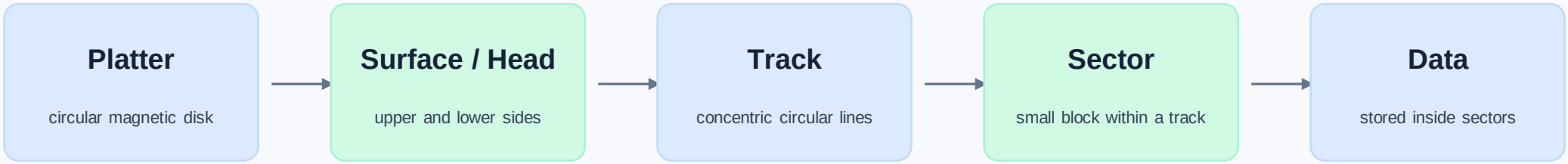
Key point

Disk scheduling mainly decides how the head moves from one cylinder request to another.

CONCEPT 01

Disk architecture: the storage hierarchy

A magnetic disk is organized in layers. The sequence matters for both capacity calculation and access-time analysis.



- Each platter has two usable surfaces unless the question states otherwise.
- Each surface contains many tracks; each track contains sectors.
- The operating system stores and fetches data from sectors.

sequence

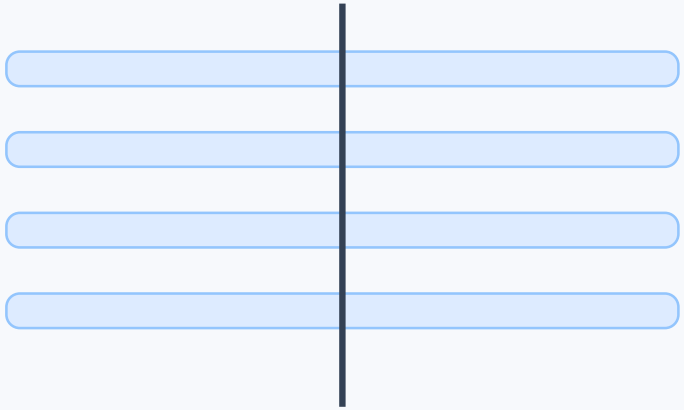
Platters × surfaces × tracks × sectors × bytes/sector

This multiplication gives total disk capacity.

CONCEPT 02

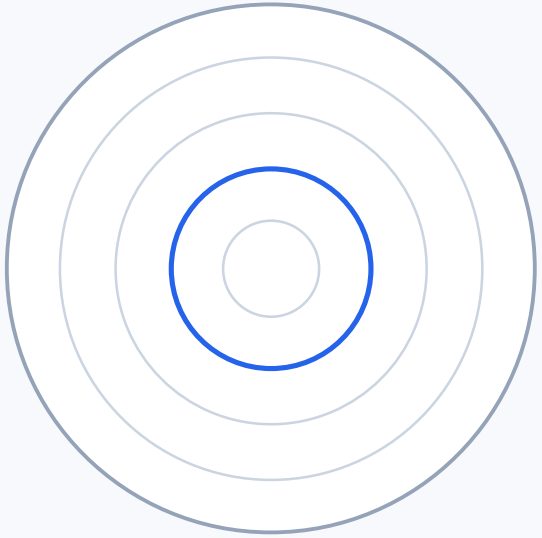
Physical structure: platter, spindle and head

The disk spins, but the read/write head mainly moves inward or outward across tracks.



Platters connected by spindle

Spindle rotates all platters in one direction



tracks

read/write head + actuator arm

Head movement

Only inward / outward track movement
The platter rotation brings sectors under the head

EXAMPLE 01

Disk capacity: one numerical pattern

Most basic disk-size questions are solved by multiplying the physical organization levels.

Given

Platters	$8 = 2^3$
Surfaces per platter	$2 = 2^1$
Tracks per surface	$256 = 2^8$
Sectors per track	$512 = 2^9$
Data per sector	$512\ B = 2^9\ B$

Capacity

$$8 \times 2 \times 256 \times 512 \times 512\ B$$

$$2^3 \times 2^1 \times 2^8 \times 2^9 \times 2^9 \text{ bytes}$$

$$= 2^{30} \text{ bytes} = 1 \text{ GiB}$$

Bits needed to represent 1 GiB byte addresses = 30 bits

CONCEPT 04

Disk access time: the main components

Total access time combines physical movement, waiting for rotation, data transfer, and optional waiting/control overhead.

Queue time
request waits in disk queue

+

Controller time
driver/controller overhead

+

Seek time
head reaches desired track

+

Rotational latency
sector comes under head

+

Transfer time
data is read/written

Access Time = Queue + Controller + Seek + Rotational Latency + Transfer

For basic disk scheduling, we usually focus on reducing seek time.

Disk access time: the main components

A disk request is not served instantly; several delays can be involved.

Queue time

Request waits in the disk queue.

Controller time

Driver/controller overhead.

Seek time

Head moves to the target track.

Rotational latency + transfer

Sector rotates under the head, then bytes are read/written.

CONCEPT 05

Why disk scheduling is needed

Many disk requests may wait together. The OS chooses a service order to reduce unnecessary head movement.

Request queue

98	183	37	122
14	124	65	67

Current head position = 53

Scheduling decision

- If we simply follow arrival order, the head may jump repeatedly.
- If we choose smarter order, average service time may improve.
- The main goal is to minimize seek time or head movement.

Seek distance ≈ |next track − current track|

Why disk scheduling is needed

Many disk requests may wait together. The OS chooses a service order.

Request queue

98, 183, 37, 122, 14, 124, 65, 67

Current head

Initial position = 53

Bad order

Head may jump repeatedly across the disk.

Goal

Reduce unnecessary head movement while maintaining reasonable fairness.

CONCEPT 06

Basic disk scheduling algorithms

Each algorithm answers the same question: which waiting request should the disk head serve next?

FCFS

Serve requests in arrival order

simple, fair, but may cause long movement

SSTF

Serve nearest track next

low average movement, possible starvation

SCAN

Move like an elevator

services requests while sweeping one direction

C-SCAN / LOOK / C-LOOK

One-direction sweep variants

more uniform waiting time in many cases

No single algorithm is always best; workload and fairness matter.

Basic disk scheduling algorithms

Each algorithm answers: which waiting request should be served next?

FCFS

Serve in arrival order. Simple and fair, but often inefficient.

SSTF

Serve nearest request next. Good movement, but possible starvation.

SCAN / LOOK

Sweep in a direction; LOOK avoids unnecessary end travel.

C-SCAN / C-LOOK

Circular one-direction service for more uniform waiting behavior.

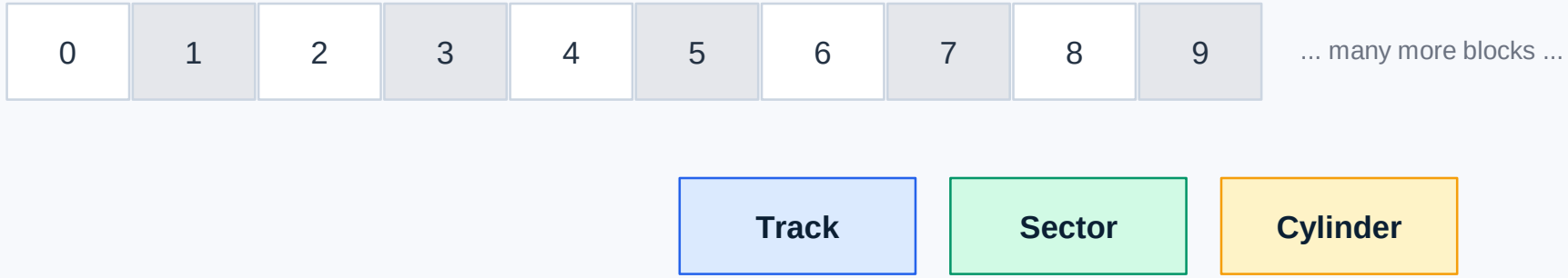
Disk structure in one picture

The OS views the disk as a long logical-block array, although hardware is circular.



Physical disk idea

Logical blocks seen by OS



For scheduling, we normally simplify a request as a cylinder number.

Example request: read block at cylinder 98

Access time: what are we minimizing?

A disk request is not served instantly. Several delays are involved.

Seek time
head moves to cylinder

Rotational latency
sector rotates under head

Transfer time
bytes are actually read/written

Simplified class model

Seek time ≈ seek distance

So we compare algorithms by total head movement.

If head moves from 53 to 98, movement = |98 − 53| = 45 cylinders

Running example setup

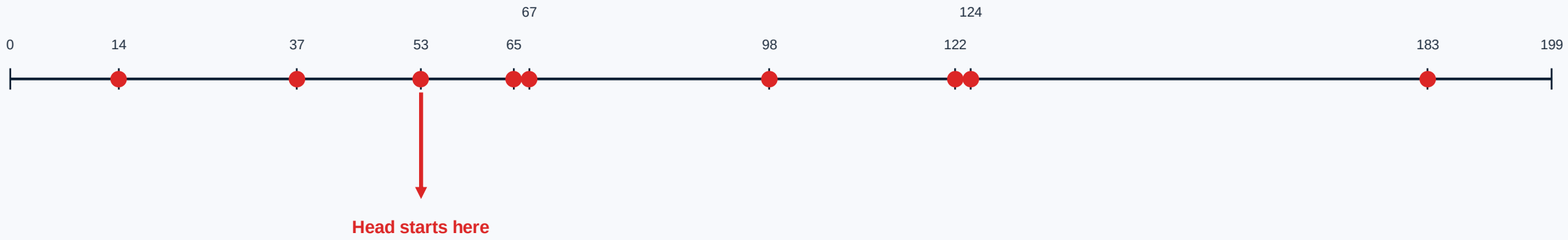
All algorithms will use the same request queue so students can compare them.

Cylinders: 0–199

Initial head: 53

Request queue

98, 183, 37, 122, 14, 124, 65, 67



Teaching method: first ask students to predict the next request, then reveal the algorithm decision.

Detailed Algorithm Walkthroughs

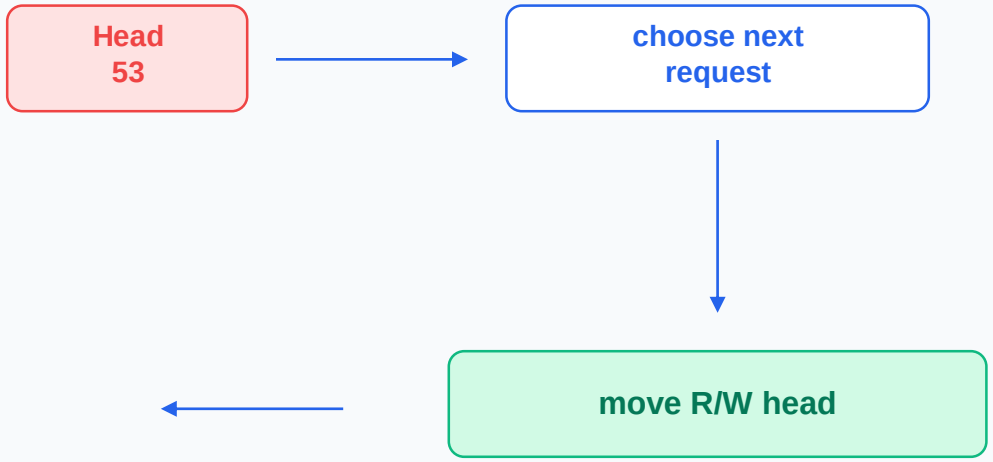
Same queue. Same initial head. Different service policies.

FCFS: First-Come, First-Served

Rule

Serve requests exactly in arrival order. Do not reorder the queue.

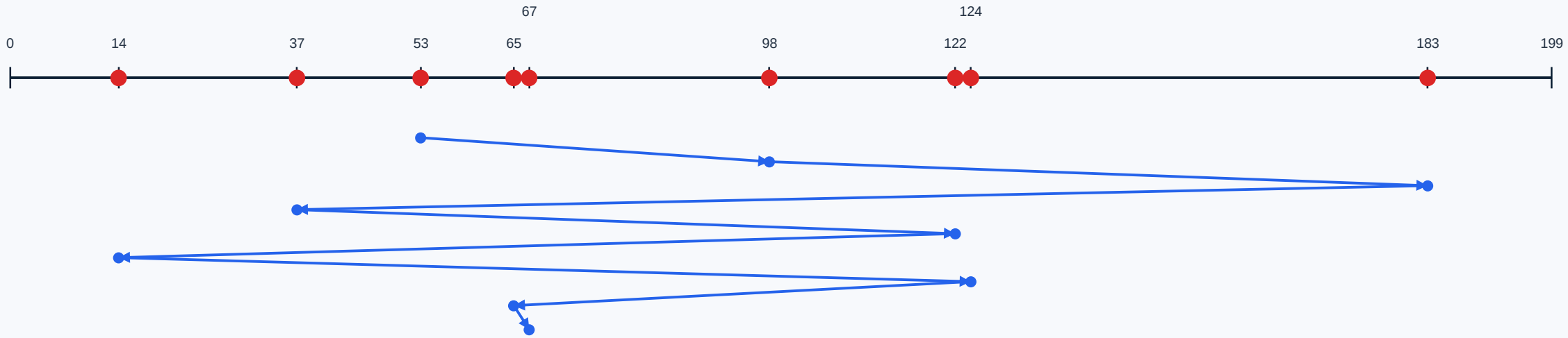
Fair by arrival order, but the head can jump a lot.



Think: which request is served next?

FCFS: First-Come, First-Served

Serve disk requests exactly in arrival order. Simple, fair, but often inefficient.



Service order

53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67

Total movement = |98−53| + |183−98| + ... + |67−65| = 640 cylinders

FCFS trace: movement after every request

Calculate total movement by adding every absolute distance from one head position to the next.

Step	R/W head movement	Δ
1.	53 → 98	45
2.	98 → 183	85
3.	183 → 37	146
4.	37 → 122	85

Step	R/W head movement	Δ
5.	122 → 14	108
6.	14 → 124	110
7.	124 → 65	59
8.	65 → 67	2

Total head movement = 45 + 85 + 146 + 85 + 108 + 110 + 59 + 2 = 640 cylinders

FCFS: advantages, disadvantages & best use

ADVANTAGES

Very simple; almost no scheduling overhead
Fair by arrival order; no starvation
Good baseline for comparison

DISADVANTAGES

Can cause very large head movement
Poor average seek time under mixed requests
Convoy effect from distant requests

BEST USE / WHEN TO USE

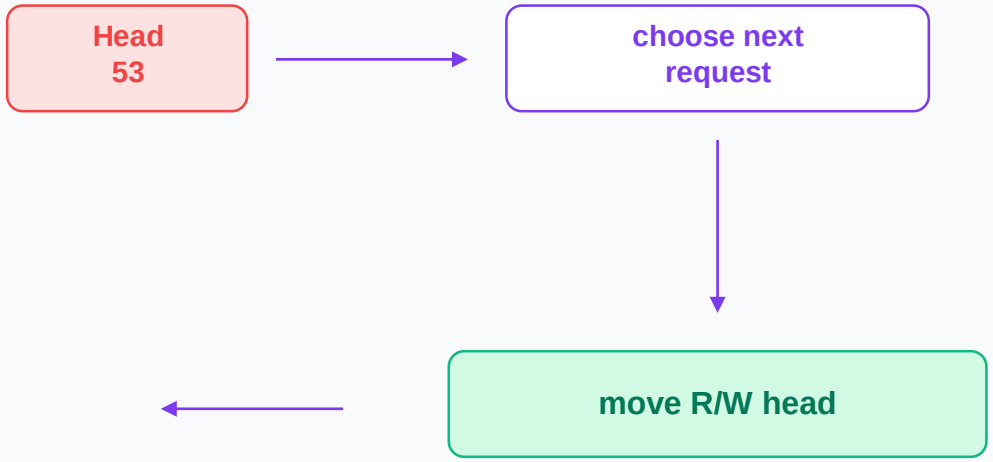
Light disk workloads with short queues
When strict arrival-order fairness matters
Useful as a reference/baseline

SSTF: Shortest Seek Time First

Rule

At each step, choose the pending request closest to the current head.

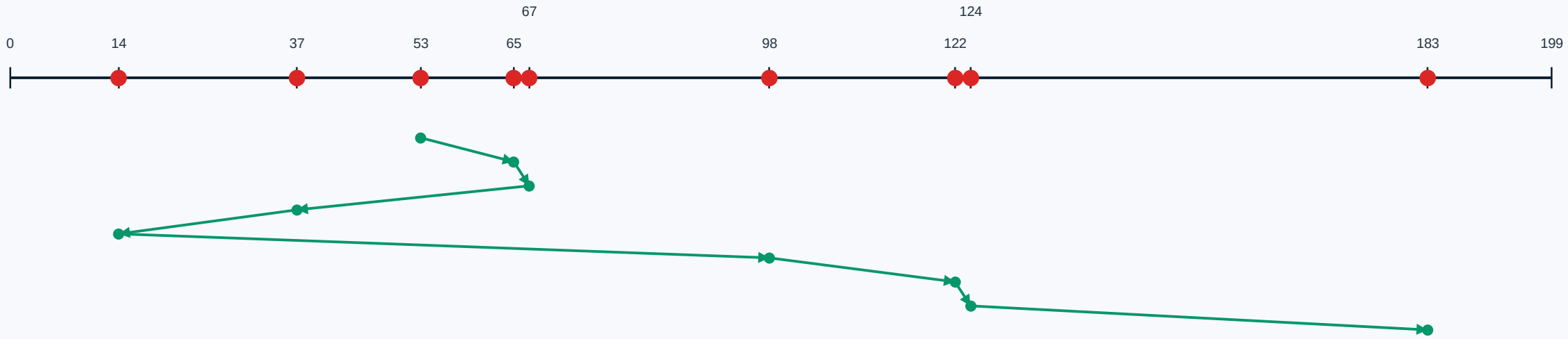
Greedy local optimization of seek distance.



Think: which request is served next?

SSTF: Shortest Seek Time First

At each step, select the pending request closest to the current head.



Service order

53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183

Total movement = 236 cylinders

Limitation: far-away requests may starve if nearby requests keep arriving.

SSTF trace: movement after every request

Calculate total movement by adding every absolute distance from one head position to the next.

Step	R/W head movement	Δ
1.	53 → 65	12
2.	65 → 67	2
3.	67 → 37	30
4.	37 → 14	23

Step	R/W head movement	Δ
5.	14 → 98	84
6.	98 → 122	24
7.	122 → 124	2
8.	124 → 183	59

Total head movement = 12 + 2 + 30 + 23 + 84 + 24 + 2 + 59 = 236 cylinders

SSTF: advantages, disadvantages & best use

ADVANTAGES

Usually lower movement than FCFS
Good local responsiveness
Simple greedy rule

DISADVANTAGES

Starvation possible for far requests
Fairness depends on request distribution
Head may stay around dense request region

BEST USE / WHEN TO USE

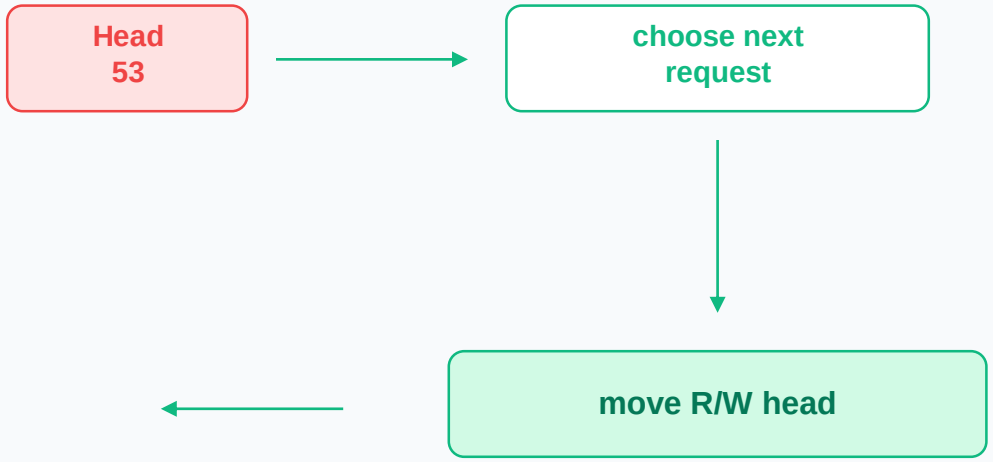
Moderate workloads where throughput matters
Requests reasonably spread
Avoid when bounded waiting is critical

SCAN: Elevator Algorithm

Rule

Move in one direction, service requests on the way, reach the physical end, then reverse.

Predictable sweep pattern, like an elevator.

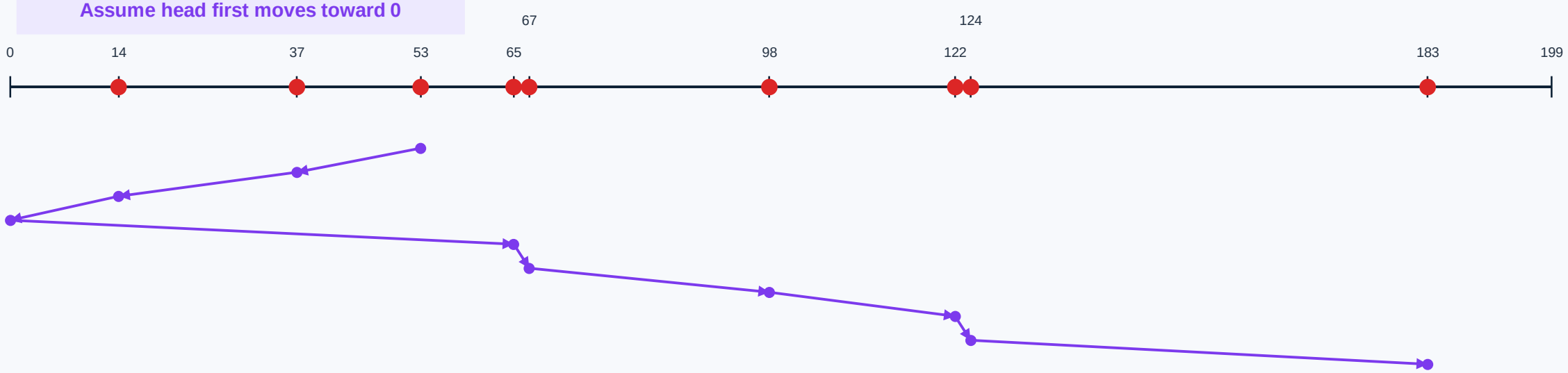


Think: which request is served next?

SCAN: the elevator algorithm

Move in one direction, serve requests on the way, then reverse at the end.

Assume head first moves toward 0



Service idea

53 → 37 → 14 → 0 → 65 → 67 → 98 → 122 → 124 → 183

Movement here = 236 tracks

Real-life analogy: elevator serves floors while moving upward/downward.

SCAN trace: movement after every request

Calculate total movement by adding every absolute distance from one head position to the next.

Step	R/W head movement	Δ	
1.	53 → 37	16	service 37
2.	37 → 14	23	service 14
3.	14 → 0	14	physical end
4.	0 → 65	65	
5.	65 → 67	2	

Step	R/W head movement	Δ
6.	67 → 98	31
7.	98 → 122	24
8.	122 → 124	2
9.	124 → 183	59

Total head movement = 16 + 23 + 14 + 65 + 2 + 31 + 24 + 2 + 59 = 236 cylinders

SCAN: advantages, disadvantages & best use

ADVANTAGES

Good throughput under heavy loads
Lower starvation risk than SSTF
Predictable sweeping behavior

DISADVANTAGES

May travel to physical end with no request
Requests just behind head may wait
Direction choice affects waiting time

BEST USE / WHEN TO USE

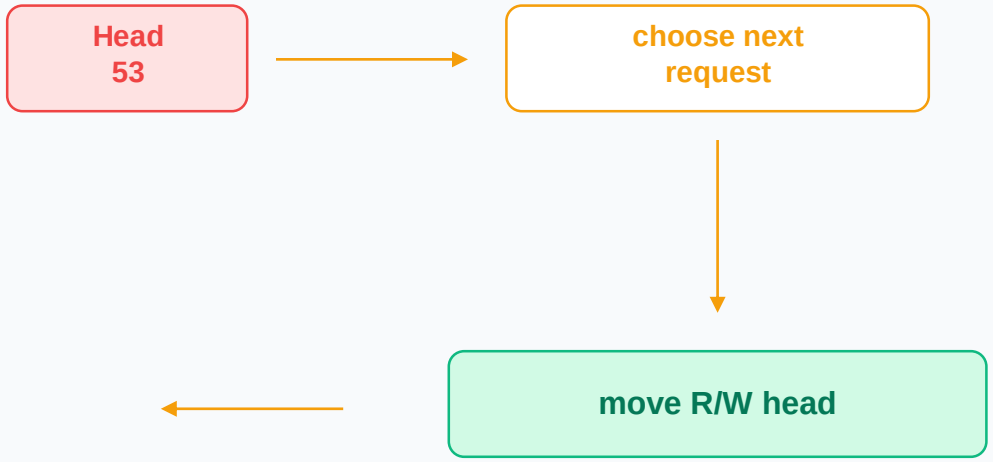
Busy HDDs with many pending requests
When throughput and bounded waiting matter
Good general-purpose sweep policy

C-SCAN: Circular SCAN

Rule

Service requests in one direction only. At the physical end, wrap to the opposite end without service.

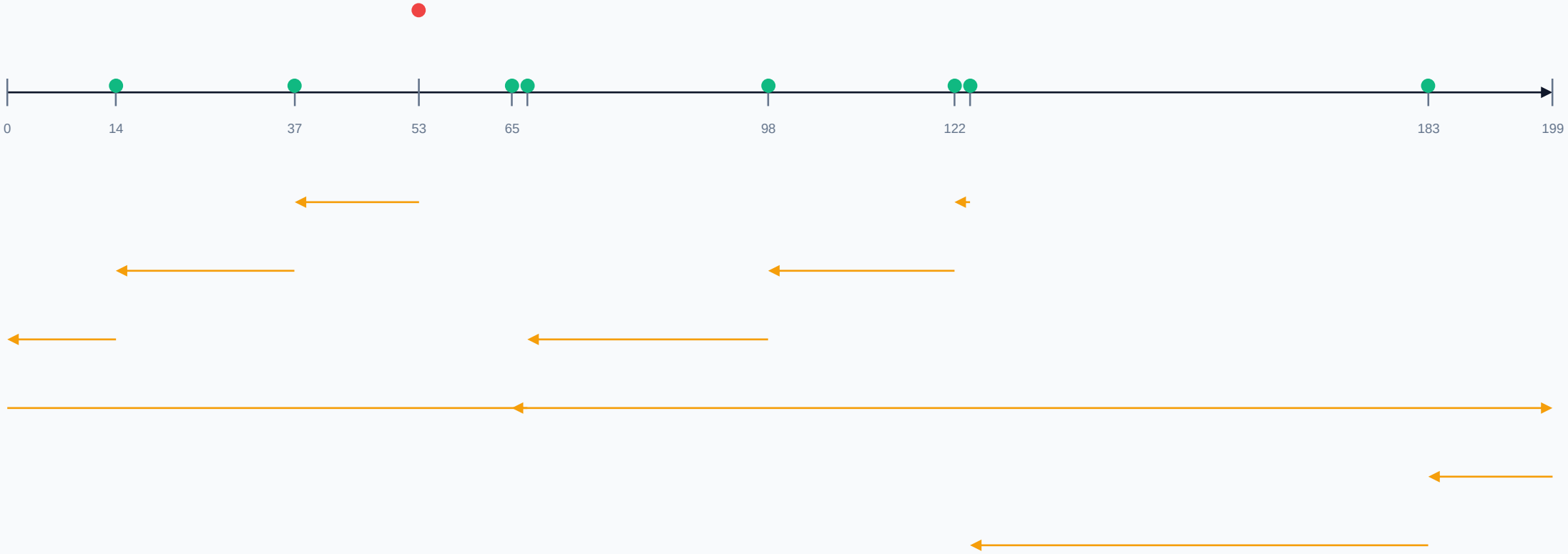
Treats cylinders like a circular queue for more uniform wait.



Think: which request is served next?

C-SCAN simulation: visual R/W-head path

Queue: 98, 183, 37, 122, 14, 124, 65, 67 start Initial head = 53



Service path: 53 → 37 → 14 → 0 → 199 → 183 → 124 → 122 → 98 → 67 → 65

Total movement = 386 cylinders

C-SCAN trace: movement after every request

Calculate total movement by adding every absolute distance from one head position to the next.

Step	R/W head movement	Δ	
1.	53 → 37	16	service
2.	37 → 14	23	service
3.	14 → 0	14	end
4.	0 → 199	199	wrap
5.	199 → 183	16	

Step	R/W head movement	Δ
6.	183 → 124	59
7.	124 → 122	2
8.	122 → 98	24
9.	98 → 67	31
10.	67 → 65	2

Total head movement = 16 + 23 + 14 + 199 + 16 + 59 + 2 + 24 + 31 + 2 = 386 cylinders

C-SCAN: advantages, disadvantages & best use

ADVANTAGES

More uniform waiting time than SCAN
Fair one-direction service pattern
Starvation unlikely in normal workload

DISADVANTAGES

Circular return can be expensive
More movement than LOOK/C-LOOK
No requests serviced during wrap

BEST USE / WHEN TO USE

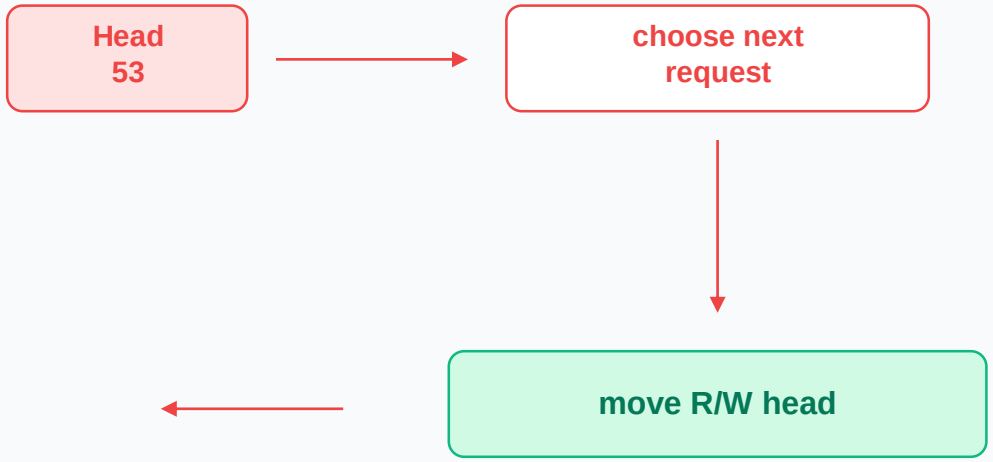
Heavy multi-user workloads
When uniform response matters
Fairness consistency over minimum movement

LOOK: SCAN without empty-end travel

Rule

Move like SCAN, but reverse at the last pending request instead of the physical end.

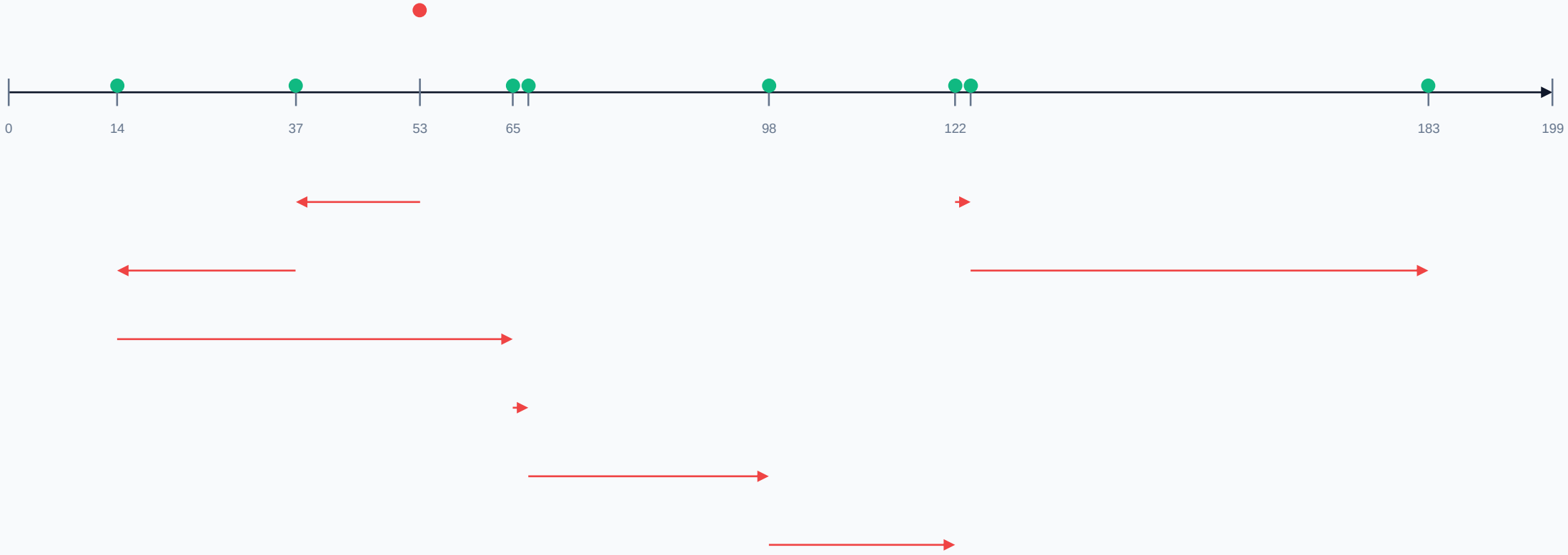
Same sweep idea, but avoids unnecessary end travel.



Think: which request is served next?

LOOK simulation: visual R/W-head path

Queue: 98, 183, 37, 122, 14, 124, 65, 67 start Initial head = 53



Service path: 53 → 37 → 14 → 65 → 67 → 98 → 122 → 124 → 183

Total movement = 208 cylinders

LOOK trace: movement after every request

Calculate total movement by adding every absolute distance from one head position to the next.

Step	R/W head movement	Δ	
1.	53 → 37	16	
2.	37 → 14	23	reverse
3.	14 → 65	51	
4.	65 → 67	2	

Step	R/W head movement	Δ
5.	67 → 98	31
6.	98 → 122	24
7.	122 → 124	2
8.	124 → 183	59

Total head movement = 16 + 23 + 51 + 2 + 31 + 24 + 2 + 59 = 208 cylinders

LOOK: advantages, disadvantages & best use

ADVANTAGES

Avoids SCAN’s empty physical-end travel
Often reduces total movement
Keeps sweep behavior and low starvation risk

DISADVANTAGES

Requests behind direction still wait
Waiting time less uniform than circular variants
Depends on direction and distribution

BEST USE / WHEN TO USE

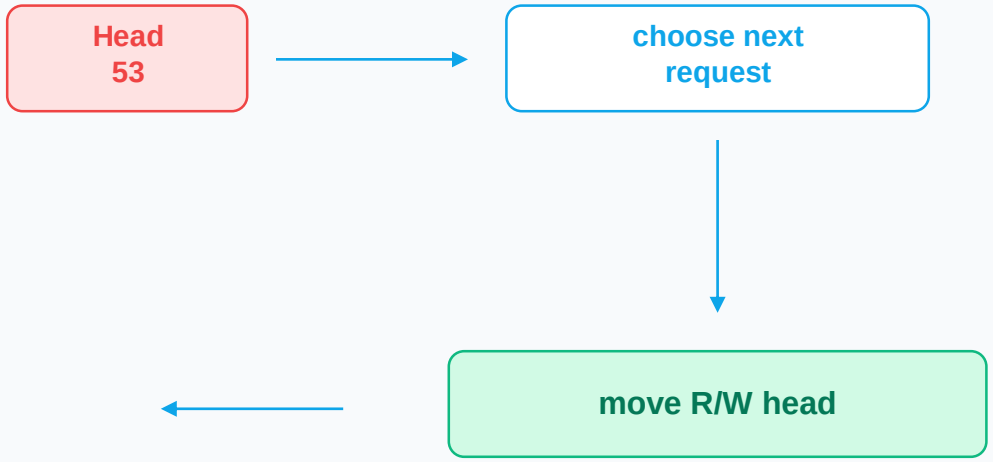
General HDD workloads clustered away from ends
When SCAN-like fairness is desired
Often practical improvement over SCAN

C-LOOK: Circular LOOK

Rule

Service in one direction only. Jump from the last request on one side to the farthest request on the other side.

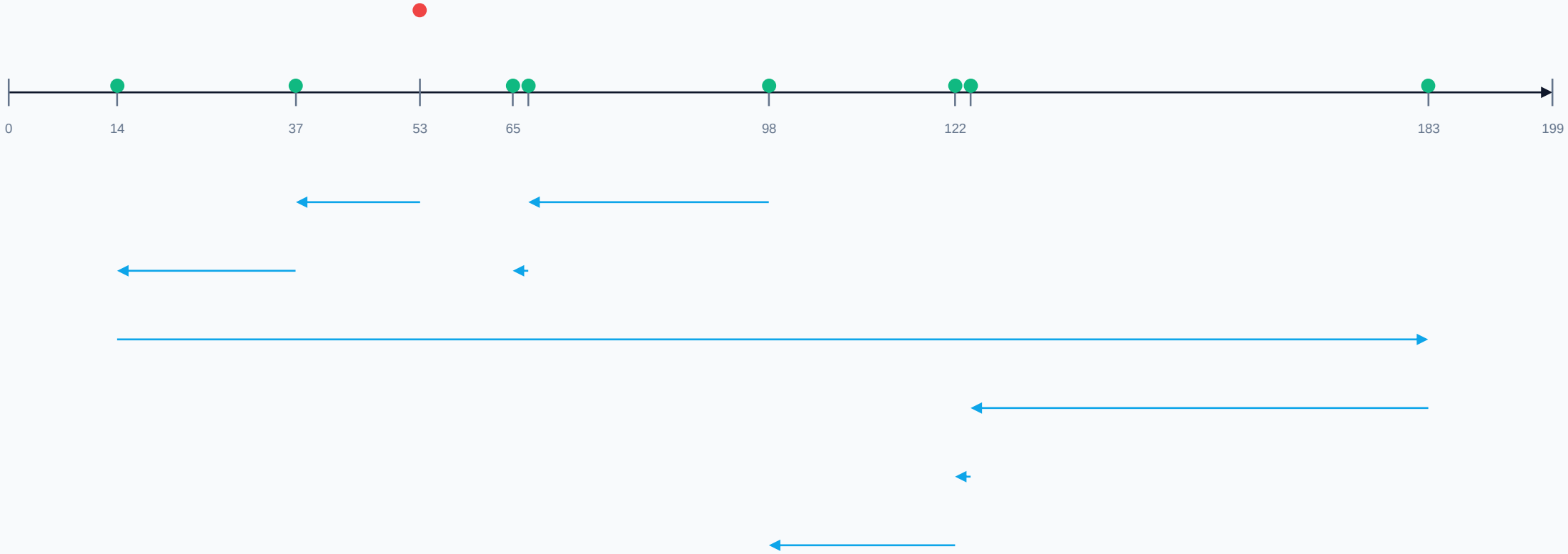
Circular fairness without physical-end travel.



Think: which request is served next?

C-LOOK simulation: visual R/W-head path

Queue: 98, 183, 37, 122, 14, 124, 65, 67 start Initial head = 53



Service path: 53 → 37 → 14 → 183 → 124 → 122 → 98 → 67 → 65

Total movement = 326 cylinders

C-LOOK trace: movement after every request

Calculate total movement by adding every absolute distance from one head position to the next.

Step	R/W head movement	Δ	
1.	53 → 37	16	
2.	37 → 14	23	last low
3.	14 → 183	169	jump
4.	183 → 124	59	

Step	R/W head movement	Δ
5.	124 → 122	2
6.	122 → 98	24
7.	98 → 67	31
8.	67 → 65	2

Total head movement = 16 + 23 + 169 + 59 + 2 + 24 + 31 + 2 = 326 cylinders

C-LOOK: advantages, disadvantages & best use

ADVANTAGES

More uniform circular waiting than LOOK
Avoids unused cylinder 0/199 travel
Often less movement than C-SCAN for sparse queues

DISADVANTAGES

Circular jump can still be long
A just-missed request may wait almost a cycle
More logic than FCFS/SCAN

BEST USE / WHEN TO USE

Heavy workloads needing uniform waiting
Requests occupy only part of disk range
When C-SCAN fairness is wanted without end travel

Same queue, different algorithms: comparison

Common queue: 98, 183, 37, 122, 14, 124, 65, 67 Head = 53 Direction = toward 0 first

Algorithm	Movement	Key trade-off
FCFS	640 cylinders	strict arrival-order fairness
SSTF	236 cylinders	low movement but starvation risk
SCAN	236 cylinders	balanced sweep, may touch physical end
C-SCAN	386 cylinders	uniform wait, expensive wrap
LOOK	**208 cylinders**	avoids empty end travel
C-LOOK	326 cylinders	circular fairness without end travel

For this specific queue, LOOK has the smallest movement. This does not make it universally best.

HDD Architecture Numericals

Capacity, cylinders, tracks, sectors and access-time calculation.

Track vs cylinder

This distinction often creates mistakes in exams.

Track

A concentric circular recording path on one disk surface.

Cylinder

Tracks at the same radius across all usable surfaces. Head can switch surface without radial movement.

Example

6 usable surfaces × 1024 tracks/surface → total tracks = 6144, but cylinders = 1024.

Exam rule

Cylinders = tracks per surface, not total tracks.

HDD architecture: exam-ready formulas

Usable surfaces / heads = platters × usable surfaces per platter

Cylinders = tracks per surface

Total tracks = usable surfaces × tracks per surface

Total sectors = total tracks × sectors per track

Disk capacity = total sectors × bytes per sector

For byte-addressable examples:
Address bits = $\log_2$(capacity in bytes)
If capacity is not a power of two: use $\lceil \log_2(\text{capacity in bytes}) \rceil$

Numerical 1: capacity, cylinders, tracks & address bits

An HDD has:
- 4 platters
- both surfaces of every platter are usable
- 1024 tracks per surface
- 256 sectors per track
- 512 bytes per sector

Find:
1) usable surfaces / heads
2) number of cylinders
3) total number of tracks
4) total number of sectors
5) total disk capacity
6) bits needed for byte addressing

Solution 1: architecture counting

Usable surfaces = $4 \times 2 = 8$
Cylinders = tracks per surface = 1024
Total tracks = $8 \times 1024 = 8192 = 2^{13}$

Total sectors = 8192×256
$= 2^{13} \times 2^{8} = 2^{21}$ sectors

Capacity = $2^{21} \times 512$ B
$= 2^{21} \times 2^{9}$ B $= 2^{30}$ B = 1 GiB

Byte-address bits = $\log_2(2^{30}) = 30$ bits

Key distinction: 1024 cylinders, but 8192 total tracks.

Numerical 2: a small architecture twist

An HDD contains:
• 6 platters
• normally 2 surfaces per platter
• 2 surfaces are reserved and cannot store user data
• 2048 tracks per usable surface
• 128 sectors per track
• 1 KiB per sector

Find:
1) usable surfaces / heads
2) cylinders
3) total tracks
4) total capacity in GiB
5) minimum bits needed to byte-address the whole disk

Solution 2: account for unusable surfaces

Potential surfaces = 6 × 2 = 12
Usable surfaces = 12 − 2 = 10
Cylinders = 2048
Total tracks = 10 × 2048 = 20,480

Total sectors = 20,480 × 128 = 2,621,440 sectors

Capacity = 2,621,440 × 1024 B
= 2,684,354,560 B = 2.5 GiB

Address bits = $\lceil \log_2(2{,}684{,}354{,}560) \rceil$ = 32 bits

Twist: cylinders depend on tracks per surface, not on usable-surface count.

Numerical 3: average HDD access time

A magnetic disk has:
- speed = 7200 RPM
- average seek time = 8 ms
- controller overhead = 0.2 ms
- transfer size = 64 KiB
- transfer rate = 128 MiB/s
- ignore queueing time

Find:
1) time for one full rotation
2) average rotational latency
3) transfer time
4) total average access time

Solution 3: seek + rotation + transfer

7200 RPM = 7200 / 60 = 120 rotations/s

One rotation time = 1 / 120 s = 8.333 ms

Average rotational latency = 8.333 / 2 = 4.167 ms

Transfer time = 64 KiB / 128 MiB/s
= 1 / 2048 s ≈ 0.488 ms

Total average access time
= Seek + Rotational latency + Transfer + Controller
= 8 + 4.167 + 0.488 + 0.2
≈ 12.855 ms

Students Outcome

Use this as the closing checklist.

Architecture

Explain platter → surface/head → track → sector → data, and distinguish track from cylinder.

Numerics

Calculate heads, cylinders, tracks, sectors, capacity, address bits and average access time.

Scheduling

Trace R/W-head movement for FCFS, SSTF, SCAN, C-SCAN, LOOK and C-LOOK.

Judgment

Compare fairness, starvation risk, end travel, circular waiting and total head movement.